



Escuela
Politécnica
Superior

Plataforma descentralizada para la validación y difusión de información veraz mediante tecnología blockchain



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:

Jorge Rafael de Julián Vicedo

Tutor:

Dr. Higinio Mora Mora

Julio 2026



Universitat d'Alacant
Universidad de Alicante

Plataforma descentralizada para la validación y difusión de información veraz mediante tecnología blockchain

NewsEra

Autor

Jorge Rafael de Julián Vicedo

Tutor

Dr. Higinio Mora Mora
Tecnología Informática y Computación



Grado en Ingeniería Informática



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

ALICANTE, Julio 2026

*A mi familia, a mis amigos, a María, a Fito y a Yrsa.
Gracias por estar siempre.*

Agradecimientos

Quiero expresar mi más sincero agradecimiento al Dr. Higinio Mora Mora, mi tutor, por su orientación, su paciencia y por guiarme con criterio en cada etapa de este trabajo.

A mi familia, por el apoyo incondicional de siempre y por confiar en mí incluso cuando yo dudaba.

A mis amigos y a María, por estar ahí en los momentos difíciles y en los buenos también, que hacen que todo tenga más sentido.

Y a Fito y a Yrsa, con todo el cariño.

Acceso abierto

Se autoriza a la Universidad de Alicante a depositar este Trabajo Fin de Grado en el Repositorio Institucional de la UA (RUA) y a difundirlo en acceso abierto, de conformidad con la normativa vigente sobre propiedad intelectual y el mandato de acceso abierto de la institución.

Resumen

El ecosistema informativo contemporáneo está sometido a una captura estructural por parte de grupos con intereses particulares. Los medios de comunicación —tanto públicos como privados— subordinan su línea editorial a sus propietarios, anunciantes y fuentes gubernamentales, mientras que los algoritmos de las grandes plataformas digitales amplifican o suprimen contenidos sin transparencia ni rendición de cuentas democrática. El ciudadano ordinario carece de infraestructura técnica para verificar de forma autónoma la autenticidad, la autoría y el historial de modificación de cualquier pieza informativa. Las soluciones existentes —agencias de *fact-checking*, moderación algorítmica— no resuelven el problema estructural: son ellas mismas entidades centralizadas con sus propias dependencias y sus propios marcos interpretativos.

Este TFG propone NewsEra, una plataforma descentralizada para la validación y difusión de información veraz mediante tecnología *blockchain*. La premisa del sistema es que la resistencia estructural frente a la captura se logra codificando las reglas de publicación, verificación y gobernanza en contratos inteligentes desplegados en una *blockchain* pública, donde ningún actor puede modificarlas unilateralmente. El sistema articula tres mecanismos complementarios: publicación abierta e inmutable, en la que cualquier ciudadano registra contenido asociado a un hash criptográfico que garantiza su integridad; validación comunitaria con responsabilidad pública, mediante la que validadores acreditados emiten juicios de veracidad que quedan permanentemente registrados en cadena; y un mecanismo de reputación y gobernanza distribuida —componente central del trabajo— que determina qué participantes pueden validar y con qué peso, calculando esa reputación a partir del historial verificable de cada usuario sin interferencia de actores externos.

La implementación sigue una metodología ágil Scrum adaptada a un proyecto individual, organizada en nueve sprints de dos semanas. El stack tecnológico se estructura en tres capas: la capa *blockchain* utiliza Solidity, Hardhat, OpenZeppelin y la red de prueba Ethereum Sepolia; la capa de *backend* emplea Node.js, TypeScript, Hono, Prisma ORM y PostgreSQL; el almacenamiento descentralizado del contenido de los artículos se realiza mediante IPFS a través de Pinata; y la interfaz de usuario se construye con React, Vite, Tailwind CSS, shadcn/ui, wagmi v2, viem y RainbowKit. De forma complementaria, el trabajo contempla la integración opcional de herramientas de inteligencia artificial para el análisis automático de contenido —detección de duplicados, identificación de sesgos y análisis de imágenes manipuladas— como apoyo al criterio humano de los validadores.

Palabras clave: blockchain, desinformación, gobernanza distribuida, contratos inteligentes, reputación descentralizada, verificación de contenidos, Web3

Abstract

The contemporary information ecosystem is subject to structural capture by groups with particular interests. Both public and private media outlets subordinate their editorial lines to their owners, advertisers and government sources, while the algorithms of major digital platforms amplify or suppress content without transparency or democratic accountability. Ordinary citizens lack the technical infrastructure to autonomously verify the authenticity, authorship and modification history of any piece of information. Existing solutions —fact-checking agencies, algorithmic moderation— fail to address this structural problem: they are themselves centralised entities with their own funding dependencies and interpretive frameworks.

This bachelor’s thesis proposes NewsEra, a decentralised platform for the validation and dissemination of truthful information using blockchain technology. The system’s premise is that structural resistance to capture is achieved by encoding the rules governing publication, verification and governance in smart contracts deployed on a public blockchain, where no single actor can unilaterally modify them. The system articulates three complementary mechanisms: open and immutable publication, whereby any citizen registers content associated with a cryptographic hash that guarantees its integrity; community validation with public accountability, through which accredited validators issue veracity judgements that are permanently recorded on-chain; and a distributed reputation and governance mechanism —the core component of the work— that determines which participants may validate and with what weight, computing that reputation from each user’s verifiable on-chain history without external interference.

The implementation follows an agile Scrum methodology adapted to an individual project, organised into nine two-week sprints. The technology stack is structured in three layers: the blockchain layer uses Solidity, Hardhat, OpenZeppelin and the Ethereum Sepolia test network; the backend layer employs Node.js, TypeScript, Hono, Prisma ORM and PostgreSQL; article content is stored in a decentralised manner using IPFS via Pinata; and the user interface is built with React, Vite, Tailwind CSS, shadcn/ui, wagmi v2, viem and RainbowKit. As a complementary feature, the work explores the optional integration of artificial intelligence tools for automated content analysis —duplicate detection, linguistic bias identification and manipulated image analysis— as support for the human judgement of validators.

Keywords: blockchain, disinformation, distributed governance, smart contracts, decentralised reputation, content verification, Web3

Índice general

Agradecimientos	v
Acceso abierto	vii
Resumen	ix
Abstract	xi
1. Introducción	1
1.1. Motivación	1
1.2. Planteamiento del problema	2
1.3. Propuesta y enfoque	2
1.4. Objetivo general	3
1.5. Objetivos específicos	3
1.6. Metodología	5
1.7. Estructura del documento	5
2. Marco Teórico	7
2.1. Información, poder y democracia	7
2.2. Tecnología Blockchain	8
2.2.1. Fundamentos criptográficos y estructura de datos	8
2.2.2. Bitcoin: el libro mayor distribuido	8
2.2.3. Ethereum y los contratos inteligentes	9
2.2.4. Tipos de redes blockchain	9
2.3. Contratos inteligentes y Solidity	9
2.3.1. Concepto y propiedades	9
2.3.2. Solidity: lenguaje de programación	10
2.4. Gobernanza distribuida y sistemas de reputación	10
2.4.1. Organizaciones Autónomas Descentralizadas (DAOs)	10
2.4.2. Mecanismos de votación	11
2.4.3. Sistemas de reputación en redes distribuidas	11
2.4.4. Mecanismos de incentivo y teoría de juegos	12
2.5. Identidad descentralizada	12
2.6. Almacenamiento descentralizado: IPFS	12
2.7. Inteligencia Artificial para el análisis de contenido (opcional)	13
2.7.1. Detección de desinformación mediante NLP	13
2.7.2. Detección de contenido multimedia manipulado	14
2.7.3. Detección de duplicados y sesgos	14
2.8. Síntesis: bases tecnológicas de NewsEra	14

3. Metodología	17
3.1. Enfoque metodológico general	17
3.2. Scrum adaptado al proyecto individual	17
3.2.1. Adaptación de roles	17
3.2.2. Adaptación de ceremonias	18
3.2.3. Artefactos	18
3.3. Planificación en fases y sprints	18
3.4. Principios de desarrollo	20
3.5. Stack tecnológico	21
3.5.1. Capa blockchain: red y framework de desarrollo	21
3.5.2. Capa de backend	22
3.5.3. Capa de frontend	22
3.5.4. Herramientas de desarrollo y control de versiones	23
3.6. Estrategia de validación	23
3.6.1. Pruebas unitarias de contratos inteligentes	24
3.6.2. Pruebas de escenario del mecanismo de gobernanza	24
3.6.3. Análisis de coste de gas	24
3.6.4. Evaluación de viabilidad del sistema	24
3.6.5. Experimentos de validación con usuarios	25
3.7. Gestión de riesgos	26
4. Diseño del sistema	29
4.1. Especificación de requisitos	29
4.1.1. Casos de uso	29
4.1.1.1. FEAT 1. Gestión de cuenta	29
4.1.1.2. FEAT 2. Publicar artículo	30
4.1.1.3. FEAT 3. Explorar contenido	31
4.1.1.4. FEAT 4. Verificar artículo	31
4.1.1.5. FEAT 5. Información institucional	32
4.1.2. Casos de uso detallados	32
4.1.3. Requisitos no funcionales	32
4.1.4. Requisitos de datos	34
4.1.5. Requisitos de interfaz	36
4.2. Visión general de la arquitectura	37
4.2.1. Capa on-chain: la fuente de verdad	37
4.2.2. Capa off-chain: eficiencia y consultabilidad	37
4.2.3. Capa de presentación: la interfaz Web3	38
4.2.4. El hash criptográfico como vínculo entre capas	38
4.2.5. Esquema global del sistema	40
4.2.6. Distribución de datos entre capas	40
4.3. Diseño de los contratos inteligentes	42
4.3.1. PublicationRegistry	42
4.3.2. ValidationRegistry	43
4.3.3. ReputationSystem	48
4.3.4. Tabla de funciones principales de los contratos	49

4.4. Modelo de datos	51
4.4.1. Almacenamiento on-chain	51
4.4.2. Esquema off-chain (PostgreSQL / Prisma)	52
4.4.3. Sincronización on-chain → off-chain	54
4.5. Diseño de la API REST	55
4.5.1. Arquitectura de capas del backend	55
4.5.2. Endpoints de la API	55
4.6. Diseño de la interfaz de usuario	57
4.6.1. Identidad visual	57
4.6.2. Estructura de la aplicación React	59
4.6.3. Flujos de interacción principales	59
4.7. Flujo de publicación y validación	60
5. Resultados	63
5.1. Resumen del prototipo implementado	63
5.2. Resultados de los contratos inteligentes	63
5.2.1. Despliegue en Sepolia	63
5.2.2. Análisis de coste de gas	64
5.2.3. Cobertura de tests	64
5.3. Métricas de implementación	64
5.4. Evaluación del cumplimiento de objetivos	65
5.5. Resultados de los experimentos de validación con usuarios	66
5.5.1. Experimento 1: intensidad del problema	66
5.5.2. Experimento 2: idoneidad del producto	66
5.6. Consideraciones de diseño: evolución del sistema	67
5.6.1. De abstención a veredicto: el tipo UNVERIFIABLE	67
5.6.2. De única ronda a sistema multironda con reapertura	67
5.6.3. De penalización simple a reputación retroactiva	68
5.6.4. De voto ponderado a voto plano con acceso gated	68
5.7. Discusión de resultados	69
5.7.1. Viabilidad técnica del mecanismo de reputación	69
5.7.2. Limitaciones observadas durante la implementación	69
6. Conclusiones	71
6.1. Conclusiones principales	71
6.2. Contribuciones del trabajo	73
6.3. Limitaciones	74
6.4. Trabajo futuro	75
6.5. Valoración personal	78
Bibliografía	79
Lista de Acrónimos y Abreviaturas	83
Glosario de Términos	85

A. Catálogo de casos de uso detallados	89
A.1. FEAT 1. Gestión de cuenta	89
A.2. FEAT 2. Publicar artículo	93
A.3. FEAT 3. Explorar contenido	96
A.4. FEAT 4. Verificar artículo	99
A.5. FEAT 5. Información institucional	103
B. Instrumentos de los experimentos de validación	105
B.1. Encuesta 1 — Experimento 1: intensidad del problema	105
B.2. Página explicativa	106
B.3. Encuesta 2 — Experimento 2: idoneidad del producto	106
B.3.1. Bloque A: evaluación de la usabilidad (System Usability Scale)	106
B.3.2. Bloque B: confianza en las propiedades de la aplicación	107
B.4. Datos piloto sintéticos y su exclusión del análisis	108
B.5. Recomendaciones de muestra	108
C. NewsEra explicado para un público no técnico	109
C.1. El problema: un punto de control es un punto de fallo	109
C.2. La solución: un periódico que no tiene redacción	109
C.3. ¿Qué es “verdad” aquí?	110
C.4. Los pilares	110
C.5. Lo nuevo: no mejora al árbitro, cambia el juego	110
C.6. La tecnología: un cuaderno que nadie puede tachar	110
C.7. En una frase	110
D. Guía de reproducibilidad y despliegue local	111
D.1. Requisitos previos	111
D.2. Puesta en marcha en un solo paso	111
D.3. Comandos habituales	112
D.4. Variables de entorno	113

Índice de figuras

4.1.	Mockup de las vistas principales de la interfaz de NewsEra. La interfaz permite a cualquier usuario publicar contenido y consultar validaciones; para emitir votos se requiere conectar una cartera MetaMask y tener reputación mínima acreditada en el contrato ReputationSystem	39
4.2.	Arquitectura global del sistema NewsEra: capas y flujos de interacción. El contenido de los artículos se almacena en IPFS; los registros de publicación, validación y reputación, en los contratos inteligentes; los metadatos e índices, en PostgreSQL; la interfaz de usuario interactúa con las tres capas.	40
4.3.	Máquina de estados de ValidationRegistry . El estado PENDING_REOPEN es transitorio: se activa internamente cuando se alcanza reopenThreshold solicitudes y se resuelve de forma inmediata al abrir la nueva ronda.	44
4.4.	Logotipo de NewsEra: monograma NE con marca de verificación integrada, generado con Gemini a partir del <i>prompt</i> de generación de imagen que se reproduce a continuación.	58

Índice de tablas

1.1.	Trazabilidad entre objetivos específicos, componentes del sistema y carácter en el alcance del trabajo.	4
2.1.	Correspondencia entre componentes de NewsEra y fundamentos del estado del arte.	15
3.1.	Planificación de <i>sprints</i> y objetivos asociados.	20
3.2.	Registro de riesgos del proyecto.	27
4.1.	Funcionalidades de alto nivel (FEAT) de STQR 1 y sus casos de uso. . . .	30
4.2.	Distribución de datos entre las capas del sistema NewsEra.	41
4.3.	Efectos reputacionales completos en ValidationRegistry . Los efectos de ronda propia se aplican al cerrar cada ronda; los retroactivos, al llamar a claimRetroactiveReputation (máximo ± 3 acumulados).	45
4.4.	Funciones principales de los contratos inteligentes de NewsEra.	50
4.5.	Paleta de color de la identidad visual de NewsEra.	58
4.6.	Actores, contratos y eventos en el flujo de publicación y validación. . . .	61
5.1.	Direcciones de despliegue previstas en Sepolia (pendiente Sprint 9). . . .	63
5.2.	Coste de gas de las operaciones principales de los contratos.	64
5.3.	Líneas de código por capa del sistema (Sprint 4).	65
5.4.	Grado de cumplimiento de los objetivos específicos del trabajo.	65
D.1.	Comandos de make más habituales.	112

Índice de Códigos

4.1. solidity	51
4.2. text	52
4.3. text	53
4.4. text	54
D.1. bash	111

1. Introducción

La información es el sustrato sobre el que se construye el conocimiento colectivo de una sociedad. Su libre circulación y su veracidad son condiciones necesarias para el ejercicio de la democracia, la toma de decisiones informadas y la cohesión social. Sin embargo, el ecosistema digital contemporáneo ha transformado profundamente la forma en que se produce, distribuye y consume información, generando condiciones que favorecen la propagación masiva de contenido falso o manipulado a una velocidad y escala sin precedentes históricos. Este trabajo aborda ese problema desde una perspectiva tecnológica y propone una solución basada en la descentralización, la criptografía y la gobernanza comunitaria.

1.1. Motivación

La información no es un bien neutral. En las sociedades contemporáneas, el control sobre qué información circula, cómo se enmarca y a quién llega constituye una forma de poder político y económico de primer orden. La teoría del espacio público elaborada por Habermas (1989) sostiene que una democracia requiere un ámbito de deliberación libre e igualitario donde los ciudadanos puedan formarse una opinión sobre los asuntos comunes. Sin embargo, los sistemas de distribución de información que estructuran ese espacio público —los medios de comunicación, tanto públicos como privados— están sometidos a presiones estructurales que los alejan de esa función democrática.

Herman y Chomsky (1988) documentaron, en su modelo de propaganda, cómo los grandes medios de comunicación operan como instituciones que producen y distribuyen información subordinada a los intereses de sus propietarios, anunciantes y fuentes gubernamentales. Esta captura estructural no requiere de conspiraciones explícitas: se ejerce a través de la propiedad concentrada, la dependencia publicitaria, la autocensura y los mecanismos de acceso a las fuentes de poder (McChesney, 2008). El resultado es un ecosistema informativo en el que la agenda pública —qué problemas se visibilizan, qué marcos interpretativos se aplican, qué voces tienen acceso al debate— refleja sistemáticamente los intereses de grupos con capacidad para influir sobre esas instituciones.

La digitalización no ha resuelto este problema; en muchos aspectos, lo ha agravado. La concentración de la distribución de información en un reducido número de plataformas tecnológicas ha añadido una nueva capa de intermediación opaca, cuyos algoritmos determinan qué contenidos se amplifican y cuáles se suprimen sin ningún mecanismo de rendición de cuentas democrática (Pariser, 2011). Al mismo tiempo, la baja barrera de entrada para la publicación digital ha generado un ecosistema ruidoso en el que distinguir información veraz de contenido manipulado se ha convertido en una tarea cognitivamente exigente para el ciudadano ordinario (Wardle & Derakhshan, 2017).

La tecnología *blockchain* ofrece, por primera vez, una infraestructura técnica capaz de sostener un sistema de información que sea estructuralmente resistente a la captura por cualquier actor individual —económico, político o institucional—, porque las reglas que gobiernan qué

se publica, quién valida y cómo se calcula la reputación están codificadas en contratos inteligentes que ningún actor puede modificar unilateralmente (Buterin, 2014). Este trabajo explora esa posibilidad.

1.2. Planteamiento del problema

El problema que este trabajo aborda no es la existencia de noticias falsas como fenómeno aislado, sino la **captura estructural de los sistemas de distribución de información** por parte de grupos con intereses particulares. Esta captura opera en tres niveles complementarios:

En primer lugar, la **dependencia estructural de los medios frente al poder económico y político**. Los medios de comunicación, tanto públicos como privados, operan dentro de estructuras de propiedad y financiación que condicionan su línea editorial. Los medios públicos están expuestos a la presión de los gobiernos de turno; los privados, a la de sus accionistas y anunciantes. En ambos casos, la información se convierte en un instrumento al servicio de intereses particulares antes que en un bien común al servicio de la ciudadanía (Herman & Chomsky, 1988; McChesney, 2008). El ciudadano no tiene acceso a los mecanismos que determinan qué se publica, qué se silencia y con qué encuadre se presenta la realidad.

En segundo lugar, la **opacidad e irresponsabilidad de los sistemas de distribución digital**. Las plataformas tecnológicas que median la circulación de información a escala masiva —redes sociales, agregadores de contenidos, motores de búsqueda— toman decisiones algorítmicas sobre qué contenidos se amplifican y cuáles se suprimen sin ningún mecanismo de transparencia ni rendición de cuentas democrática (Pariser, 2011). Estas decisiones tienen consecuencias políticas directas, pero escapan al escrutinio público y no están sujetas a ninguna forma de gobernanza colectiva.

En tercer lugar, la **ausencia de infraestructura ciudadana para la verificación autónoma**. El ciudadano ordinario carece de herramientas accesibles y técnicamente fiables para verificar de forma independiente la autenticidad, la autoría y el historial de modificación de un contenido informativo. No existe un registro público, inmutable y auditable de qué se publicó, quién lo publicó y si ha sido alterado desde entonces, lo que hace imposible exigir responsabilidades a los actores que manipulan la información (Hasan & Salah, 2019).

1.3. Propuesta y enfoque

NewsEra no pretende mejorar los sistemas de distribución de información existentes ni añadir una capa de verificación sobre los medios tradicionales. Su propuesta es más radical: **construir una infraestructura alternativa de publicación y validación de información que sea estructuralmente resistente a la captura por cualquier grupo de poder**, ya sea económico, político o institucional.

La clave de esta resistencia estructural reside en que las reglas que gobiernan el sistema no están en manos de ninguna entidad —no hay una empresa que pueda cambiar el algoritmo, ni un gobierno que pueda exigir la eliminación de contenidos, ni un consejo editorial que pueda imponer una línea—. Esas reglas están codificadas en contratos inteligentes desplegados en una *blockchain* pública, donde son visibles, auditables e inmutables para cualquier participante (Buterin, 2014).

Sobre esta base, el sistema articula tres mecanismos complementarios. En primer lugar, cualquier ciudadano puede publicar contenido informativo en la plataforma; ese contenido queda registrado con un hash criptográfico que garantiza su inmutabilidad y permite detectar cualquier alteración posterior (Nakamoto, 2008). En segundo lugar, una comunidad de validadores con reputación acreditada emite juicios sobre la veracidad del contenido; esos juicios quedan registrados permanentemente, haciendo responsables públicamente a quienes validan (Lesaege et al., 2019). En tercer lugar, un mecanismo de reputación y gobernanza —núcleo del trabajo— determina qué participantes tienen capacidad de validar y con qué peso, calculando esa reputación a partir del historial verificable de cada usuario, sin que ningún actor externo pueda interferir en ese cálculo.

El resultado es un espacio de información donde la credibilidad no la otorga ningún medio, ningún gobierno ni ninguna plataforma, sino la participación colectiva y trazable de la propia ciudadanía. De forma complementaria, el proyecto contempla la integración opcional de herramientas de Inteligencia Artificial (IA) para el análisis automático de contenido —detección de duplicados, identificación de sesgos o análisis de imágenes manipuladas— como apoyo al criterio humano de los validadores (Zhou & Zafarani, 2020).

1.4. Objetivo general

El objetivo general del presente TFG es **diseñar e implementar un prototipo funcional de NewsEra**: una plataforma descentralizada para la validación y difusión de información veraz mediante tecnología *blockchain* y mecanismos de gobernanza distribuida, en la que la confianza no dependa de entidades centrales sino de la participación colectiva y trazable de los usuarios. Con ello se pretende demostrar la viabilidad técnica y social de aplicar esta tecnología a la resolución del problema de la captura estructural de los sistemas de información.

1.5. Objetivos específicos

Para alcanzar el objetivo general, se definen los siguientes objetivos específicos:

1. **OE1 — Análisis del estado del arte.** Revisar la literatura científica y técnica existente en materia de desinformación digital, sistemas *blockchain* para la verificación de contenidos, mecanismos de gobernanza distribuida y sistemas de reputación en redes P2P, con el fin de identificar las soluciones previas y las brechas que justifican la propuesta de NewsEra.
 2. **OE2 — Diseño de la arquitectura del sistema.** Definir la arquitectura de la plataforma, especificando los componentes de la capa *blockchain* (contratos inteligentes, modelo de datos en cadena), la capa de backend *off-chain* y la interfaz de usuario, así como los flujos de interacción entre publicadores, validadores y la red.
 3. **OE3 — Implementación del sistema descentralizado de publicación y verificación de contenidos.** Desarrollar en Solidity los contratos inteligentes que soporten el registro de publicaciones, el proceso de verificación por parte de validadores y la
-

consulta del historial de validaciones, garantizando la trazabilidad e inmutabilidad de todos los registros en la *blockchain*.

4. **OE4 — Diseño e implementación del mecanismo de reputación y gobernanza para validadores.** Diseñar e implementar el núcleo del sistema: el mecanismo distribuido que calcula y actualiza la reputación de cada validador en función de su historial de participación, que regula el acceso al rol de validador y que codifica las reglas de gobernanza de la comunidad en contratos inteligentes auditables y resistentes a la manipulación.
5. **OE5 — Desarrollo del prototipo de interfaz de usuario.** Construir una interfaz web funcional que permita a los participantes publicar contenido informativo, emitir y consultar verificaciones, visualizar el estado de reputación de los validadores y explorar el historial de validaciones almacenado en la *blockchain*.
6. **OE6 — Integración opcional de herramientas de Inteligencia Artificial.** Estudiar e integrar, como capa de análisis complementaria al criterio humano, herramientas de IA para la detección automática de contenido duplicado, la identificación de sesgos lingüísticos o el análisis de imágenes manipuladas, evaluando su aportación al proceso de verificación comunitaria.
7. **OE7 — Evaluación de la viabilidad del prototipo.** Analizar el prototipo resultante desde una perspectiva técnica y social, verificando que el sistema cumple los principios de descentralización, trazabilidad y confianza sin dependencia de entidades centrales, e identificando las limitaciones actuales y las líneas de trabajo futuro.

La Tabla 1.1 resume la correspondencia entre cada objetivo específico, los componentes del sistema que lo sustentan y su carácter obligatorio u opcional en el alcance del presente trabajo.

Tabla 1.1: Trazabilidad entre objetivos específicos, componentes del sistema y carácter en el alcance del trabajo.

Obj.	Descripción resumida	Componente principal	Carácter
OE1	Estado del arte	Revisión bibliográfica	Obligatorio
OE2	Arquitectura del sistema	Diseño global (blockchain + backend + UI)	Obligatorio
OE3	Publicación y verificación descentralizada	Contratos inteligentes (Solidity)	Obligatorio
OE4	Reputación y gobernanza de validadores	Contratos inteligentes (núcleo del sistema)	Obligatorio
OE5	Prototipo de interfaz de usuario	Frontend web	Obligatorio
OE6	Integración de herramientas de IA	Capa de análisis complementaria	Opcional
OE7	Evaluación de viabilidad	Prototipo completo	Obligatorio

1.6. Metodología

El trabajo sigue una metodología estructurada en tres fases secuenciales. La primera fase consiste en la elaboración de un estado del arte exhaustivo sobre las tecnologías y mecanismos implicados: desinformación digital, *blockchain* y contratos inteligentes, sistemas de identidad descentralizada, mecanismos de gobernanza distribuida y técnicas de IA para el análisis de contenido. A partir de este análisis se identifican las brechas que motivan la propuesta y se extraen los principios de diseño del sistema. La segunda fase comprende el diseño de la arquitectura y la implementación del prototipo funcional, priorizando el mecanismo de reputación y gobernanza como componente central. La tercera fase evalúa el prototipo resultante en términos de descentralización, trazabilidad y cumplimiento de los objetivos planteados, identificando asimismo las limitaciones actuales y las líneas de trabajo futuro.

1.7. Estructura del documento

El presente documento se organiza en seis capítulos cuyo contenido se describe a continuación.

El **Capítulo 1** —el presente— contextualiza el problema de la desinformación digital, motiva la propuesta de NewsEra, formula los objetivos del trabajo y describe la estructura del documento.

El **Capítulo 2** desarrolla el estado del arte en los tres ámbitos tecnológicos sobre los que se sustenta el sistema: la tecnología *blockchain* y los contratos inteligentes, los sistemas de identidad y gobernanza descentralizadas, y las técnicas de IA aplicadas a la detección de desinformación.

El **Capítulo 3** describe la metodología de desarrollo empleada, las herramientas y tecnologías seleccionadas para cada capa del sistema, y la planificación temporal del trabajo.

El **Capítulo 4** presenta el diseño e implementación del sistema NewsEra: la arquitectura global, los contratos inteligentes de publicación y verificación, el mecanismo de reputación y gobernanza de validadores, la capa de backend y el prototipo de interfaz de usuario.

El **Capítulo 5** recoge los resultados obtenidos del prototipo funcional, analiza el comportamiento del mecanismo de gobernanza en distintos escenarios de uso y evalúa el cumplimiento de los objetivos planteados.

El **Capítulo 6** sintetiza las principales aportaciones del trabajo, expone las limitaciones identificadas durante el desarrollo y propone las líneas de investigación y desarrollo futuro que permitirían evolucionar NewsEra hacia un sistema de producción.

2. Marco Teórico

Este capítulo desarrolla los fundamentos teóricos y tecnológicos sobre los que se sustenta el diseño de NewsEra. Se revisan, en primer lugar, los enfoques académicos que conceptualizan la relación entre información, poder y democracia. A continuación se examinan las tecnologías que componen la arquitectura del sistema: la *blockchain* y los contratos inteligentes, los mecanismos de gobernanza distribuida y sistemas de reputación, y la identidad descentralizada. El capítulo concluye con una revisión de las técnicas de Inteligencia Artificial (IA) aplicadas al análisis de contenido informativo, componente de carácter opcional en el sistema.

2.1. Información, poder y democracia

La noción de que los medios de comunicación sirven a la democracia proporcionando a los ciudadanos información veraz e independiente tiene un fundamento normativo bien establecido en la teoría política. La esfera pública conceptualizada por Habermas (1989) es el espacio de deliberación racional donde los ciudadanos forman opinión sobre los asuntos comunes, condición que presupone acceso equitativo a información libre de distorsiones estratégicas. Esta concepción normativa contrasta, sin embargo, con el funcionamiento real de los sistemas mediáticos contemporáneos.

Herman y Chomsky (1988) desarrollaron el modelo de propaganda para describir cómo los grandes medios de comunicación, lejos de actuar como contrapoder, operan como instituciones al servicio de las élites económicas y políticas. Cinco filtros estructurales —la propiedad concentrada, la dependencia publicitaria, el acceso privilegiado a las fuentes gubernamentales, la presión externa organizada y la ideología dominante— condicionan sistemáticamente qué información se produce, cómo se enmarca y qué voces tienen acceso al espacio público. Este análisis, ampliado por McChesney (2008) para el contexto de la concentración digital, evidencia que la desinformación no es solo el resultado de actores maliciosos externos, sino en gran medida un producto estructural de los propios sistemas de distribución de información.

La digitalización ha añadido una dimensión algorítmica a esta captura estructural. Pariser (2011) demostró que los algoritmos de personalización de contenido crean burbujas informativas que refuerzan las creencias previas del usuario, reducen la exposición a perspectivas diversas y facilitan la manipulación de la agenda pública por parte de quienes controlan esos algoritmos. Es en este contexto donde la propuesta de una infraestructura informativa descentralizada adquiere su dimensión política: no como mejora técnica de los sistemas existentes, sino como alternativa estructural a su captura.

2.2. Tecnología Blockchain

2.2.1. Fundamentos criptográficos y estructura de datos

Una *blockchain* es un registro distribuido, ordenado cronológicamente e inmutable de transacciones, mantenido de forma colectiva por una red de nodos sin necesidad de una autoridad central de coordinación (Nakamoto, 2008). Su diseño combina tres primitivas criptográficas fundamentales: las funciones hash, los árboles de Merkle y la criptografía de clave pública y privada.

Una función hash criptográfica —como SHA-256— transforma una entrada de longitud arbitraria en una salida de longitud fija (el *digest*) de forma determinista y prácticamente irreversible: cualquier modificación mínima en la entrada produce un hash completamente diferente (*efecto avalancha*). Esta propiedad es la base de la inmutabilidad de la *blockchain*: cada bloque contiene el hash del bloque anterior, formando una cadena en la que alterar un bloque invalida todos los posteriores.

Los **árboles de Merkle** (Merkle Trees) son estructuras de datos jerárquicas en las que cada nodo hoja contiene el hash de una transacción, y cada nodo interno contiene el hash de la concatenación de sus nodos hijos (Nakamoto, 2008). La raíz del árbol (Merkle Root) resume criptográficamente el conjunto completo de transacciones de un bloque, lo que permite verificar la pertenencia de una transacción concreta al bloque en tiempo logarítmico sin necesidad de descargar el bloque completo.

La **criptografía asimétrica** (par clave pública/privada) permite a cada participante firmar digitalmente sus transacciones con su clave privada. Cualquier nodo puede verificar esa firma con la clave pública correspondiente, sin que sea necesario revelar la clave privada ni recurrir a una autoridad certificadora.

2.2.2. Bitcoin: el libro mayor distribuido

El protocolo Bitcoin, descrito por Nakamoto (2008), resolvió el problema del doble gasto en un entorno sin confianza mediante la combinación de una cadena de bloques encadenados criptográficamente con un mecanismo de consenso basado en prueba de trabajo (*Proof of Work*, PoW). En PoW, los nodos compiten por encontrar un valor entero arbitrario denominado *nonce* (*number used once*) que, incluido en la cabecera del bloque, produce un hash del bloque que cumple un objetivo de dificultad: dicho hash debe ser numéricamente inferior a un umbral prefijado, lo que en la práctica equivale a exigir que comience por un número mínimo de ceros. Como el hash es impredecible, la única estrategia viable es el ensayo sistemático de millones de valores de *nonce* por segundo. El umbral de dificultad se ajusta periódicamente para que la red produzca un nuevo bloque cada diez minutos aproximadamente, con independencia de la potencia de cómputo total disponible. El nodo que encuentra primero un *nonce* válido difunde el bloque a la red, que lo verifica y lo incorpora a la cadena.

El consenso en Bitcoin es probabilístico: una transacción se considera definitiva cuando está enterrada bajo suficientes bloques posteriores como para que revertirla requiera más poder de cómputo que el que controla honestamente el resto de la red (regla del 51 %). Este diseño garantiza la inmutabilidad del registro sin necesidad de ninguna entidad central, pero tiene un elevado coste energético y una capacidad de procesamiento limitada.

2.2.3. Ethereum y los contratos inteligentes

Ethereum, propuesto por Buterin (2014), extiende el modelo de Bitcoin incorporando una máquina virtual de propósito general —la Ethereum Virtual Machine (EVM)— que permite ejecutar código arbitrario en la *blockchain*. Esto hace posible los **contratos inteligentes**: programas cuyo código y estado se almacenan en la cadena, se ejecutan de forma determinista en todos los nodos y no pueden ser alterados una vez desplegados sin el consenso de la red. El resultado es que las reglas de un sistema —en el caso de NewsEra, las reglas de publicación, verificación y gobernanza— pueden codificarse en contratos que ningún actor puede modificar unilateralmente.

Ethereum también resolvió parte de las limitaciones energéticas de PoW mediante la transición a *Proof of Stake* (PoS) en 2022 (conocida como *The Merge*) (Ethereum Foundation, 2022). En PoS, los validadores deben comprometer (*stake*) una cantidad de criptomoneda como garantía; si intentan validar bloques fraudulentos, pierden esa garantía (*slashing*). Este mecanismo mantiene la seguridad de la red con un consumo energético radicalmente inferior al de PoW.

2.2.4. Tipos de redes blockchain

Las redes *blockchain* se clasifican según su modelo de acceso (Ding et al., 2023):

- **Pública** (*permissionless*): cualquier nodo puede unirse, leer y escribir en la cadena. Ofrece máxima descentralización y resistencia a la censura, pero menor rendimiento (Ethereum, Bitcoin).
- **Privada**: acceso restringido a un conjunto predefinido de participantes. Mayor rendimiento, pero dependiente de una entidad central de control (Hyperledger Fabric).
- **Consorcio**: un grupo de organizaciones comparte el control de la red. Equilibrio entre rendimiento y descentralización parcial.

Para NewsEra, una red pública es la opción coherente con el objetivo de resistencia estructural a la captura: ningún actor puede controlar quién participa ni qué reglas se aplican.

2.3. Contratos inteligentes y Solidity

2.3.1. Concepto y propiedades

Un contrato inteligente (*smart contract*) es un programa almacenado en la *blockchain* que se ejecuta automáticamente cuando se cumplen las condiciones codificadas en su lógica, sin intervención humana y sin posibilidad de interferencia externa una vez desplegado (Buterin, 2014). Sus propiedades fundamentales son la **inmutabilidad** del código desplegado, la **transparencia** (cualquiera puede auditar el código), la **autonomía** (se ejecuta sin depender de ningún tercero) y el **determinismo** (dado el mismo estado y los mismos parámetros, produce siempre el mismo resultado en todos los nodos de la red).

Estas propiedades hacen de los contratos inteligentes el mecanismo idóneo para implementar las reglas de gobernanza de NewsEra: las condiciones bajo las que un usuario puede

convertirse en validador, el algoritmo de cálculo de reputación y el proceso de registro de publicaciones y verificaciones son reglas públicas, auditables y que ningún actor puede alterar de forma unilateral.

2.3.2. Solidity: lenguaje de programación

Solidity es el lenguaje de programación orientado a objetos más utilizado para el desarrollo de contratos inteligentes en la EVM. Con una sintaxis influida por C++, Python y JavaScript, Solidity compila a *bytecode* ejecutable por la EVM y es el estándar *de facto* para el ecosistema Ethereum.

Dado que el código de un contrato inteligente es inmutable una vez desplegado, las vulnerabilidades de seguridad tienen consecuencias especialmente graves: los fondos o datos protegidos por un contrato defectuoso pueden quedar expuestos de forma permanente. Los ataques más frecuentes incluyen la reentrancia (*reentrancy*), el desbordamiento aritmético (*integer overflow/underflow*) y la dependencia de valores predecibles como la marca de tiempo del bloque (Wang et al., 2023). El desarrollo de contratos inteligentes seguros requiere, por tanto, un ciclo de diseño que incluya revisión formal del código, pruebas exhaustivas y, cuando sea posible, verificación formal.

Una característica fundamental de Solidity para la arquitectura de sistemas como NewsEra es el mecanismo de **eventos** (**event/emit**): los contratos pueden emitir eventos que quedan registrados en el log de transacciones de Ethereum y son indexables por sistemas externos sin incurrir en el coste de almacenamiento en estado persistente. Este mecanismo es la base de la sincronización entre la capa *on-chain* y los sistemas *off-chain*: un servicio externo se suscribe a los eventos del contrato para mantener una réplica actualizada del estado sin interrogar la cadena en cada consulta.

2.4. Gobernanza distribuida y sistemas de reputación

2.4.1. Organizaciones Autónomas Descentralizadas (DAOs)

Una Decentralized Autonomous Organization (DAO) es una organización cuyas reglas de funcionamiento —quién puede participar, cómo se toman las decisiones, cómo se gestionan los recursos colectivos— están codificadas en contratos inteligentes en una *blockchain* pública (Ding et al., 2023). La ausencia de dirección central y la automatización de las reglas de gobernanza hacen de las DAOs la arquitectura de referencia para implementar sistemas comunitarios resistentes a la captura por actores individuales.

Las DAOs han demostrado su viabilidad en contextos muy diversos: desde la gestión de protocolos financieros descentralizados (DeFi) hasta la coordinación de proyectos de código abierto y la resolución descentralizada de disputas. Sin embargo, la investigación empírica ha identificado también limitaciones estructurales relevantes: Feichtinger et al. (2023), en un estudio de 21 DAOs activas, encontraron alta concentración de poder de voto, elevados costes ocultos de participación en la gobernanza *on-chain* y una proporción significativa de actividad de votación sin efecto real. Messias et al. (2023) confirmaron que en las DAOs de Compound y Uniswap, tan solo entre 3 y 5 votantes eran suficientes para determinar el resultado de la mayoría de las propuestas.

Estas limitaciones evidencian que el diseño del mecanismo de gobernanza es crítico: la mera descentralización técnica no garantiza una distribución equitativa del poder de decisión. El diseño de NewsEra debe tener en cuenta estas dinámicas para evitar que el sistema reproduzca las asimetrías que pretende combatir.

2.4.2. Mecanismos de votación

Los mecanismos de votación *on-chain* más empleados en las DAOs actuales son (Ding et al., 2023):

- **Un token, un voto** (*token-weighted voting*): el poder de voto es proporcional a los tokens que posee cada participante. Incentiva la participación de grandes tenedores pero reproduce la concentración de poder económico.
- **Un participante, un voto** (*one-person-one-vote*): requiere mecanismos de verificación de identidad para evitar ataques Sybil (Douceur, 2002) —en los que una entidad crea múltiples identidades falsas para obtener influencia desproporcionada sobre el sistema—.
- **Votación cuadrática** (*quadratic voting*): cada participante dispone de un presupuesto fijo de créditos de voto; emitir n votos sobre una propuesta cuesta n^2 créditos. El coste creciente desincentiva el acaparamiento del poder de voto, mientras que el presupuesto compartido permite expresar preferencias intensas: gastar más créditos en lo que más importa a costa de perder influencia en otras decisiones (Weyl & Posner, 2019).
- **Gobernanza delegada** (*liquid democracy*): los participantes pueden delegar su voto en representantes de confianza, combinando la participación directa con la representación.

2.4.3. Sistemas de reputación en redes distribuidas

Un sistema de reputación asigna a cada participante de una red una puntuación que refleja la calidad de su comportamiento histórico, con el objetivo de alinear los incentivos individuales con el bienestar colectivo (Adler & de Alfaro, 2007). En redes distribuidas, el sistema de reputación sustituye a la autoridad central como mecanismo de confianza: un participante con alta reputación es de fiar no porque lo avale ninguna institución, sino porque su historial verificable así lo demuestra.

Los desafíos técnicos de los sistemas de reputación distribuidos incluyen la resistencia a ataques Sybil —en los que un actor crea múltiples identidades para inflar su propia reputación—, el problema del *whitewashing* —abandonar una identidad con mala reputación y empezar de nuevo con una nueva— y la agregación justa de valoraciones de fuentes con distinto nivel de confianza (Feichtinger et al., 2023).

Adler y de Alfaro (2007) propusieron, en el contexto de Wikipedia, un sistema de reputación de contenidos basado en el análisis de la supervivencia de las ediciones: el texto que persiste en el tiempo sin ser revertido por otros editores es indicativo de calidad, y los editores cuyas contribuciones perduran acumulan reputación. Este principio —la reputación como función del historial verificable de contribuciones— es directamente aplicable al contexto de validación comunitaria de contenidos de NewsEra.

2.4.4. Mecanismos de incentivo y teoría de juegos

El diseño de un sistema de gobernanza distribuida es, en esencia, un problema de diseño de mecanismos: definir las reglas del juego de forma que los participantes racionales y egoístas actúen de modo que produzca el resultado colectivamente deseable (Ding et al., 2023). En el contexto de NewsEra, el objetivo es que los validadores emitan juicios honestos sobre la veracidad del contenido.

Los mecanismos más empleados para alinear incentivos en este tipo de sistemas son la **penalización económica** (*slashing*): los validadores que emiten juicios maliciosos o inconsistentes con el consenso pierden parte de su garantía depositada; y la **reputación acumulativa**: los validadores cuyo historial de juicios es coherente con el consenso de la comunidad acumulan reputación, que les otorga mayor peso y visibilidad dentro del sistema. Kleros (Lesaege et al., 2019) combina ambos mecanismos en un sistema de resolución de disputas descentralizado en el que los jurados pierden su depósito si votan en minoría, incentivando la coordinación hacia el juicio honesto.

2.5. Identidad descentralizada

El problema de la identidad es central en cualquier sistema de reputación: si una entidad puede crear múltiples identidades sin coste (ataque Sybil), el sistema de reputación es trivialmente manipulable. La identidad descentralizada aborda este problema sin recurrir a una autoridad central de certificación.

Los Decentralized Identifier (DID) son identificadores únicos globales que un sujeto puede crear y controlar de forma autónoma, sin necesidad de que ninguna autoridad central los emita o los revoque (World Wide Web Consortium, 2022). Un DID se resuelve en un documento DID que contiene las claves criptográficas públicas del sujeto, lo que permite verificar de forma independiente que quien afirma controlar ese identificador es efectivamente quien posee la clave privada correspondiente.

Las Verifiable Credential (VC) son afirmaciones firmadas digitalmente por un emisor (por ejemplo, una universidad, un organismo oficial o la propia comunidad) sobre un sujeto identificado por su DID (Mazzocca et al., 2025). A diferencia de los certificados digitales tradicionales, las VCs permiten una divulgación selectiva: el titular puede demostrar que posee cierta credencial sin revelar la totalidad de la información que contiene, preservando su privacidad.

La adopción de DIDs y VCs como estándar del W3C ha impulsado el desarrollo de un ecosistema de identidad auto-soberana (*Self-Sovereign Identity*, SSI) en el que el usuario controla sus propios datos de identidad y decide con quién los comparte (Allen, 2016; Mazzocca et al., 2025). En NewsEra, este estándar ofrece una base para implementar el acceso al rol de validador vinculado a una identidad verificable y resistente a ataques Sybil, sin comprometer el anonimato de los participantes que no deseen revelar su identidad real.

2.6. Almacenamiento descentralizado: IPFS

El InterPlanetary File System (IPFS) es un protocolo de red entre iguales diseñado para el almacenamiento y la distribución descentralizada de contenido (Benet, 2014). A diferencia del modelo tradicional basado en localización (*location-based addressing*), en el que un

recurso se identifica por la dirección del servidor que lo aloja, IPFS utiliza *direccionamiento por contenido* (*content addressing*): cada objeto se identifica por el hash criptográfico de su contenido, lo que garantiza su integridad de forma verificable y sin dependencia de ningún servidor central.

Cuando un nodo IPFS publica un archivo, la red asigna a ese contenido un identificador inmutable denominado CID (*Content Identifier*). Cualquier nodo que solicite ese CID recibirá exactamente el mismo contenido, independientemente de quién lo sirva, y puede verificar su integridad computando el hash localmente. Esta propiedad hace de IPFS una solución idónea para sistemas que requieren publicación permanente y verificable de documentos sin depender de una infraestructura centralizada.

Para garantizar la persistencia del contenido en la red IPFS, es necesario que al menos un nodo lo mantenga *anclado* (*pinned*). Servicios de anclaje como Pinata actúan como infraestructura de respaldo: reciben el CID generado por el publicador y se comprometen a mantener el contenido accesible en la red, ofreciendo una API REST que simplifica la integración en aplicaciones *web*.

En NewsEra, IPFS/Pinata actúa como capa de almacenamiento de artículos: el contenido textual se publica en IPFS y únicamente su CID queda registrado en la cadena de bloques como *contentHash*. Este diseño minimiza el coste de gas —almacenar texto completo *on-chain* sería económicamente inviable— y mantiene la verificabilidad: cualquier usuario puede obtener el artículo desde IPFS y comprobar que su hash coincide con el registrado en el contrato.

2.7. Inteligencia Artificial para el análisis de contenido (opcional)

■ Carácter opcional de esta sección La integración de herramientas de IA para el análisis de contenido es un componente opcional del sistema NewsEra (objetivo OE6). Esta sección revisa el estado del arte para orientar su posible implementación futura, pero no es un requisito del prototipo descrito en este trabajo.

2.7.1. Detección de desinformación mediante NLP

La detección automática de desinformación mediante Natural Language Processing (NLP) es un campo de investigación activo que ha experimentado avances sustanciales desde la irrupción de los modelos de lenguaje preentrenados basados en la arquitectura *Transformer* (Zhou & Zafarani, 2020). Los enfoques previos —basados en características léxicas, estilométricas y de redes de propagación— han sido ampliamente superados por modelos como BERT, RoBERTa y sus derivados, que capturan representaciones contextuales profundas del texto y permiten clasificar el contenido con una precisión superior al 95 % en los conjuntos de evaluación estándar (Shu et al., 2017).

La tarea de detección de noticias falsas se formula habitualmente como un problema de clasificación binaria (verdadero/falso) o multiclase (según el grado de manipulación), entrenado sobre conjuntos de datos etiquetados como FakeNewsNet, LIAR o WELFake. Sin embargo, los modelos entrenados en un dominio concreto presentan dificultades de generalización a

otros dominios o idiomas, lo que limita su aplicabilidad directa en un sistema global como NewsEra (Zhou & Zafarani, 2020).

2.7.2. Detección de contenido multimedia manipulado

Más allá del análisis de texto, la desinformación moderna incorpora imágenes y vídeos manipulados mediante técnicas de *deep learning* (*deepfakes*). Los enfoques de detección más eficaces combinan redes convolucionales (CNN) para el análisis de artefactos de compresión y señales de manipulación en frecuencias espaciales con modelos de atención para la detección de inconsistencias temporales en secuencias de vídeo (Hasan & Salah, 2019).

2.7.3. Detección de duplicados y sesgos

La detección de contenido duplicado o parafraseado (*near-duplicate detection*) puede abordarse mediante técnicas de *embedding* semántico que proyectan los documentos en un espacio vectorial de alta dimensión, donde la similitud coseno mide el grado de solapamiento semántico independientemente de las diferencias superficiales de formulación. El análisis de sesgos lingüísticos —detección de lenguaje cargado emocionalmente, sensacionalismo o encuadres sesgados— se puede abordar mediante clasificadores entrenados sobre corpus anotados con métricas de objetividad periodística (Zhou & Zafarani, 2020).

En el contexto de NewsEra, estas herramientas actuarían como una capa de análisis previo que asiste al criterio humano de los validadores, proporcionando señales automáticas sobre la novedad del contenido, su alineación con patrones conocidos de manipulación y la presencia de elementos multimedia de autenticidad cuestionable. La decisión final de validación permanece siempre en manos de la comunidad.

2.8. Síntesis: bases tecnológicas de NewsEra

El recorrido realizado en este capítulo permite identificar las bases tecnológicas específicas sobre las que se apoya el diseño de NewsEra. La Tabla 2.1 resume la correspondencia entre cada componente del sistema y la tecnología o concepto del estado del arte que lo fundamenta.

Tabla 2.1: Correspondencia entre componentes de NewsEra y fundamentos del estado del arte.

Componente de NewsEra	Fundamentación teórica	Referencia
Registro inmutable de publicaciones	Cadena de bloques + hashes SHA-256 + árboles de Merkle	(Nakamoto, 2008)
Ejecución de reglas sin intermediarios	Contratos inteligentes en la EVM	(Buterin, 2014)
Gobernanza comunitaria	DAOs y mecanismos de votación <i>on-chain</i>	(Ding et al., 2023)
Reputación de validadores	Sistemas de reputación en redes P2P	(Adler & de Alfaro, 2007; Lesaege et al., 2019)
Resistencia a ataques Sybil	Reputación acumulativa <i>on-chain</i> (prototipo); identidad descentralizada DIDs/VCs W3C (solución de producción)	(Adler & de Alfaro, 2007); (World Wide Web Consortium, 2022)
Almacenamiento descentralizado de artículos	IPFS con direccionamiento por contenido (CID)	(Benet, 2014)
Análisis de contenido (opcional)	NLP con modelos <i>Transformer</i>	(Zhou & Zafarani, 2020)

3. Metodología

Este capítulo describe el enfoque metodológico adoptado para la realización del presente TFG. Se presenta el marco de desarrollo elegido, su adaptación al contexto de un proyecto individual académico, la planificación en fases, las hipótesis tecnológicas que guiarán la implementación y la estrategia de validación del prototipo.

3.1. Enfoque metodológico general

La metodología descrita en la propuesta del trabajo se articula en tres fases secuenciales: elaboración del estado del arte, diseño de la propuesta a partir de sus conclusiones, e implementación de un prototipo que demuestre la viabilidad del sistema (Onyekachi, 2019). Esta secuencia se corresponde con una aproximación iterativa e incremental al desarrollo de software, en la que cada fase retroalimenta a las siguientes y el conocimiento acumulado durante el estado del arte determina las decisiones de diseño e implementación.

Para organizar la fase de implementación de forma estructurada y adaptable, se adopta **Scrum** como marco de trabajo ágil (Schwaber & Sutherland, 2020). La elección de Scrum responde a tres razones principales: la naturaleza exploratoria del proyecto —en el que las decisiones tecnológicas concretas evolucionan a medida que avanza el conocimiento del dominio—, la posibilidad de entregar valor incremental con cada sprint —lo que facilita el seguimiento del progreso con el tutor—, y la alineación con los objetivos específicos del trabajo, que son independientes entre sí y pueden abordarse de forma progresiva.

3.2. Scrum adaptado al proyecto individual

Scrum fue concebido originalmente para equipos de desarrollo de entre tres y nueve personas (Schwaber & Sutherland, 2020). Su aplicación a un proyecto de desarrollo individual como un TFG requiere adaptar los roles y las ceremonias del marco original manteniendo sus principios esenciales: transparencia, inspección y adaptación.

3.2.1. Adaptación de roles

En el contexto de este trabajo, los tres roles de Scrum se distribuyen de la siguiente forma:

- **Product Owner:** el tutor del trabajo, responsable de definir y priorizar los requisitos del sistema, validar los incrementos al final de cada sprint y asegurar que el producto se alinea con los objetivos del TFG.
- **Scrum Master:** el propio estudiante, responsable de garantizar que el proceso se sigue correctamente, identificar impedimentos y buscar soluciones para mantener el ritmo de trabajo.

- **Desarrollador:** el propio estudiante, responsable de la implementación técnica de cada incremento.

3.2.2. Adaptación de ceremonias

Las ceremonias de Scrum se adaptan a la dinámica de un proyecto individual supervisado:

- ***Sprint Planning:*** al inicio de cada sprint, el estudiante selecciona del *product backlog* los elementos a desarrollar, los descompone en tareas concretas y define el objetivo del sprint.
- ***Daily Scrum:*** sustituido por un diario de desarrollo personal en el que se registran diariamente el progreso, los impedimentos encontrados y las decisiones técnicas tomadas. Este registro sirve de base para la memoria del trabajo.
- ***Sprint Review:*** reunión con el tutor al final de cada sprint en la que se presenta el incremento desarrollado, se valida su corrección y se ajusta el *backlog* si procede.
- ***Sprint Retrospective:*** reflexión personal al final de cada sprint sobre el proceso de trabajo, con el objetivo de identificar mejoras para el siguiente.

3.2.3. Artefactos

Los artefactos de Scrum utilizados en el proyecto son:

- ***Product Backlog:*** lista priorizada de todos los requisitos del sistema, derivada directamente de los objetivos específicos OE1–OE7 definidos en el Capítulo 1. El *backlog* se refinará continuamente a medida que el conocimiento del dominio evolucione durante el desarrollo.
- ***Sprint Backlog:*** subconjunto de elementos del *product backlog* seleccionados para cada sprint, con las tareas concretas asociadas a cada uno.
- **Incremento:** el resultado entregable al final de cada sprint: un componente funcional del sistema (un contrato inteligente, un módulo de la API o un componente de la interfaz) acompañado de sus pruebas automatizadas.

3.3. Planificación en fases y sprints

El proyecto se organiza en diez fases, cada una de las cuales corresponde a uno o más sprints de dos semanas de duración. La duración de cada sprint se establece en dos semanas, lo que permite revisiones frecuentes con el tutor sin fragmentar excesivamente el trabajo.

Sprint 0 — Fundamentos (completado): elaboración del estado del arte, definición de objetivos y justificación del trabajo. Esta fase corresponde a los capítulos 1 y 2 de la presente memoria.

- Sprint 1 — Diseño de la arquitectura:** definición detallada de la arquitectura del sistema (OE2): componentes de cada capa, interfaces de comunicación, modelo de datos *on-chain* y *off-chain*, y selección tecnológica definitiva para cada módulo.
- Sprints 2–3 — Contratos inteligentes de registro y verificación:** implementación de los contratos Solidity responsables del registro inmutable de publicaciones y del proceso de verificación por parte de los validadores (OE3). Incluye las pruebas unitarias de cada función sobre *Hardhat Network* (red local).
- Sprint 4 — Mecanismo de reputación:** implementación del contrato `ReputationSystem` (OE4): cálculo y actualización de reputación de validadores, control de acceso por roles (`OpenZeppelin AccessControl`) y barrera de acceso mínima como primera defensa contra ataques Sybil. Incluye pruebas unitarias de todos los escenarios de modificación de reputación y acceso no autorizado.
- Sprint 5 — Integración y auditoría de contratos (completado):** integración de los tres contratos (OE4): módulo de despliegue unificado con Hardhat Ignition, pruebas de extremo a extremo de la capa *on-chain* completa (flujos de publicación, votación, reapertura y reputación retroactiva), análisis estático de seguridad con Slither y exportación de ABIs para las capas superiores.
- Sprint 6 — Mecanismo de reputación (II):** ampliación de `ValidationRegistry` (OE3/OE4) con una recompensa reputacional por publicar contenido que alcanza consenso `TRUE` y una penalización si resuelve `FALSE` o `UNVERIFIABLE`, y con una vía de acceso meritocrático al rol de validador mediante predicciones de práctica que no participan en el consenso real.
- Sprint 7 — Capa de *backend*:** desarrollo de la API REST que orquesta la comunicación entre la *blockchain*, la base de datos *off-chain* y el frontend. Incluye el modelo de datos y las pruebas de integración de los endpoints.
- Sprint 8 — Prototipo de interfaz de usuario:** desarrollo del frontend web (OE5): publicación de contenido, consulta del estado de verificación, visualización de la reputación de los validadores y conexión con una cartera digital.
- Sprint 9 — Integración, pruebas y despliegue público:** integración de todas las capas del sistema y pruebas de extremo a extremo sobre los escenarios de uso principales. Ajuste del rendimiento y coste de gas de los contratos. Despliegue definitivo de los contratos en la testnet pública **Sepolia** para disponer de un prototipo verificable externamente de cara a la entrega y la defensa.
- Sprint 10 — Evaluación y documentación:** análisis de los resultados del prototipo (OE7), elaboración de los capítulos de resultados y conclusiones de la memoria, y preparación de la defensa.

La Tabla 3.1 resume la correspondencia entre los sprints, los objetivos específicos cubiertos y los entregables esperados.

Tabla 3.1: Planificación de *sprints* y objetivos asociados.

<i>Sprint</i>	Fase	Entregable	OE	Estado
0	Fundamentos	Marco teórico + objetivos	OE1	Completado
1	Arquitectura	Documento de diseño del sistema	OE2	Completado
2	Contratos (I)	Contrato de registro de publicaciones	OE3	Completado
3	Contratos (II)	Contrato de verificación + tests	OE3	Completado
4	Reputación	Contrato de reputación + tests	OE4	Completado
5	Integración	Contratos integrados + auditoría Slither + ABIs	OE4	Completado
6	Reputación (II)	Recompensa por publicación + acceso meritocrático por predicción	OE3/OE4	Planificado
7	Backend	API REST + base de datos	OE2	Planificado
8	Frontend	Interfaz web funcional	OE5	Planificado
9	Integración	Sistema integrado + tests E2E + despliegue Sepolia	OE7	Planificado
10	Evaluación	Informe de resultados + memoria	OE7	En curso

3.4. Principios de desarrollo

El diseño e implementación del prototipo de NewsEra se rige por un conjunto de principios de ingeniería de software que condicionan tanto la estructura del código como la elección del stack tecnológico. Estos principios no constituyen restricciones arbitrarias, sino decisiones conscientes adoptadas para maximizar la corrección, la mantenibilidad y la velocidad de implementación en el contexto de un proyecto académico individual con un horizonte temporal definido.

El primer principio rector es **KISS** (*Keep It Simple, Stupid*): se priorizan soluciones simples y directas sobre soluciones más elegantes pero de mayor complejidad. En el contexto de un prototipo de TFG, la complejidad innecesaria tiene dos efectos negativos concretos: ralentiza la implementación al multiplicar las decisiones de diseño que deben tomarse durante el desarrollo, y dificulta la verificación de la corrección del sistema, ya que las rutas de ejecución se vuelven más difíciles de razonar y de cubrir con pruebas. Cuando existe una solución suficientemente buena y una solución óptima, el prototipo adopta la primera si la diferencia de complejidad es significativa y el beneficio marginal es reducido.

El segundo principio es **DRY** (*Don't Repeat Yourself*): cada regla de negocio o fragmento de lógica del sistema existe en un único lugar del código. La duplicación de lógica introduce invariablemente inconsistencias cuando el requisito evoluciona, ya que obliga a localizar y actualizar manualmente todas las copias. En el contexto de un sistema distribuido entre capas heterogéneas —contratos Solidity, servidor TypeScript e interfaz de usuario—, DRY implica también definir con precisión qué capa es la fuente de verdad para cada dato y evitar recalcular en el cliente lo que el servidor ya ha procesado, o validar en el servidor lo que el contrato ya hace cumplir de forma irrevocable.

El tercer principio, y quizás el más determinante en la arquitectura del sistema, es la **separación de responsabilidades**: cada capa del sistema tiene un rol bien definido y no

invade el del resto. Los contratos inteligentes no almacenan texto completo; el servidor no firma transacciones directamente en nombre del usuario; el frontend no contiene lógica de negocio que deba mantenerse sincronizada con el servidor. Esta separación limita el impacto de los cambios a la capa en la que se producen y facilita la sustitución independiente de cada componente. Estas decisiones se reflejan directamente en la elección del stack tecnológico: se prefieren *frameworks* ligeros con convenciones claras (Hono sobre Express), lenguajes con tipado estático para detectar errores en tiempo de compilación (TypeScript) y herramientas que reducen la toma de decisiones durante la implementación, permitiendo al desarrollador concentrarse en la lógica del dominio.

3.5. Stack tecnológico

Las siguientes secciones describen el stack tecnológico adoptado para el prototipo. Las decisiones reflejan el resultado del análisis realizado durante el Sprint 1 y quedan documentadas en detalle en el Capítulo 4.

3.5.1. Capa blockchain: red y framework de desarrollo

Para el desarrollo de los contratos inteligentes se propone trabajar sobre la **red Ethereum**. La elección de Ethereum se justifica por ser la plataforma de referencia para contratos inteligentes, con el ecosistema de herramientas, documentación y comunidad más maduro disponible (Buterin, 2014). El ciclo de desarrollo y las pruebas unitarias y de integración se ejecutan sobre **Hardhat Network**, el nodo Ethereum local que incorpora el propio framework de desarrollo, que permite iterar sin costes de red ni tiempos de confirmación. El despliegue definitivo del prototipo se realiza en la testnet pública **Sepolia** al final del Sprint 9, como paso previo a la entrega, para que el evaluador pueda verificar los contratos en un explorador de bloques público sin necesidad de reproducir el entorno local.

Como alternativa a considerar durante el *Sprint 1* se contempla **Polygon PoS**, una red compatible con la EVM que reduce el coste de las transacciones en varios órdenes de magnitud respecto a la red principal de Ethereum, lo que puede resultar relevante para un prototipo con un volumen significativo de operaciones de registro.

Para el entorno de desarrollo local, compilación y pruebas de contratos inteligentes se propone **Hardhat**, por las siguientes razones:

- Ecosistema maduro con amplia documentación y soporte de la comunidad.
- Flujo de trabajo en JavaScript/TypeScript, coherente con el backend y el frontend propuestos.
- *Hardhat Network*: nodo Ethereum local para pruebas que permite simular condiciones de red, manipular el estado de la cadena e inspeccionar el coste de gas de cada operación.
- Integración nativa con las principales bibliotecas del ecosistema (ethers.js, OpenZeppelin, Chai).

Como biblioteca de contratos auditados y reutilizables se propone **OpenZeppelin Contracts**, que proporciona implementaciones de referencia para patrones de control de acceso,

gobernanza y gestión de roles que pueden adaptarse al mecanismo de reputación de NewsEra, reduciendo el riesgo de introducir vulnerabilidades en código crítico.

3.5.2. Capa de backend

Se emplea **Node.js con TypeScript** como entorno de ejecución, y **Hono** como framework web. Hono es un framework ultraligero y de alto rendimiento, con soporte nativo de TypeScript, un modelo de rutas y *middleware* equivalente al de Express pero con una API más ergonómica y sin configuración adicional. Hono se ha consolidado como uno de los frameworks más adoptados para nuevas APIs REST en el entorno JavaScript moderno.

Para el acceso a la base de datos se utiliza **Prisma ORM**, que genera tipos TypeScript automáticamente a partir del esquema de la base de datos y gestiona las migraciones de forma integrada. El almacenamiento *off-chain* del contenido completo de las publicaciones y sus metadatos se realiza en **PostgreSQL**, base de datos relacional con soporte nativo para tipos JSON y búsqueda de texto completo. Cada registro incluye como campo clave el hash del contenido registrado en la *blockchain*, garantizando la trazabilidad verificable entre almacenamiento *off-chain* y registro *on-chain*.

Para la interacción con los contratos inteligentes desde el servidor se utiliza la biblioteca **viem**, elegida por su tipado estático completo en TypeScript y su rendimiento generalmente superior en operaciones de decodificación de ABI respecto a ethers.js.

Para el almacenamiento descentralizado del contenido completo de los artículos se utiliza **IPFS (InterPlanetary File System)** a través del servicio de *pinning* **Pinata**. Cuando un usuario publica un artículo, el frontend sube el contenido a IPFS mediante la API de Pinata y obtiene un CID único que identifica ese contenido de forma permanente y verificable. Este CID se almacena en el campo `ipfsCid` del modelo de datos *off-chain*. A diferencia de almacenar el contenido en PostgreSQL, IPFS garantiza que el cuerpo del artículo no depende de un servidor centralizado para su disponibilidad.

3.5.3. Capa de frontend

Se emplea **React** con **Vite** como *bundler*, configurando la aplicación como una SPA (*Single Page Application*) con renderizado en el cliente. La elección de este modelo frente a **Next.js**—considerado en fases previas del diseño— responde a un criterio de alineación con el principio KISS: en una aplicación Web3 donde la práctica totalidad de las vistas requieren acceso a la cartera del usuario, la distinción entre Server Components y Client Components del App Router de Next.js añade una capa de complejidad arquitectónica sin aportar beneficio significativo. React con Vite ofrece un modelo mental más simple, tiempos de compilación más cortos y una experiencia de depuración más directa, factores especialmente valiosos en el contexto de un proyecto académico individual.

Para la navegación entre páginas se utiliza **React Router v6**, solución de enrutamiento de referencia en el ecosistema React para aplicaciones de tipo SPA. Los estilos se implementan con **Tailwind CSS**, un framework de utilidades CSS que permite construir interfaces sin escribir hojas de estilo personalizadas. La biblioteca de componentes de referencia es **shadcn/ui**, una colección de componentes de código abierto construidos sobre Radix UI y Tailwind CSS que el desarrollador incorpora directamente al repositorio, sin dependencia de un paquete externo.

Para la interacción con la *blockchain* desde el frontend se emplea **wagmi v2** junto con **viem**, que ofrecen *hooks* de React con tipado estático completo para las operaciones de lectura y escritura en contratos. La UX de conexión de cartera se gestiona a través de **RainbowKit**, que proporciona un componente de conexión compatible con MetaMask, WalletConnect y otras carteras habituales del ecosistema Ethereum con una integración mínima de código. Toda la interacción con la *blockchain* y con la API del servidor ocurre íntegramente en el cliente, mediante *hooks* de React y **fetch** nativo, sin lógica de negocio delegada al servidor de frontend.

3.5.4. Herramientas de desarrollo y control de versiones

El entorno de desarrollo utiliza las siguientes herramientas:

- **Visual Studio Code** como IDE, con las extensiones de Solidity, ESLint y Prettier para el análisis estático y el formateo consistente del código.
- **Git y GitHub** para el control de versiones. El repositorio de implementación del prototipo es público y está disponible en:

<https://github.com/jrdj1/NewsEra>

- **Chai** como biblioteca de aserciones para las pruebas de contratos inteligentes, integrada en el entorno Hardhat.
- **Vitest** para las pruebas unitarias del backend y los componentes del frontend: ofrece compatibilidad completa con la API de Jest pero con soporte nativo de ESM y TypeScript y una velocidad de ejecución significativamente superior.
- **Bruno** para el desarrollo y prueba manual de los *endpoints* de la API REST: cliente HTTP de código abierto que almacena las colecciones de peticiones en el repositorio como archivos de texto plano, a diferencia de Postman, que las gestiona en la nube.
- **Docker y Docker Compose** para el despliegue del entorno local. La base de datos PostgreSQL y cualquier servicio auxiliar se ejecutan como contenedores definidos en un archivo `docker-compose.yml` en la raíz del repositorio. Esto garantiza que el entorno de desarrollo es reproducible en cualquier máquina sin necesidad de instalar ni configurar manualmente los servicios de base de datos.
- **Pinata** como servicio de *pinning* para IPFS. El contenido completo de los artículos se almacena en la red IPFS mediante la API de Pinata, que ofrece un nivel gratuito suficiente para el prototipo. El CID (*Content Identifier*) asignado por IPFS a cada artículo se registra en el modelo de datos *off-chain* y sirve como referencia de almacenamiento descentralizado del contenido completo.

3.6. Estrategia de validación

La validación del prototipo se articuló en cinco niveles complementarios, con el objetivo de demostrar el cumplimiento de los objetivos específicos OE3, OE4 y OE7:

3.6.1. Pruebas unitarias de contratos inteligentes

Cada función de los contratos inteligentes fue sometida a un conjunto de pruebas automatizadas que cubren los siguientes escenarios: el camino feliz (*happy path*), las condiciones de contorno y los intentos de uso indebido que el contrato rechaza mediante errores personalizados. Se estableció como umbral mínimo una cobertura del 80 % de las instrucciones; el resultado final superó ese umbral en todos los contratos (ver Capítulo 5). Las pruebas se ejecutaron sobre *Hardhat Network* y se automatizaron en el pipeline de integración continua del repositorio de GitHub.

3.6.2. Pruebas de escenario del mecanismo de gobernanza

Dado que el mecanismo de reputación y gobernanza es el componente nuclear del sistema (OE4), se diseñaron pruebas de escenario específicas que simulan situaciones de uso adverso:

- **Ataque Sybil:** un actor intenta crear múltiples cuentas para acumular reputación artificial. El sistema detecta y penaliza este comportamiento mediante la puerta de entrada por reputación mínima (`MIN_REPUTATION_TO_VALIDATE`).
- **Validador malicioso:** un validador emite sistemáticamente juicios contrarios al consenso de la comunidad. El mecanismo degrada su reputación hasta excluirlo del proceso de validación.
- **Colusión de validadores:** un grupo de validadores coordina sus votos para validar contenido falso. El diseño incorpora el mecanismo de reapertura por rondas como salvaguarda ante esta amenaza.
- **Validador honesto bajo presión:** un validador con buena reputación enfrenta contenido ambiguo. El sistema se comporta de forma predecible y justa, ajustando la reputación retroactivamente según el veredicto final consolidado.

3.6.3. Análisis de coste de gas

Cada operación de los contratos inteligentes tiene un coste en gas que determina su viabilidad económica en la red real. Se midió el coste de las operaciones principales —registro de publicación, emisión de verificación, actualización de reputación— y se compararon con los umbrales habituales en la red Ethereum. Los resultados de este análisis se recogen en el Capítulo 5.

3.6.4. Evaluación de viabilidad del sistema

Al concluir los sprints de implementación de contratos se llevó a cabo una evaluación cualitativa y cuantitativa orientada a responder la pregunta central del trabajo: ¿es técnicamente viable construir una plataforma de información descentralizada resistente a la captura por grupos de poder? Esta evaluación contempló:

- El grado de cumplimiento de cada objetivo específico (OE1–OE7).
-

- Las limitaciones técnicas identificadas durante el desarrollo (escalabilidad, coste, usabilidad).
- La comparación entre las propiedades del sistema y las de las soluciones centralizadas equivalentes.
- Las condiciones sociales y técnicas necesarias para que el sistema pueda operar en un entorno real.

3.6.5. Experimentos de validación con usuarios

Los cuatro niveles anteriores validan que el sistema funciona correctamente y de forma segura, pero no responden a una pregunta distinta y igualmente necesaria: si el problema que motiva el proyecto es percibido como real por sus destinatarios, y si la solución propuesta resulta comprensible y utilizable para ellos. Siguiendo la metodología de experimentación propia de *Lean Startup* (Ries, 2011), se diseñaron dos experimentos independientes, cada uno con una hipótesis, un Minimum Viable Product (MVP) y un criterio de éxito definidos de antemano.

Experimento 1: intensidad del problema

Hipótesis. Una proporción significativa de los usuarios de redes sociales y consumidores habituales de noticias *online* perciben la desinformación como un problema grave en su consumo de información, y no disponen de un método fiable para verificarla.

MVP. Encuesta estructurada, sin exposición previa al prototipo ni a ninguna solución concreta, para evitar sesgar la percepción del problema con la propia respuesta propuesta.

Instrumento. Preguntas en escala Likert de 5 puntos sobre frecuencia de exposición a contenido dudoso, confianza en las fuentes de información actuales, experiencia previa de haber compartido o creído contenido posteriormente desmentido, y disponibilidad de un método propio de verificación.

Criterio de éxito. Al menos el 60 % de las respuestas califica el problema como grave o muy grave (4 o 5 en la escala), y al menos el 50 % declara no disponer de un método fiable de verificación en la actualidad.

Experimento 2: idoneidad del producto

Hipótesis. Al interactuar con el prototipo funcional, los usuarios comprenden la propuesta de valor de NewsEra, perciben el mecanismo de consenso comunitario como confiable, y expresarían disposición a utilizar la plataforma.

MVP. El propio prototipo desplegado, en su estado funcional real —no una maqueta ni una simulación—: los participantes interactúan con los flujos principales (consultar el *feed*, publicar un artículo, validar contenido) sobre la aplicación completa descrita en el Capítulo 4.

Instrumento. Cuestionario posterior a la interacción que combina el *System Usability Scale* (Brooke, 1996) —diez ítems estandarizados, puntuación de 0 a 100— con preguntas específicas que aíslan, sin nombrar la tecnología subyacente, la confianza percibida en cada propiedad arquitectónica (inmutabilidad, consenso colectivo, reputación), la fricción de la interacción *Web3* (firma de transacciones) como constructo independiente, e intención de uso futura.

Criterio de éxito. Puntuación SUS media igual o superior a 68 —el umbral de referencia habitual para considerar “aceptable” la usabilidad de un sistema (Brooke, 1996)— y al menos el 60 % de los participantes expresa intención de uso.

Ambos experimentos son complementarios: el primero valida que el problema existe con independencia de la solución concreta; el segundo, que la solución construida resulta comprensible y utilizable para resolverlo. Un resultado positivo en el primero y negativo en el segundo apuntaría a un problema de diseño de interfaz, no de planteamiento del proyecto —distinción relevante para orientar el trabajo futuro—. Las limitaciones propias de una muestra de conveniencia, de tamaño reducido y no representativa de la población general, se discuten en el §6.3 junto con el resto de limitaciones del trabajo. El contenido íntegro de ambos bloques de preguntas, listo para su implementación, se recoge en el Anexo B; los resultados obtenidos al ejecutarlos se recogen en el Capítulo 5.

3.7. Gestión de riesgos

Todo proyecto de desarrollo de software incorpora incertidumbres que deben ser identificadas y gestionadas de forma proactiva. La Tabla 3.2 recoge los riesgos principales identificados para este proyecto, junto con su probabilidad estimada, su impacto y la estrategia de mitigación propuesta.

Tabla 3.2: Registro de riesgos del proyecto.

Riesgo	Prob.	Impacto	Mitigación
Complejidad mayor de lo prevista en los contratos de gobernanza	Alta	Alto	Reducir el alcance del OE4 al prototipo mínimo viable; delegar la gobernanza compleja a trabajo futuro
Coste de gas inviable en Ethereum mainnet	Media	Medio	Evaluar Polygon PoS o Layer 2 como alternativa en Sprint 1
Vulnerabilidades de seguridad en los contratos	Media	Alto	Uso de OpenZeppelin, pruebas exhaustivas y revisión del código con herramientas de análisis estático (Slither)
Cambios en las APIs o herramientas del ecosistema blockchain	Baja	Medio	Fijar versiones de dependencias; seguir el <i>changelog</i> de Hardhat y ethers.js
Desviación temporal en algún sprint	Alta	Bajo	Reducir el alcance del sprint afectado y redistribuir tareas no críticas al backlog

4. Diseño del sistema

Este capítulo presenta el diseño arquitectónico de NewsEra resultante del Sprint 1. Constituye el documento de diseño que articula las decisiones tomadas sobre la estructura del sistema antes de la implementación: qué componentes existen, qué responsabilidades tiene cada uno, cómo se comunican entre sí y qué datos almacena cada capa. El diseño toma como punto de partida los objetivos específicos OE2–OE5 definidos en el Capítulo 1 y el stack tecnológico seleccionado en el Capítulo 3.

La arquitectura de NewsEra responde a un principio central: las reglas del sistema —quién puede publicar, cómo se valida el contenido, cómo se acumula y pierde reputación— se codifican en contratos inteligentes públicos e inmutables desplegados en la *blockchain* de Ethereum. Ningún actor puede modificarlas unilateralmente. El resto del sistema —backend, base de datos, interfaz de usuario— existe para hacer ese núcleo accesible y usable, pero no puede subvertir sus reglas (Buterin, 2014).

El código fuente completo del prototipo —contratos inteligentes, backend y frontend— está disponible públicamente en el repositorio de GitHub del proyecto:

<https://github.com/jrdj1/NewsEra>

4.1. Especificación de requisitos

Antes de abordar el diseño arquitectónico, esta sección recoge la Especificación de Requisitos de Software (ERS) elaborada durante el Sprint 1, siguiendo la estructura de contenidos recomendada por el estándar (Institute of Electrical and Electronics Engineers, 1998): el catálogo de casos de uso que debe soportar el prototipo, organizado en un sistema de temas (STQR), funcionalidades de alto nivel (FEAT) y casos de uso concretos (UC), junto con los requisitos no funcionales, de datos y de interfaz que condicionan su implementación. El resto del capítulo desarrolla el diseño técnico que satisface estos requisitos.

4.1.1. Casos de uso

Todos los casos de uso del prototipo se agrupan bajo un único sistema temático, **STQR 1. Sistema de publicación y verificación de contenidos**, dividido en cinco funcionalidades de alto nivel. La Tabla 4.1 resume el alcance de cada una.

4.1.1.1. FEAT 1. Gestión de cuenta

En NewsEra no existe un sistema de cuentas tradicional con credenciales: la cartera conectada *es* la cuenta. Sin cartera conectada, el sistema ofrece un perfil público limitado a las funcionalidades de solo lectura de FEAT 3 (Explorar contenido); conectar la cartera desbloquea las siguientes funcionalidades:

Tabla 4.1: Funcionalidades de alto nivel (FEAT) de STQR 1 y sus casos de uso.

FEAT	Nombre	Casos de uso
FEAT 1	Gestión de cuenta	UC 1–11
FEAT 2	Publicar artículo	UC 12–19
FEAT 3	Explorar contenido	UC 20–31
FEAT 4	Verificar artículo	UC 32–40
FEAT 5	Información institucional	UC 41–42

- **UC 1.** Conectar cartera.
- **UC 2.** Actualizar / modificar datos del perfil enriquecido (nombre, avatar, email opcional).
- **UC 3.** Revisar artículos guardados como favoritos.
- **UC 4.** Revisar artículos publicados propios.
- **UC 5.** Revisar votaciones anteriores (ganadoras, perdedoras, sin resolver, y métricas de acierto).
- **UC 6.** Revisar reputación actual.
- **UC 7.** Revisar evolución histórica de la reputación.
- **UC 8.** Reclamar reputación neta (retroactiva).
- **UC 9.** Revisar solicitudes de reapertura realizadas.
- **UC 10.** Consultar notificaciones.
- **UC 11.** Desconectar cartera.

4.1.1.2. FEAT 2. Publicar artículo

- **UC 12.** Redactar artículo siguiendo la plantilla estándar de redacción, común a todas las publicaciones.
- **UC 13.** Guardar borrador antes de confirmar la publicación.
- **UC 14.** Etiquetar el artículo por categoría o temática.
- **UC 15.** Enlazar otros artículos de NewsEra mediante hipervínculos internos.
- **UC 16.** Incluir referencias bibliográficas.
- **UC 17.** Consultar vista previa del artículo antes de confirmar.
- **UC 18.** Consultar el `contentHash` calculado antes de firmar la transacción.
- **UC 19.** Confirmar la publicación *on-chain* firmando la transacción con la cartera.

Autoría y fecha de publicación no se modelan como casos de uso independientes: son atributos generados automáticamente por el sistema como efecto de UC 19 (autoría = dirección que firma la transacción; fecha = *timestamp* del bloque), no acciones que el actor inicie de forma separada.

4.1.1.3. FEAT 3. Explorar contenido

Es la única funcionalidad accesible sin cartera conectada:

- **UC 20.** Consultar el *feed* de artículos recientes.
- **UC 21.** Buscar / filtrar artículos por estado de consenso, etiquetas o autor.
- **UC 22.** Ver el detalle de un artículo (contenido y metadatos).
- **UC 23.** Ver el estado de consenso del artículo.
- **UC 24.** Ver el recuento de votos del artículo.
- **UC 25.** Consultar el ranking de validadores por reputación.
- **UC 26.** Consultar el perfil público de una dirección (reputación, historial de validación y artículos publicados).
- **UC 27.** Acceder a la verificación del artículo.
- **UC 28.** Ordenar los resultados del *feed* o la búsqueda.
- **UC 29.** Seguir un artículo para recibir notificaciones de cambios de estado.
- **UC 30.** Compartir un artículo mediante enlace directo.
- **UC 31.** Guardar o quitar un artículo de favoritos.

4.1.1.4. FEAT 4. Verificar artículo

Concentra las herramientas que apoyan la decisión de un validador antes y durante el voto, además de las acciones que exigen cartera conectada y reputación suficiente:

- **UC 32.** Consultar la bibliografía y los enlaces internos citados por el artículo.
 - **UC 33.** Comparar el artículo con otros de NewsEra sobre el mismo tema.
 - **UC 34.** Consultar el historial de rondas anteriores y su resultado.
 - **UC 35.** Votar el artículo (**TRUE** / **FALSE** / **UNVERIFIABLE**).
 - **UC 36.** Solicitar la reapertura de un artículo concluido.
 - **UC 37.** Reclamar reputación retroactiva sobre el artículo.
 - **UC 38.** Ver el progreso hacia el quórum (votos emitidos frente a los necesarios para cerrar la ronda).
-

- **UC 39.** Consultar qué validadores ya han votado en la ronda actual.
- **UC 40.** Ver el efecto reputacional estimado antes de votar.

4.1.1.5. FEAT 5. Información institucional

- **UC 41.** Consultar la landing page (propuesta de valor y tres pilares del proyecto).
- **UC 42.** Consultar la página “Sobre el proyecto” (funcionamiento, descripción de la Decentralized Autonomous Organization (DAO) y enlace público a la memoria del Trabajo Fin de Grado (TFG)).

La gobernanza descentralizada mediante una DAO plenamente operativa — votación de propuestas, gestión de roles vía interfaz de usuario— queda fuera del alcance de STQR 1 y se documenta como línea de trabajo futuro (§6.4), no como un caso de uso del prototipo.

4.1.2. Casos de uso detallados

El desarrollo completo de los 42 casos de uso —actor, precondiciones, flujo principal, flujos alternativos y postcondiciones— se recoge en el Anexo A para no interrumpir la narrativa de diseño de este capítulo. Los seis casos de uso que implican una transacción *on-chain* o una verificación criptográfica, y que por tanto concentran la complejidad funcional del sistema, son: UC 1 (Conectar cartera), UC 2 (Actualizar datos del perfil), UC 19 (Confirmar publicación *on-chain*), UC 35 (Votar artículo), UC 36 (Solicitar reapertura) y UC 37 (Reclamar reputación retroactiva).

4.1.3. Requisitos no funcionales

Rendimiento.

- **RNF 1.** El coste de gas de las operaciones *on-chain* (`registerPublication`, `submitValidation`, `requestReopen`, `claimRetroactiveReputation`) debe mantenerse dentro de límites razonables mediante *Tight Variable Packing* [objetivo de diseño; verificación empírica prevista en Sprint 5/8].
- **RNF 2.** El tiempo de confirmación de una transacción está condicionado por el tiempo de bloque de la red (12 s en Sepolia); la interfaz debe reflejar estados de espera sin bloquear la interacción.

Seguridad.

- **RNF 3.** Ningún componente del sistema almacena ni gestiona claves privadas del usuario; toda firma se realiza desde su cartera, gestionada mediante `wagmi` y `RainbowKit`.
 - **RNF 4.** El backend no firma transacciones en nombre de ningún usuario.
 - **RNF 5.** Los contratos deben superar un análisis estático (Slither) sin *findings* de severidad *High/Critical*.
-

- **RNF 6.** El control de acceso a funciones sensibles se implementa mediante `AccessControl` (roles), nunca mediante comprobaciones *ad-hoc*.
- **RNF 7. Transparencia.** Las reglas de publicación, validación y reputación están codificadas en contratos públicos y verificables en Etherscan; cualquier persona puede auditar el código fuente y el estado *on-chain* sin depender de un intermediario.
- **RNF 8. Inmutabilidad.** Una vez registrado un `contentHash` o emitido un voto, no puede alterarse ni eliminarse: los contratos no exponen funciones de borrado ni edición sobre datos ya confirmados.
- **RNF 9. Descentralización.** Ningún actor único controla el resultado del consenso: el backend nunca firma transacciones y la lógica de consenso reside íntegramente en los contratos. El almacenamiento en IPFS vía Pinata introduce un punto de centralización parcial en el *pinning* — si el contenido no ha sido replicado por otros nodos, su disponibilidad depende de ese servicio, aunque el `contentHash` *on-chain* permite verificar la integridad de cualquier copia recuperada.
- **RNF 10. Resistencia a la captura.** Los parámetros de gobernanza (`quorumThreshold`, `superMajorityBps`, `reopenThreshold`) se fijan en el constructor sin funciones de modificación posterior al despliegue.
- **RNF 11. Auditabilidad.** Toda acción relevante emite un evento *on-chain* que permite reconstruir el historial completo del sistema sin acceso privilegiado.
- **RNF 12. Trazabilidad.** Toda acción queda vinculada a una dirección pública y a un bloque con marca de tiempo verificable.

Usabilidad.

- **RNF 13.** La interfaz debe ser legible y funcional en dispositivos móviles.
- **RNF 14.** Todo estado de carga y error debe comunicarse explícitamente al usuario, sin pantallas en blanco.
- **RNF 15.** El perfil enriquecido (UC 2) no debe exigir contraseña; la propiedad de la dirección se verifica mediante `personal_sign`.

Mantenibilidad.

- **RNF 16.** Separación estricta de capas en el backend: enrutador → servicio → repositorio; el enrutador no accede directamente a Prisma.
 - **RNF 17.** Las carpetas `blockchain`, `backend` y `frontend` son proyectos independientes, sin *workspaces* ni monorepo.
 - **RNF 18.** Toda interacción con la *blockchain*, en backend y frontend, se realiza mediante `viem` — nunca `ethers.js` — para mantener un cliente único y consistente.
 - **RNF 19.** El backend se implementa con Hono como *framework* HTTP, TypeScript en modo estricto y módulos ESM nativos, sin CommonJS.
-

- **RNF 20.** Las pruebas manuales de la API se realizan con Bruno, no con herramientas equivalentes como Postman.

Portabilidad / compatibilidad.

- **RNF 21.** El backend requiere Node.js 20 LTS y PostgreSQL 16 vía Docker, con Prisma como ORM.
- **RNF 22.** El frontend debe operar contra red local (Hardhat Network) y Sepolia sin cambios de código, solo de configuración de `wagmi`.
- **RNF 23.** Los contratos deben ser compatibles con Solidity `^0.8.20`, Hardhat `^2.22` y OpenZeppelin Contracts `^5.x`, desplegables tanto en Hardhat Network como en Ethereum Sepolia.
- **RNF 24.** El frontend se implementa como SPA con React + Vite, React Router v6, Tailwind CSS v4 y shadcn/ui sobre Radix UI.

Escalabilidad.

- **RNF 25.** El diseño prioriza la simplicidad (KISS) frente a la escalabilidad de producción: `docker-compose` es suficiente para el prototipo; no se requiere orquestación (Kubernetes).

Cumplimiento / privacidad.

- **RNF 26.** El email opcional del perfil enriquecido (UC 2) es un dato personal; su almacenamiento debe ser *opt-in* explícito y no condicionar el acceso a ninguna funcionalidad.

Cobertura de pruebas.

- **RNF 27.** Cobertura de tests de los contratos inteligentes $\geq 80\%$ (`npx hardhat coverage`), con Chai para contratos y Vitest para backend y frontend.

4.1.4. Requisitos de datos

Principios generales.

- **RD 1.** La *blockchain* es la única fuente de verdad para autoría, `contentHash`, votos, estado de consenso por ronda y reputación. PostgreSQL almacena una réplica de solo lectura poblada por el indexador de eventos, nunca al revés.
- **RD 2.** El `contentHash` vincula la entidad *on-chain* (`Publication` en `PublicationRegistry`) con su réplica *off-chain* y con el contenido almacenado en IPFS.
- **RD 3.** Toda escritura *on-chain* debe reflejarse en PostgreSQL a través del indexador de eventos, no mediante escritura directa del frontend a la base de datos.

Publicación (`Publication`).

- **RD 4.** *On-chain*: `author`, `timestamp`, `exists` — inmutable, indexado por `contentHash`.
-

- **RD 5.** *Off-chain*: `title`, `body`, `authorAddress`, `tags[]`, `ipfsCid`, `consensusState`, `currentRound`, `reopenRequestCount`, `createdAt`.
- **RD 6.** El `body` almacenado en PostgreSQL debe satisfacer `keccak256(body) == contentHash`; cualquier discrepancia invalida el registro como corrupto.
- **RD 7.** `tags` es una lista de cadenas libres definida por el autor al publicar (UC 14).

Rondas de validación (Round).

- **RD 8.** Cada publicación tiene N rondas, cada una con `round`, `state`, `result`, `completed`.
- **RD 9.** Una vez `completed = true`, los campos `state` y `result` de esa ronda son inmutables.

Validaciones (Validation).

- **RD 10.** Cada voto: `contentHash`, `validatorAddress`, `vote`, `round`, `txHash`, `createdAt`.
- **RD 11.** Restricción de unicidad: un `validatorAddress` solo puede votar una vez por artículo, independientemente de la ronda; el voto es vinculante para las rondas posteriores salvo reclamación retroactiva (UC 37).

Validador / reputación (Validator).

- **RD 12.** *On-chain*: reputación y estado de registro, gestionados exclusivamente por `ReputationSystem`.
- **RD 13.** *Off-chain*: `address` (clave primaria), `reputationScore`, `lastSyncBlock`, `registeredAt`, `updatedAt`.
- **RD 14.** `reputationScore` *off-chain* debe ser una réplica exacta del valor *on-chain* en todo momento; cualquier desincronización se resuelve reindexando desde `lastSyncBlock`.

Reapertura y reclamación retroactiva.

- **RD 15.** `ReopenRequest`: `contentHash`, `requesterAddress`, `count`, `txHash`, `createdAt` — único por (`contentHash`, `requesterAddress`).
- **RD 16.** `RetroactiveClaim`: `contentHash`, `validatorAddress`, `netDelta`, `txHash`, `claimedAt` — único por (`contentHash`, `validatorAddress`, `txHash`).

Perfil enriquecido (UserProfile) — solo *off-chain*.

- **RD 17.** `address` (clave primaria), `displayName`, `avatarUrl`, `email` (opcional, *opt-in*), `updatedAt`.
- **RD 18.** No existe contraseña ni credencial almacenada; toda modificación requiere una firma válida (`personal_sign`) de la dirección propietaria.

Favoritos (Favorite) — solo *off-chain*.

- **RD 19.** `userAddress`, `contentHash`, `createdAt` — único por (`userAddress`, `contentHash`), sin contrapartida *on-chain*.

Notificaciones (Notification**) — solo *off-chain*.**

- **RD 20.** `userAddress`, `contentHash`, `type` (`REOPENED` / `CONSENSUS_REACHED` / `RETROACTIVE_APPLIED`), `read` (por defecto `false`), `createdAt`.
- **RD 21.** Poblada por el indexador de eventos en el mismo punto en que procesa `VotingReopened`, `ConsensusReached` y `RetroactiveClaimed` para los artículos que el usuario sigue o votó; no se calcula bajo demanda.
- **RD 22.** Sin contrapartida *on-chain*; editable por su propietario (marcar como leída). Se recomienda purgar notificaciones leídas con antigüedad superior a un umbral razonable, para evitar el crecimiento indefinido de la tabla [nota de diseño, no bloqueante para el prototipo].

Retención e integridad.

- **RD 23.** Ningún dato *on-chain* puede eliminarse ni editarse una vez confirmado (RNF 8).
- **RD 24.** Los datos exclusivamente *off-chain* (perfil enriquecido, favoritos, notificaciones) sí son editables o eliminables por su propietario, ya que no forman parte del protocolo de consenso.

4.1.5. Requisitos de interfaz

Interfaz de usuario.

- **RI 1.** Navegación mediante rutas de SPA (React Router v6): `/`, `/publish`, `/article/:hash`, `/validators`, `/validators/:address`, `/profile`, `/about`.
- **RI 2.** Cabecera fija global con logo, navegación y `ConnectButton` de RainbowKit, visible en todas las rutas.
- **RI 3.** Plantilla estándar de redacción para artículos (UC 12): todos los artículos se renderizan con el mismo formato visual, independientemente del autor.
- **RI 4.** Diseño responsive, legible en dispositivos móviles (RNF 13).
- **RI 5.** Estados de carga y error explícitos en toda vista que dependa de datos remotos, sea del backend o de la *blockchain* (RNF 14).
- **RI 6.** Toda acción que requiera firma de cartera muestra el *hash* de transacción y su estado (pendiente / confirmada / fallida) mediante `useWaitForTransactionReceipt`.
- **RI 7.** Los errores de revert conocidos del contrato (`InsufficientReputation`, `AlreadyValidated`, `VotingNotOpen`, `ReopenNotAvailable`, `AlreadyRequestedReopen`, `NothingToClaim`) se traducen a mensajes en lenguaje natural; nunca se muestra el error crudo de Solidity al usuario.

Interfaces externas.

- **RI 8.** Interfaz con la cartera del usuario vía **wagmi** + RainbowKit (MetaMask, Wallet-Connect).
- **RI 9.** Interfaz con la *blockchain* vía **viem**, contra Hardhat Network o Ethereum Sepolia según la configuración de red activa.
- **RI 10.** Interfaz con IPFS a través de la API de Pinata, para subida y recuperación del contenido de los artículos.
- **RI 11.** Interfaz REST entre frontend y backend bajo **/api/v1**, formato JSON, con estructura de error uniforme **{"error": {"code", "message"}}**. Códigos: **NOT_FOUND** (404), **CONFLICT** (409), **UNPROCESSABLE** (422), **FORBIDDEN** (403), **UNAUTHORIZED** (401 — credenciales de servicio ausentes o inválidas, usado en **POST /api/v1/sync/events**) e **INTERNAL_ERROR** (500).
- **RI 12.** Interfaz de verificación pública de los contratos desplegados en el explorador de bloques Etherscan (Sepolia).

4.2. Visión general de la arquitectura

El sistema NewsEra se estructura en tres capas con responsabilidades claramente diferenciadas: la capa *on-chain*, la capa *off-chain* y la capa de presentación. Esta separación refleja una distinción fundamental entre los datos que deben ser verificables e inmutables —y por tanto viven en la *blockchain*— y los datos que no requieren esa garantía pero sí eficiencia en la consulta —y por tanto viven en una base de datos relacional convencional.

4.2.1. Capa on-chain: la fuente de verdad

La capa *on-chain* está formada por tres contratos inteligentes escritos en Solidity y desplegados en la testnet Ethereum Sepolia. Esta capa almacena exclusivamente los datos que requieren garantías de inmutabilidad y verificabilidad pública: huellas digitales (*hashes*) de publicaciones, votos de validación y puntuaciones de reputación de los validadores. No almacena el cuerpo completo de los artículos: el coste de almacenamiento en cadena hace inviable guardar texto extenso, y en cualquier caso la inmutabilidad del hash es suficiente para garantizar la integridad del contenido (Ding et al., 2023).

Las reglas de gobernanza del sistema —cuántos votos forman quórum, cuánto aumenta o disminuye la reputación tras cada validación, qué umbral mínimo de reputación permite validar— están codificadas directamente en los contratos. Su modificación requeriría un nuevo despliegue, lo que exige consenso y transparencia.

4.2.2. Capa off-chain: eficiencia y consultabilidad

La capa *off-chain* está formada por un servidor backend construido con Node.js, TypeScript y el framework Hono, y una base de datos PostgreSQL accedida mediante el ORM Prisma. Esta capa almacena el contenido completo de las publicaciones, sus metadatos y un índice para la búsqueda y el filtrado eficiente.

El backend actúa también como indexador de eventos de la *blockchain*: escucha en tiempo real los eventos emitidos por los contratos (`PublicationRegistered`, `ValidationSubmitted`, `ReputationUpdated`) y actualiza la base de datos para mantener una réplica consistente del estado *on-chain*. Esta réplica acelera las consultas del frontend y evita que cada petición de lectura tenga que interrogar directamente a la *blockchain*.

La independencia entre capas garantiza que, si la base de datos *off-chain* se corrompe o se pierde, el estado completo del sistema puede reconstruirse relanzando el indexador contra el historial de eventos registrado en la cadena.

4.2.3. Capa de presentación: la interfaz Web3

La capa de presentación es una aplicación React con Vite como bundler, que implementa el patrón SPA (Single Page Application) con renderizado completamente en cliente. La elección de React+Vite sobre Next.js responde a la filosofía KISS adoptada en el proyecto (§3.4): en una aplicación Web3 donde la práctica totalidad de las vistas requieren acceso a la cartera del usuario, la distinción Server/Client Components de Next.js App Router añade complejidad conceptual sin aportar beneficio significativo. React+Vite ofrece un modelo mental más simple, compilación incremental más rápida y depuración más directa. El enrutamiento se gestiona con React Router v6. Los componentes de interfaz utilizan Tailwind CSS y shadcn/ui. La interacción con los contratos inteligentes y la cartera se implementa mediante wagmi v2, viem y RainbowKit, que son independientes del framework de frontend.

La aplicación expone siete rutas principales que cubren los flujos de uso fundamentales: `/` (portada con listado de artículos y su estado de validación), `/publish` (formulario de publicación de nuevo contenido), `/article/:hash` (detalle de artículo con panel de votación para validadores), `/validators` (directorío de validadores con ranking de reputación), `/validators/:address` (perfil público de validador con historial), `/profile` (perfil propio con estado de reputación y cartera conectada), y `/about` (información sobre el proyecto y enlace a la memoria).

La Figura 4.1 muestra los wireframes de las vistas principales del prototipo.

4.2.4. El hash criptográfico como vínculo entre capas

El elemento que mantiene la coherencia entre la capa *on-chain* y la *off-chain* es el hash criptográfico del contenido de cada publicación. Cuando un usuario publica un artículo, el frontend calcula el hash keccak256 del cuerpo del artículo antes de enviarlo a ningún servidor. Ese hash se registra en la *blockchain* como identificador permanente e inmutable de esa publicación. A continuación, el contenido completo se envía al backend, que lo almacena vinculado a ese mismo hash.

Cualquier actor puede, en cualquier momento, calcular el hash del contenido que obtiene del backend y compararlo con el registrado en la cadena. Si coinciden, el contenido no ha sido modificado. Si no coinciden, el backend ha sido manipulado —lo que es detectable de forma independiente, sin necesidad de confiar en ninguna entidad (Nakamoto, 2008).

[PENDIENTE: capturas/mockups de la interfaz]

Deben incluirse capturas de pantalla del prototipo funcional o, en su defecto, wireframes de las siguientes vistas:

- **Portada (/):** listado de artículos con indicadores visuales de su estado de consenso (PENDING / DEFINITIVE / DISPUTED), puntuación de reputación del autor y botón de conexión de cartera.
- **Detalle de artículo (/article/:hash):** cuerpo del artículo, historial de votos emitidos (dirección del validador, tipo de voto), panel de votación (TRUE / FALSE / UNVERIFIABLE) para usuarios con reputación suficiente, y resultado del consenso si ya se ha alcanzado.
- **Publicar artículo (/publish):** formulario con título, contenido y etiquetas; previsualización del hash keccak256 calculado localmente antes de firmar la transacción.
- **Perfil de validador (/validators/:address):** puntuación de reputación actual, historial de validaciones con resultado de cada una, y evolución temporal de la reputación.

Figura 4.1: Mockup de las vistas principales de la interfaz de NewsEra. La interfaz permite a cualquier usuario publicar contenido y consultar validaciones; para emitir votos se requiere conectar una cartera MetaMask y tener reputación mínima acreditada en el contrato `ReputationSystem`.

4.2.5. Esquema global del sistema

La Figura 4.2 muestra el esquema completo de NewsEra con sus tres capas y los flujos de interacción principales. El diagrama debe representar los tres bloques verticales (capa de presentación, capa *off-chain* y capa *on-chain*), los componentes principales de cada uno, y las flechas de comunicación: usuario → SPA → API → *blockchain* para escrituras; eventos *on-chain* → indexador → PostgreSQL → SPA para lecturas.

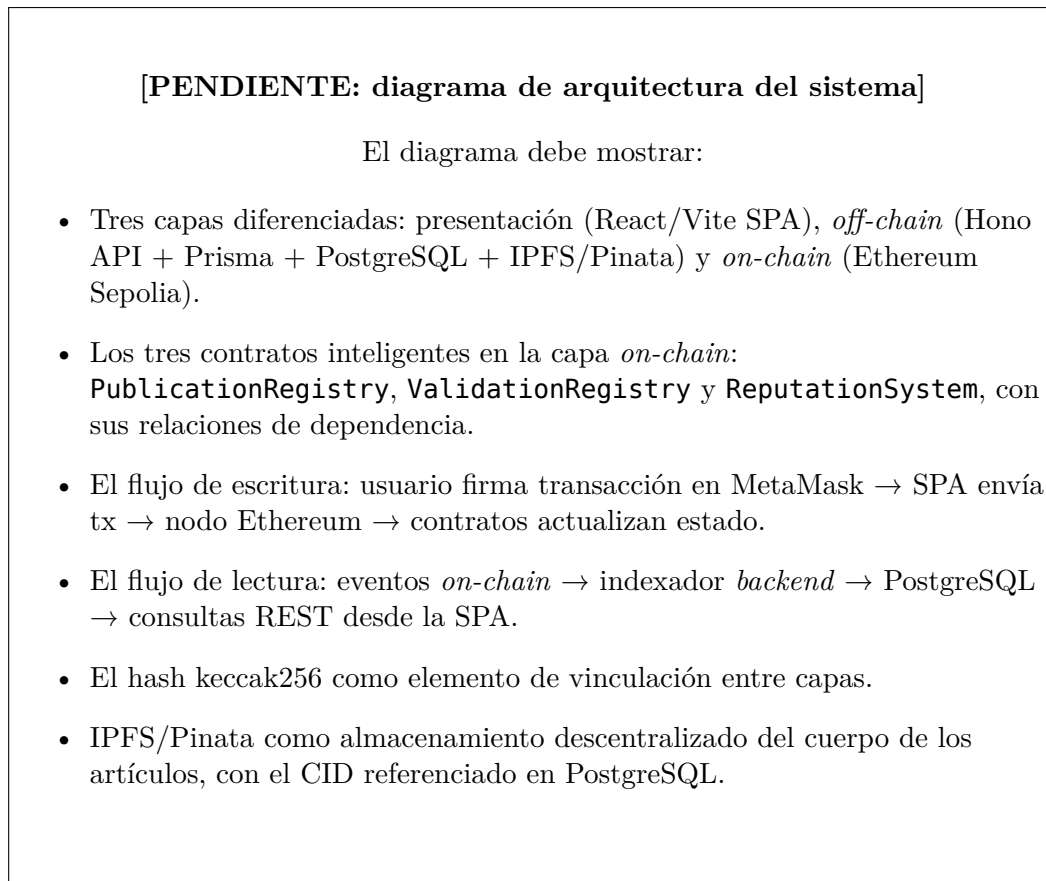


Figura 4.2: Arquitectura global del sistema NewsEra: capas y flujos de interacción. El contenido de los artículos se almacena en IPFS; los registros de publicación, validación y reputación, en los contratos inteligentes; los metadatos e índices, en PostgreSQL; la interfaz de usuario interactúa con las tres capas.

4.2.6. Distribución de datos entre capas

La Tabla 4.2 resume qué datos residen en cada capa del sistema y el motivo de esa distribución.

Tabla 4.2: Distribución de datos entre las capas del sistema NewsEra.

Dato	Capa	Motivo
Hash de publicación (bytes32)	On-chain (PublicationRegistry)	Inmutabilidad y verificabilidad pública
Dirección del autor y timestamp	On-chain (PublicationRegistry)	Atribución inalterable
Votos de validación (dirección, tipo, timestamp)	On-chain (ValidationRegistry)	Historial auditable de cada validador
Puntuación de reputación por dirección	On-chain (ReputationSystem)	La reputación no puede ser alterada por el backend
Cuerpo del artículo	Off-chain (IPFS vía Pinata)	Almacenamiento descentralizado; CID registrado en BD
Metadatos y CID de IPFS	Off-chain (PostgreSQL)	Índice consultable; reconstructible desde eventos on-chain
Índice de búsqueda y filtros por etiqueta	Off-chain (PostgreSQL)	Consultas complejas inviables en EVM
Réplica de reputación y votos (cache)	Off-chain (PostgreSQL)	Acelerar consultas; reconstructible desde eventos
Historial de transacciones (tx_hash)	Off-chain (PostgreSQL)	Enlace auditable entre evento on-chain y registro off-chain

4.3. Diseño de los contratos inteligentes

El núcleo de NewsEra se articula en tres contratos inteligentes con responsabilidades bien delimitadas.

■ ¿Verifican los contratos la veracidad automáticamente? Una pregunta natural al leer este diseño es si los contratos inteligentes determinan automáticamente si una noticia es verdadera o falsa. La respuesta es no: los contratos no tienen acceso a fuentes externas de información ni pueden razonar sobre el contenido de los artículos. Lo que **ValidationRegistry** hace es **agregar** los juicios emitidos por validadores humanos y aplicar las reglas de consenso de forma determinista e inmutable. El contrato responde a la pregunta «¿cuántos validadores votaron cada opción y se ha alcanzado supermayoría?», no a «¿es este artículo verdadero?». La veracidad es un juicio humano; el contrato garantiza que ese juicio colectivo no pueda ser manipulado por ningún actor una vez emitido. Esta distinción es fundamental para comprender los límites y el valor real del sistema.

Este diseño aplica el principio de separación de responsabilidades (*Separation of Concerns*) al nivel de los contratos, siguiendo los patrones de modularidad recomendados para sistemas de contratos en producción (Ding et al., 2023). Cada contrato expone una API mínima hacia el exterior y delega en los demás únicamente a través de interfaces explícitas, lo que facilita el análisis de seguridad y la actualización futura de componentes individuales.

Los tres contratos utilizan la biblioteca OpenZeppelin Contracts como base para los patrones de control de acceso. En concreto, **AccessControl** de OpenZeppelin se emplea para restringir qué contratos pueden llamar a las funciones sensibles de **ReputationSystem**: solo **ValidationRegistry** tiene el rol **VALIDATOR_ROLE** que le permite modificar puntuaciones de reputación. Esta restricción impide que cualquier actor externo, incluido el propietario del contrato, manipule directamente las puntuaciones (Feichtinger et al., 2023).

4.3.1. PublicationRegistry

PublicationRegistry es el contrato responsable del registro inmutable de publicaciones. Su función es simple y su superficie de ataque mínima: cualquier dirección Ethereum puede registrar una publicación proporcionando el hash keccak256 de su contenido. El contrato almacena la tripla (**contentHash**, **author**, **timestamp**) y emite el evento **PublicationRegistered** para que los sistemas *off-chain* puedan indexarlo.

El diseño es deliberadamente minimalista. El contrato no valida el contenido ni restringe quién puede publicar: la apertura a la publicación es un principio fundamental del sistema. Lo que garantiza el contrato es que, una vez registrado un hash, ese registro es permanente e inmutable: nadie puede eliminarlo ni modificar la autoría o el momento de publicación.

Diseño: PublicationRegistry

Almacenamiento: `mapping(bytes32 => Publication)` donde **Publication** es un struct con **author** (address), **timestamp** (uint256) y **exists** (bool).

Función principal: `registerPublication(bytes32 contentHash)` — visibilidad external. Revierte si el hash ya está registrado.
Evento: `PublicationRegistered(bytes32 indexed contentHash, address indexed author, uint256 timestamp)`.
Función de consulta: `getPublication(bytes32 contentHash)` — visibilidad external view. Devuelve la struct completa.

`ValidationRegistry` consulta `getPublication` para conocer al autor de un artículo en el momento de aplicar la recompensa por publicación (§4.3.2); esta es la única dependencia de `PublicationRegistry` hacia otro contrato del sistema, y es de solo lectura.

4.3.2. ValidationRegistry

`ValidationRegistry` gestiona el proceso de validación de publicaciones a lo largo de múltiples rondas de votación. El contrato verifica que cada validador tiene reputación suficiente, registra su voto y evalúa el consenso al alcanzar el quórum. A diferencia de un sistema de única ronda, el diseño contempla la **reapertura del proceso** cuando suficientes validadores nuevos solicitan revisar un veredicto existente, permitiendo que el conocimiento colectivo evolucione con nueva evidencia.

Cada voto puede ser de uno de tres tipos: **TRUE** (el contenido es veraz), **FALSE** (el contenido es falso o engañoso) o **UNVERIFIABLE** (el validador no puede determinar la veracidad con la evidencia disponible).

Voto plano con acceso gated

Todos los votos de validadores acreditados computan con el mismo peso, con independencia de su reputación acumulada. Esta decisión preserva la **independencia epistémica** de cada juicio: el incentivo racional de cada validador es evaluar el artículo con su propio criterio, porque su voto tiene el mismo impacto que el de cualquier otro. La resistencia Sybil la proporcióna el umbral mínimo de reputación para acceder al rol de validador —las cuentas nuevas no pueden votar—, sin necesidad de asignar pesos diferenciales. El razonamiento completo de esta decisión, incluyendo las alternativas consideradas, se expone en la Sección 5.6.

UNVERIFIABLE como veredicto de primera clase

El tipo de voto **UNVERIFIABLE** no es una abstención: computa en el cálculo del consenso, puede constituir el resultado definitivo de una ronda si obtiene supermayoría, y acarrea las mismas consecuencias reputacionales que cualquier otro voto incorrecto respecto al consenso final. Esta decisión elimina el incentivo a usarlo como opción dominante de riesgo cero. El razonamiento completo se expone en la Sección 5.6.

Umbral de certeza: quórum y supermayoría

El mecanismo de consenso opera con dos umbrales independientes:

1. **Quórum mínimo (`quorumThreshold`):** número mínimo de votos que debe alcanzar una ronda para que el contrato evalúe si hay consenso. Por debajo de este umbral la publicación permanece en estado **PENDING**.
2. **Supermayoría (`superMajorityBps`):** porcentaje mínimo que debe acumular la opción ganadora sobre el total de votos de la ronda para que el consenso sea *definitivo*. Se expresa en puntos básicos ($6667 = 66,67\%$). Si se alcanza el quórum pero ninguna opción obtiene supermayoría, el resultado es **DISPUTED**.

Mecanismo de reapertura por rondas

Una vez concluida una ronda (estado **DEFINITIVE** o **DISPUTED**), el proceso de validación no queda permanentemente cerrado. Cualquier validador con reputación suficiente que aún no haya votado ese artículo puede registrar una **solicitud de reapertura** mediante la función `requestReopen`. Cuando el número de solicitudes acumuladas alcanza el umbral `reopenThreshold`, el contrato abre automáticamente una nueva ronda de votación: el estado pasa a **PENDING** y los solicitantes —y cualquier otro validador nuevo— pueden emitir su voto.

Este mecanismo implementa el principio de *supervivencia del veredicto* propuesto por Adler y de Alfaro (2007) en el contexto de Wikipedia: la calidad de un juicio no se mide únicamente por su acuerdo con la mayoría en el momento en que se emite, sino por su capacidad de resistir el escrutinio de nuevos participantes independientes a lo largo del tiempo. En NewsEra, un veredicto que sobrevive múltiples rondas de desafío acumula evidencia progresiva de solidez; uno que es revertido por nuevas rondas revela que el conocimiento disponible ha evolucionado.

La Figura 4.3 muestra la máquina de estados resultante del mecanismo de rondas.

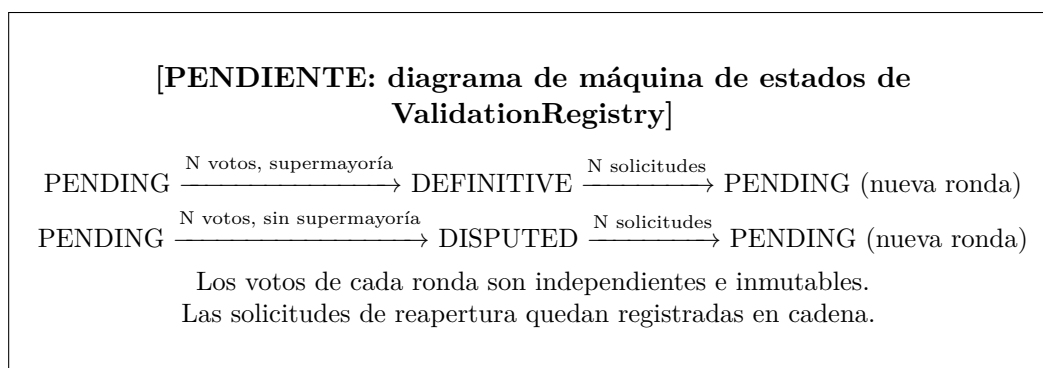


Figura 4.3: Máquina de estados de **ValidationRegistry**. El estado **PENDING_REOPEN** es transitorio: se activa internamente cuando se alcanza `reopenThreshold` solicitudes y se resuelve de forma inmediata al abrir la nueva ronda.

Efectos reputacionales: ronda propia y reputación retroactiva

El sistema de reputación opera en dos niveles complementarios.

Efectos de la ronda propia (aplicados inmediatamente al cerrar cada ronda):

- **Consenso DEFINITIVE:** los validadores que votaron la opción ganadora reciben `+REPUTATION_REWARD` (+5); los que votaron cualquier otra opción reciben `-REPUTATION_PENALTY` (−3).
- **Consenso DISPUTED:** no se modifica la reputación de ningún participante. El sistema reconoce que no hubo certeza suficiente.

Reputación retroactiva (modelo *pull*, reclamada por el validador):

Cada vez que cierra una nueva ronda sobre un artículo, los resultados de esa ronda revelan información sobre la calidad de los juicios emitidos en rondas anteriores. Un validador puede llamar a `claimRetroactiveReputation` para recibir los ajustes acumulados desde su ronda hasta la más reciente:

- Si la nueva ronda **confirma** el resultado de tu ronda: los que acertaron en su ronda reciben `+RETROACTIVE_DELTA` (+1); los que fallaron reciben −1.
- Si la nueva ronda **contradice** el resultado de tu ronda: los que acertaron en su ronda reciben −1; los que fallaron reciben +1 (reivindicación parcial).
- **Cap retroactivo:** el efecto acumulado queda limitado a `±RETROACTIVE_CAP` (±3) por artículo, independientemente del número de rondas.

El modelo *pull* —en lugar de aplicar los ajustes automáticamente— es una decisión deliberada de diseño: si el contrato iterara sobre todos los validadores históricos al cerrar cada nueva ronda, el coste de gas escalaría de forma proporcional al número total de participantes en el artículo, pudiendo resultar prohibitivo en artículos con muchas rondas. Con el modelo *pull*, el validador activa el cálculo cuando lo desee, pagando únicamente el gas de sus propios ajustes.

La Tabla 4.3 resume todos los escenarios posibles.

Tabla 4.3: Efectos reputacionales completos en `ValidationRegistry`. Los efectos de ronda propia se aplican al cerrar cada ronda; los retroactivos, al llamar a `claimRetroactiveReputation` (máximo ±3 acumulados).

Momento	Situación	Condición	Efecto
Ronda propia	DEFINITIVE	Votaste la opción ganadora	+5
Ronda propia	DEFINITIVE	Votaste otra opción	−3
Ronda propia	DISPUTED	Cualquier voto	0
Retroactivo	Ronda N confirma tu ronda	Acertaste en tu ronda	+1
Retroactivo	Ronda N confirma tu ronda	Fallaste en tu ronda	−1
Retroactivo	Ronda N contradice tu ronda	Acertaste en tu ronda	−1
Retroactivo	Ronda N contradice tu ronda	Fallaste en tu ronda	+1

Recompensa por publicación

Además de los efectos sobre quienes votan, la primera vez que la ronda de un artículo alcanza **DEFINITIVE** el contrato aplica un efecto reputacional al **autor** de la publicación, obtenido de `PublicationRegistry.getPublication(contentHash).author`:

- **Resultado TRUE:** el autor recibe `+PUBLISH_REPUTATION_REWARD (+8)`.
- **Resultado UNVERIFIABLE:** el autor recibe `-PUBLISH_REPUTATION_PENALTY_UNVERIFIABLE (-8)` — publicó una afirmación que la comunidad no pudo confirmar ni refutar.
- **Resultado FALSE:** el autor recibe `-PUBLISH_REPUTATION_PENALTY_FALSE (-15)`, la penalización más severa del sistema, reservada para el contenido probado falso.
- **Consenso DISPUTED:** sin efecto sobre el autor, igual que para los votantes.

Este efecto se aplica una sola vez por artículo — reaperturas posteriores (`VotingReopened`) no vuelven a evaluarlo— y se calcula en el mismo punto donde ya se resuelven los efectos de los votantes de la ronda, sin introducir un recorrido adicional sobre la lista de validadores: el coste de gas de leer un único autor es constante, a diferencia del recorrido acotado que sí requiere el cálculo de la reputación retroactiva.

A diferencia de los efectos sobre los votantes, esta recompensa invierte la proporción `reward:penalty` (los validadores tienen 5:3; aquí es 8:15) de forma deliberada: el problema que NewsEra aborda es la desinformación en el origen, no solo su detección, por lo que publicar contenido probado falso debe costar sensiblemente más que el beneficio de publicar contenido veraz.

Predicciones: acceso guiado sin publicar

El umbral mínimo de reputación para votar (`MIN_REPUTATION_TO_VALIDATE` en `ReputationSystem`) plantea un problema de arranque: ninguna dirección nueva puede acumular reputación votando, porque no puede votar sin reputación previa. La única vía de entrada hasta ahora era el registro manual del administrador (`registerValidator`) o publicar un artículo que alcance consenso `TRUE`. Para ofrecer una vía adicional, sin publicar nada, el contrato expone `submitPrediction(bytes32 contentHash, uint8 vote)`:

- Solo puede invocarla una dirección con `canValidate(msg.sender) == false` — una vez alcanzado el umbral, la dirección debe usar `submitValidation`, no este canal paralelo.
- Solo sobre un artículo cuyo estado ya sea `DEFINITIVE` — es decir, un artículo que la comunidad ya resolvió como `TRUE`, `FALSE` o `UNVERIFIABLE`. Revierte si el artículo está `PENDING` o `DISPUTED`: no hay todavía una respuesta fijada contra la que evaluar la predicción.
- Una dirección solo puede predecir una vez sobre cada artículo.
- La resolución es **inmediata y síncrona**, en la misma transacción: se compara el voto con `rounds[contentHash][currentRound].result` y se aplica al instante `+PREDICTION_REWARD (+1)` si coincide o `-PREDICTION_PENALTY (-1)` si no. No depende del consenso en curso de ningún otro artículo ni se resuelve junto con nadie: es asíncrona respecto a cualquier votación real de la red.

■ Una rampa de acceso guiada, no una prueba de criterio Puesto que el artículo objetivo ya está resuelto de forma pública e inmutable en el momento de predecir, cualquier dirección puede consultar la respuesta correcta antes de responder y acertar siempre. Ocultar el resultado en la interfaz no cambiaría esto: el dato sigue siendo legible directamente del contrato por cualquiera con conocimiento técnico mínimo, así que ocultarlo solo engañaría al usuario casual sin proteger nada real. Por ello, el mecanismo se documenta explícitamente como lo que es: una **rampa de acceso guiada y deliberadamente accesible**, no una prueba de criterio bajo incertidumbre — coherente con el principio de apertura ya aplicado a la publicación (§4.3.1). Su único límite natural es que cada artículo solo sirve una vez por dirección, y por tanto está acotado por cuántos artículos ya alcanzaron **DEFINITIVE** en un momento dado.

Fundamento epistemológico del consenso

El diseño anterior descansa en una postura epistemológica explícita que conviene enunciar antes de pasar a **ReputationSystem**.

Verdad operacional, no verdad absoluta. NewsEra no pretende descubrir la verdad objetiva de un artículo. Lo que el sistema produce es un *veredicto comunitario verificable*: la opinión agregada de un conjunto de validadores con historial probado, expresada *on-chain*, resistente a la manipulación retrospectiva y auditable por cualquier persona. Este veredicto puede coincidir con la verdad factual, pero su valor estructural reside en que *ningún actor puede manipularlo unilateralmente ni borrarlo*, no en que sea infalible. El quórum y la supermayoría —definidos arriba— son las condiciones de representatividad y certeza colectiva que operacionalizan esa noción de veredicto; **UNVERIFIABLE** es, dentro de ese marco, un reconocimiento legítimo de que la evidencia disponible no permite zanjar la cuestión.

DISPUTED como reconocimiento de la controversia. Cuando el quórum se alcanza pero ninguna opción obtiene supermayoría, el sistema no elige al ganador por diferencia mínima ni penaliza a nadie. El estado **DISPUTED** expresa que la comunidad está genuinamente dividida. Esta etiqueta es informativa por sí sola: un artículo marcado como controvertido transmite al lector una señal distinta a uno marcado como **TRUE**, **FALSE** o **UNVERIFIABLE**. No toda ausencia de consenso es equivalente a falsedad o a inverificabilidad.

Diseño: ValidationRegistry

Enumeraciones: `VoteType { TRUE, FALSE, UNVERIFIABLE }` y `ConsensusState { PENDING, DEFINITIVE, DISPUTED, PENDING_REOPEN }`.

Almacenamiento por ronda: `mapping(bytes32 => mapping(uint256 => RoundInfo))` public rounds donde `RoundInfo` contiene `result`, `state` y `completed`. `mapping(bytes32 => uint256)` public currentRound y `mapping(bytes32 => mapping(uint256 => uint256))` public roundVoteCount (votos por artículo y por ronda).

Almacenamiento por votante: `mapping(bytes32 => mapping(address => VoteType))` private _vote y `mapping(bytes32 => mapping(address => uint256))` public voterRound (inmutables una vez emitidos).

Funciones principales: `submitValidation(bytes32, uint8)` — emite voto en la ronda actual; `requestReopen(bytes32)` — registra solicitud de reapertura; si `reopenRequestCount >= reopenThreshold`, abre nueva ronda; `claimRetroactiveReputation(bytes32)` — aplica ajustes retroactivos de rondas posteriores a la del votante; `submitPrediction(bytes32, uint8)` — registra y resuelve de forma inmediata una predicción sobre un artículo ya **DEFINITIVE**, para direcciones con `canValidate == false`.

Parámetros de gobernanza (constructor): `quorumThreshold`, `superMajorityBps`, `reopenThreshold`.

Dependencias de contrato: `IReputationSystem` (efectos de reputación) y `PublicationRegistry` (consulta del autor para la recompensa por publicación).

Constantes: `REPUTATION_REWARD=5`, `REPUTATION_PENALTY=3`, `RETROACTIVE_DELTA=1`, `RETROACTIVE_CAP=3`, `PUBLISH_REPUTATION_REWARD=8`, `PUBLISH_REPUTATION_PENALTY_UNVERIFIABLE=8`, `PUBLISH_REPUTATION_PENALTY_FALSE=15`, `PREDICTION_REWARD=1`, `PREDICTION_PENALTY=1`.

Eventos: `ValidationSubmitted(..., uint256 round)`, `ConsensusReached(..., uint256 round)`, `ReopenRequested(...)`, `VotingReopened(..., uint256 newRound)`, `RetroactiveClaimed(..., int256 netDelta)`, `PredictionSubmitted(bytes32 indexed contentHash, address indexed predictor, uint8 vote, uint256 round)`.

4.3.3. ReputationSystem

`ReputationSystem` es el componente central del sistema (OE4). Gestiona las puntuaciones de reputación de todos los validadores, controla qué direcciones tienen acceso al rol de validador y expone las funciones de modificación de reputación exclusivamente a `ValidationRegistry`, usando `AccessControl` de OpenZeppelin para imponer esa restricción en el propio contrato.

El diseño del mecanismo de reputación sigue el patrón de *stake-based reputation*: la reputación se acumula participando correctamente y se pierde votando contra el consenso. No puede comprarse, transferirse ni delegarse: solo se obtiene mediante participación verificable y honesta (Messias et al., 2023). La integración de DIDs y VCs del W3C (World Wide Web Consortium, 2022) —solución de nivel de producción para la resistencia Sybil descrita en el §2.5— queda diferida como trabajo futuro dado que su implementación completa excede el alcance del prototipo actual, que adopta la reputación acumulativa como primera barrera de acceso. Este diseño es inherentemente resistente a ataques Sybil en el sentido siguiente: crear múltiples cuentas no proporciona ventaja porque cada cuenta comienza con reputación cero —insuficiente para validar— y debe acumular reputación de forma independiente a través de un historial verificable de participación.

Los validadores se incorporan al sistema con una puntuación de reputación inicial configurada por el administrador del contrato, que actúa como mecanismo de arranque inicial (*bootstrapping*): los primeros validadores del sistema son designados manualmente, y a partir de ahí el mecanismo de reputación opera de forma completamente autónoma. Este es un

compromiso de diseño habitual en sistemas de gobernanza descentralizada de nueva creación (Feichtinger et al., 2023).

Diseño: ReputationSystem

Almacenamiento (privado): `mapping(address => uint256) _reputation` y `mapping(address => bool) _registered`. El acceso externo se realiza mediante las funciones de lectura `getReputation` e `isRegisteredValidator`.

Rol de control de acceso: `bytes32 public constant VALIDATOR_ROLE`. Solo las direcciones con este rol (asignado a `ValidationRegistry` en el despliegue) pueden llamar a `increaseReputation` y `decreaseReputation`.

Funciones de lectura: `getReputation(address validator)` y `canValidate(address validator)` — devuelven la puntuación y si supera el umbral mínimo.

Funciones de escritura (restringidas): `increaseReputation(address validator, uint256 amount)` y `decreaseReputation(address validator, uint256 amount)` — solo invocable por `ValidationRegistry` mediante el control de acceso basado en roles.

Evento: `ReputationUpdated(address indexed validator, uint256 newScore, bool increased)`.

 Resistencia a ataques Sybil del mecanismo de reputación Un ataque Sybil consiste en crear múltiples identidades falsas para obtener una influencia desproporcionada en el sistema. En NewsEra, este ataque es estructuralmente ineficaz por dos razones:

1. La reputación inicial de cualquier dirección nueva es cero, insuficiente para participar como validador. No existe un camino rápido para adquirir reputación sin pasar por un proceso de participación progresiva y verificable.
2. Cada voto de validación queda registrado permanentemente en la cadena, vinculado a una dirección Ethereum. El historial de participación de cada dirección es público y auditable, lo que desincentiva el comportamiento oportunista: votar aleatoriamente o de forma coordinada contra el consenso degrada la reputación de todas las cuentas implicadas.

Este diseño contrasta con los sistemas de reputación basados en *token weighting*, en los que la reputación puede comprarse directamente en el mercado, creando un vector de ataque financiero (Feichtinger et al., 2023).

4.3.4. Tabla de funciones principales de los contratos

La Tabla 4.4 resume las funciones de la API pública de los tres contratos.

Tabla 4.4: Funciones principales de los contratos inteligentes de NewsEra.

Contrato	Función	Visibilidad	Descripción
PublicationRegistry	registerPublication	external	Registra el hash de una publicación nueva
PublicationRegistry	getPublication	external view	Devuelve autor y timestamp de un hash dado
ValidationRegistry	submitValidation	external	Registra un voto de validación; calcula consenso si hay quórum
ValidationRegistry	getVote	external view	Devuelve el tipo de voto y la ronda de un validador en una publicación
ValidationRegistry	hasVoted	external view	Comprueba si una dirección ya ha votado en alguna ronda
ValidationRegistry	getRoundVoters	external view	Devuelve la lista de votantes de una ronda concreta
ValidationRegistry	requestReopen	external	Registra solicitud de reapertura; si se alcanza reopenThreshold , inicia nueva ronda
ValidationRegistry	claimRetroactiveReputation	external	Aplica ajustes retroactivos de reputación de rondas posteriores a la del votante
ValidationRegistry	submitPrediction	external	Predicción sobre un artículo ya DEFINITIVE , resuelta al instante (solo si canValidate == false)
ReputationSystem	getReputation	external view	Devuelve la puntuación de reputación de una dirección
ReputationSystem	canValidate	external view	Indica si una di-

4.4. Modelo de datos

4.4.1. Almacenamiento on-chain

Las estructuras de datos *on-chain* están diseñadas para minimizar el consumo de gas, almacenando únicamente los datos estrictamente necesarios para garantizar la inmutabilidad e integridad del sistema. En Solidity, cada slot de almacenamiento ocupa 32 bytes; el empaquetado eficiente de variables de menor tamaño en un mismo slot es una técnica habitual de optimización de gas (Ding et al., 2023).

El Fragmento de código 4.4.1 muestra las estructuras de datos principales de los contratos `ValidationRegistry` y `ReputationSystem`.

Estructuras de datos on-chain (Solidity)

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 // --- PublicationRegistry ---
5 struct Publication {
6     address author;          // 20 bytes
7     uint96 timestamp;        // 12 bytes (packed with author)
8     bool exists;             // 1 byte (packed in same slot)
9 }
10 mapping(bytes32 => Publication) public publications;
11
12 // --- ValidationRegistry ---
13 enum VoteType { TRUE, FALSE, UNVERIFIABLE }
14 enum ConsensusState { PENDING, DEFINITIVE, DISPUTED, PENDING_REOPEN }
15
16 struct RoundInfo {
17     VoteType result;
18     ConsensusState state;
19     bool completed;
20 }
21
22 mapping(bytes32 => ConsensusState) public consensusState;
23 mapping(bytes32 => uint256) public currentRound;
24 mapping(bytes32 => uint256) public reopenRequestCount;
25
26 // --- ReputationSystem ---
27 mapping(address => uint256) private _reputation;
28 mapping(address => bool) private _registered;
29 uint256 public constant MIN_REPUTATION_TO_VALIDATE = 10;
30 uint256 public constant REPUTATION_REWARD = 5;
31 uint256 public constant REPUTATION_PENALTY = 3;

```

El empaquetado de `address` (20 bytes) con `uint96` (12 bytes) en un único slot de 32 bytes reduce el coste de escritura en almacenamiento en aproximadamente un 50% respecto a almacenar ambos en slots separados, siguiendo la práctica recomendada por el patrón *Tight Variable Packing* (Ding et al., 2023). Los mappings bidimensionales de `ValidationRegistry`

—indexados por `contentHash` y por número de ronda— se describen en el §4.3.2.

4.4.2. Esquema off-chain (PostgreSQL / Prisma)

La base de datos *off-chain* almacena el contenido completo de las publicaciones y mantiene una réplica del estado *on-chain* para acelerar las consultas. El esquema se define mediante Prisma ORM, que genera los tipos TypeScript a partir de la definición del modelo y gestiona las migraciones.

El campo `contentHash` es el vínculo entre ambas capas: actúa como clave única en la tabla `publications` y como clave foránea lógica en `validations`. Su presencia en ambos lados garantiza que cualquier registro *off-chain* puede ser verificado contra la *blockchain* en tiempo constante. El campo `ipfsCid` almacena el identificador de contenido (CID) devuelto por Pinata tras la subida del artículo a IPFS; su carácter opcional (`String?`) permite que registros creados por el indexador a partir de eventos on-chain queden sin CID hasta que el frontend complete el paso de escritura en la API.

Esquema Prisma (schema.prisma)

```
1 datasource db {
2   provider = "postgresql"
3   url      = env("DATABASE_URL")
4 }
5
6 generator client {
7   provider = "prisma-client-js"
8 }
9
10 model Publication {
11   id          Int           @id @default(autoincrement())
12   contentHash String        @unique
13   title       String
14   body        String
15   authorAddress String
16   tags        String[]
17   ipfsCid     String?
18   createdAt   DateTime     @default(now())
19   validations Validation[]
20   reopenRequests ReopenRequest[]
21
22   @@map("publications")
23 }
24
25 model Validation {
26   id          Int           @id @default(autoincrement())
27   contentHash String
28   validatorAddress String
29   vote        String
30   round       Int          @default(0)
31   txHash      String?
```

```

32   createdAt      DateTime   @default(now())
33   publication     Publication @relation(fields: ..., references: ...)
34
35   @@unique([contentHash, validatorAddress])
36   @@map("validations")
37 }
38
39 model Validator {
40   address      String   @id
41   reputationScore Int    @default(0)
42   lastSyncBlock BigInt  @default(0)
43   updatedAt    DateTime @updatedAt
44
45   @@map("validators")
46 }

```

La restricción `@@unique([contentHash, validatorAddress])` en el modelo `Validation` impide que el backend registre más de un voto por validador y publicación, replicando la invariante del contrato. El campo `round` registra en qué ronda de votación se emitió cada voto, permitiendo al indexador reconstruir el historial multironda.

Los modelos `ReopenRequest` y `RetroactiveClaim` persisten off-chain los eventos correspondientes a `requestReopen` y `claimRetroactiveReputation`, con restricciones de unicidad que replican las del contrato:

Modelos adicionales en `schema.prisma`

```

1 model ReopenRequest {
2   id          Int          @id @default(autoincrement())
3   contentHash String
4   requesterAddress String
5   txHash      String?
6   createdAt   DateTime     @default(now())
7   publication  Publication  @relation(fields: ..., references: ...)
8
9   @@unique([contentHash, requesterAddress])
10  @@map("reopen_requests")
11 }
12
13 model RetroactiveClaim {
14   id          Int          @id @default(autoincrement())
15   contentHash String
16   validatorAddress String
17   netDelta    Int
18   txHash      String?
19   createdAt   DateTime     @default(now())
20
21   @@unique([contentHash, validatorAddress, txHash])
22   @@map("retroactive_claims")
23 }

```

El perfil enriquecido, los favoritos y las notificaciones (RD 17–RD 22) no tienen contrapartida *on-chain*: son datos exclusivamente *off-chain* que el usuario gestiona directamente, sin pasar por el indexador de eventos.

Modelos sin contrapartida on-chain (schema.prisma)

```

1 model UserProfile {
2   address      String   @id
3   displayName  String?
4   avatarUrl    String?
5   email        String?
6   updatedAt    DateTime @updatedAt
7
8   @@map("user_profiles")
9 }
10
11 model Favorite {
12   id           Int       @id @default(autoincrement())
13   userAddress  String
14   contentHash  String
15   createdAt    DateTime @default(now())
16
17   @@unique([userAddress, contentHash])
18   @@map("favorites")
19 }
20
21 model Notification {
22   id           Int       @id @default(autoincrement())
23   userAddress  String
24   contentHash  String
25   type         String    // REOPENED | CONSENSUS_REACHED |
26                 ↳ RETROACTIVE_APPLIED
27   read         Boolean   @default(false)
28   createdAt    DateTime @default(now())
29
30   @@map("notifications")
31 }
```

4.4.3. Sincronización on-chain → off-chain

El backend mantiene la base de datos *off-chain* sincronizada con el estado *on-chain* mediante un indexador de eventos. El indexador utiliza la función `watchContractEvent` de la biblioteca `viem` para suscribirse en tiempo real a los tres eventos relevantes de los contratos:

- **PublicationRegistered**: cuando se emite este evento, el indexador crea o actualiza el registro correspondiente en la tabla `publications` de PostgreSQL, actualizando el campo `authorAddress` con la dirección del autor registrada en cadena.
- **ValidationSubmitted**: el indexador registra el nuevo voto en la tabla `validations`, incluyendo el `txHash` de la transacción para trazabilidad.

- **ReputationUpdated:** el indexador actualiza el campo `reputationScore` del validador correspondiente en la tabla `validators` y registra el `lastSyncBlock` del evento.

Este diseño garantiza que el backend puede reconstruirse completamente desde cero relanzando el indexador contra el historial de eventos *on-chain*, desde el bloque de despliegue de los contratos hasta el bloque actual. La base de datos *off-chain* es, en todo momento, una vista derivada del estado *on-chain*, no una fuente de verdad independiente (Mazzocca et al., 2025).

4.5. Diseño de la API REST

4.5.1. Arquitectura de capas del backend

El backend se organiza en tres capas internas con responsabilidades bien delimitadas:

- **Capa de enrutamiento (Router Hono):** recibe las peticiones HTTP, valida los parámetros de entrada y delega la lógica de negocio a la capa de servicios. No accede directamente a la base de datos.
- **Capa de servicios (Services):** contiene la lógica de negocio: cálculo de hashes, orquestación de consultas, transformación de datos entre el formato de la *blockchain* (hex, wei, etc.) y el formato de la API (JSON, strings). Tampoco accede directamente a la base de datos.
- **Capa de repositorios (Repositories / Prisma):** encapsula todos los accesos a PostgreSQL. Cada repositorio expone métodos tipados en TypeScript para las operaciones CRUD de cada modelo, evitando que la lógica SQL se disperse por el código.

Esta separación facilita las pruebas unitarias de la capa de servicios mediante la inyección de repositorios simulados (*mocks*), sin necesidad de una base de datos real.

 **Distinción crítica:** escrituras on-chain vs off-chain Las operaciones de escritura en la *blockchain* (`registerPublication`, `submitValidation`) las ejecuta el **frontend** directamente contra los contratos, utilizando la cartera del usuario como firmante. El backend **no firma transacciones** en nombre del usuario: no tiene acceso a su clave privada y no debe tenerlo. El rol del backend es únicamente indexar los eventos resultantes de esas transacciones y servir el contenido *off-chain* a través de la API.

4.5.2. Endpoints de la API

Los endpoints de la API REST se agrupan en tres recursos principales: publicaciones, validadores y sincronización interna.

GET **/api/v1/publications**

Lista publicaciones con filtros opcionales por etiqueta, autor y estado de validación. Soporta paginación mediante los parámetros `page` y `limit`.

GET **/api/v1/publications/:hash**

Devuelve el detalle completo de una publicación identificada por su `contentHash`: cuerpo del artículo, autor, timestamp, lista de validaciones y estado de consenso actual.

POST **/api/v1/publications**

Registra el contenido completo de una publicación ya registrada on-chain. El cliente envía el cuerpo del artículo; el backend verifica que el hash del cuerpo recibido coincide con el `contentHash` indicado (que debe existir en la blockchain) y lo almacena en PostgreSQL.

GET **/api/v1/validators/:address**

Devuelve el perfil de un validador: dirección, puntuación de reputación (sincronizada desde on-chain), número total de validaciones y porcentaje de aciertos sobre el consenso.

GET **/api/v1/validators/:address/history**

Devuelve el historial de validaciones emitidas por un validador, con el resultado de cada voto, si coincidió con el consenso final y el impacto en su reputación.

GET **/api/v1/validators**

Lista los validadores registrados ordenados por puntuación de reputación descendente. Útil para el panel de ranking de la interfaz.

POST

/api/v1/sync/events

Dispara manualmente una re-sincronización del indexador desde el último bloque procesado hasta el bloque actual. Uso interno; protegido mediante autenticación de servicio (Authorization: Bearer con token de servicio).

4.6. Diseño de la interfaz de usuario

4.6.1. Identidad visual

La identidad visual de NewsEra se diseñó para transmitir cuatro valores concretos: **confianza**, **transparencia**, **accesibilidad** y **progreso**. Estos valores condicionan tanto la paleta de color como el concepto del logotipo, y se aplican de forma deliberadamente minimalista: el objetivo es que la atención del usuario recaiga sobre el contenido de los artículos, no sobre la propia interfaz.

Paleta de color

La paleta parte de una base neutra (blanco y una escala de grises) sobre la que se añade un único color de marca y una paleta funcional reducida, reservada exclusivamente para comunicar el estado de consenso de un artículo. Esta separación es intencional: el color de marca se usa solo en navegación y llamadas a la acción, mientras que los colores de estado nunca comparten superficie con él, evitando que la interfaz compita visualmente consigo misma.

Logotipo

El logotipo es un monograma geométrico que combina las iniciales **N** y **E** de NewsEra. La barra horizontal central de la E se redibuja sutilmente como una marca de verificación (*checkmark*), vinculando visualmente la identidad de la marca con su propósito —la verificación comunitaria de contenido— sin recurrir a un icono figurativo adicional. Un ligero ángulo de inclinación hacia la derecha en el trazo de la E sugiere avance y progreso. El resultado es una marca reducible a tamaños pequeños (favicon, icono de aplicación), en un único color sólido (el azul de marca de la Tabla 4.5), sin gradientes, sombras ni efectos tridimensionales, alineada con el minimalismo del resto de la interfaz.

El logotipo de la Figura 4.4 se generó con Gemini (Google) a partir del siguiente *prompt*, redactado para materializar el concepto descrito arriba en un único color y sin elementos decorativos:

“Minimalist flat vector logo mark, the letters ‘N’ and ‘E’ merged into a single geometric monogram, bold clean sans-serif geometric letterforms, the crossbar of the E subtly redrawn as a checkmark tick to suggest verification and trust, slight forward-leaning slant on the E to suggest progress and forward motion, single solid cobalt blue color (#2563EB) on a transparent or white background, no

Tabla 4.5: Paleta de color de la identidad visual de NewsEra.

Uso	Color	Justificación
Acento de marca (logotipo, enlaces, navegación activa)	Azul cobalto #2563EB	El azul es, de forma consistente en branding institucional y financiero, el color asociado a la confianza y la estabilidad; evita además el rojo/naranja característico de los sitios sensacionalistas que el proyecto busca diferenciarse de
Fondo y texto base	Blanco #FFFFFF / gris oscuro #09090B	Máximo contraste y mínimo ruido visual, coherente con el objetivo de mantener el foco en el artículo
Texto secundario y bordes	Escala de grises neutros	Jerarquía visual sin introducir color adicional
Estado de consenso TRUE	Verde esmeralda #059669	Codificación funcional, no decorativa: permite reconocer el estado de un artículo de un vistazo
Estado de consenso FALSE	Rojo #DC2626	Uso puntual y acotado a una etiqueta pequeña, nunca como superficie grande, para no reproducir la estética de alarma permanente de los sitios de desinformación
Estado de consenso UNVERIFIABLE / PENDING	Ámbar #D97706	Color de alerta neutro, sin connotación positiva ni negativa



Figura 4.4: Logotipo de NewsEra: monograma NE con marca de verificación integrada, generado con Gemini a partir del *prompt* de generación de imagen que se reproduce a continuación.

gradients, no shadows, no 3D effects, no decorative elements, extremely simple and reducible to a small app icon / favicon size, flat design, vector illustration style, generous negative space, symmetric balanced composition, tech startup branding, trustworthy and modern feel”

4.6.2. Estructura de la aplicación React

La aplicación se estructura como una SPA React con React Router v6; las rutas gestionadas se detallan en RI 1 (§4.1.5).

Toda la interacción con la blockchain usa *hooks* de wagmi: `useReadContract` para lecturas, `useWriteContract` para transacciones y `useWaitForTransactionReceipt` para esperar la confirmación en cadena. El componente de conexión de cartera (RainbowKit) se mantiene en el layout raíz y expone la dirección activa al resto de la aplicación mediante el contexto de wagmi.

4.6.3. Flujos de interacción principales

Los flujos de conexión de cartera, publicación y emisión de voto se documentan como casos de uso detallados en la Sección 4.1.2 (UC 1, UC 19 y UC 35 respectivamente) y se integran en una única narrativa completa, de extremo a extremo, en la Sección 4.7. El panel de reputación combina lectura directa *on-chain* (`ReputationSystem.getReputation(address)` vía `useReadContract`) con la consulta del historial detallado al backend (`GET /api/v1/validators/:address`) equilibrando autenticidad y eficiencia (UC 6, UC 7).

Toda operación de escritura en la *blockchain* comparte el mismo patrón de gestión de estados, descrito a continuación.

■ Gestión de estados en operaciones con blockchain Cada operación de escritura en la *blockchain* —registro de publicación, emisión de voto— requiere que el usuario confirme una transacción en su cartera y espera la inclusión de esa transacción en un bloque (habitualmente en torno a 12 segundos en Sepolia Ethereum Foundation, 2022). El diseño de la UX debe gestionar explícitamente los siguientes estados para cada operación:

1. **Idle:** estado inicial, sin transacción en curso.
2. **Pending:** el usuario ha iniciado la operación; se espera su confirmación en la cartera.
3. **Confirming:** la transacción ha sido firmada y enviada a la red; se espera su inclusión en un bloque.
4. **Confirmed:** la transacción ha sido incluida en un bloque; la operación se ha completado con éxito.
5. **Failed:** la transacción ha sido rechazada (por el usuario, por la red o por el contrato).

Los *hooks* `useWriteContract` y `useWaitForTransactionReceipt` de wagmi v2 exponen directamente estos estados, simplificando su manejo en los componentes React.

4.7. Flujo de publicación y validación

Esta sección describe el flujo completo e integrado del ciclo de vida de una publicación en NewsEra, desde que un usuario escribe un artículo hasta que recibe un veredicto de consenso por parte de la comunidad de validadores. Este flujo integra las tres capas del sistema y demuestra el cumplimiento de los objetivos OE3 y OE4.

1. **El usuario conecta su cartera.** El usuario accede a la interfaz de NewsEra y conecta su cartera Ethereum mediante RainbowKit. wagmi registra la dirección activa y la pone a disposición de todos los Client Components.
 2. **El frontend calcula el hash del artículo.** El usuario escribe el cuerpo del artículo en el formulario de publicación. El frontend aplica la función keccak256 de viem sobre la representación UTF-8 del texto y obtiene un valor de 32 bytes (**bytes32**) que identifica de forma única ese contenido exacto. Cualquier modificación posterior al texto, aunque sea de un solo carácter, producirá un hash completamente diferente.
 3. **El usuario registra el hash en la blockchain.** Mediante el *hook* `useWriteContract`, el frontend invoca `PublicationRegistry.registerPublication(contentHash)`. El usuario confirma la transacción en su cartera. El contrato verifica que el hash no ha sido registrado previamente; si es así, almacena la tripla (`contentHash`, `author`, `timestamp`) y emite el evento `PublicationRegistered`.
 4. **El frontend espera la confirmación on-chain.** Mediante el *hook* `useWaitForTransactionReceipt`, el frontend espera la inclusión de la transacción en un bloque. La interfaz muestra el estado *Confirming...* al usuario durante esta espera.
 5. **El frontend envía el contenido al backend.** Una vez confirmada la transacción, el frontend envía el título, el cuerpo y las etiquetas del artículo al endpoint `POST /api/v1/publications` del backend. El backend verifica que el hash del cuerpo recibido coincide con el `contentHash` indicado y que ese hash existe en la *blockchain*. Si ambas verificaciones son correctas, almacena la publicación en PostgreSQL.
 6. **El indexador del backend procesa el evento.** En paralelo, el indexador del backend detecta el evento `PublicationRegistered` emitido por el contrato, actualiza el registro de la publicación en PostgreSQL con los datos *on-chain* (autor y timestamp confirmados) y pone la publicación en estado *pendiente de validación*.
 7. **Los validadores ven la publicación.** La publicación aparece en la lista de artículos pendientes de validación para todos los validadores registrados con reputación suficiente. La lista se sirve mediante el endpoint REST del backend.
 8. **Un validador lee el artículo y emite su voto.** El validador accede a la página de detalle de la publicación, lee el contenido completo y decide su voto. El frontend verifica, mediante una llamada de lectura a `ReputationSystem.canValidate(validatorAddress)`, que el validador tiene reputación mínima suficiente. A continuación lanza la transacción `ValidationRegistry.submitValidation(contentHash, vote)`.
-

9. **El contrato registra el voto y comprueba el quórum.** `ValidationRegistry` almacena el voto del validador. Si el número total de votos alcanza el quórum configurado (`quorumThreshold`), el contrato calcula el consenso por mayoría entre votos `TRUE` y `FALSE` y emite el evento `ConsensusReached`.
10. **El contrato actualiza las reputaciones.** Inmediatamente tras calcular el consenso, `ValidationRegistry` llama a `ReputationSystem.increaseReputation` para cada validador que votó en la dirección del consenso ganador y a `decreaseReputation` para cada validador que votó de forma diferente al consenso — independientemente de si emitieron `TRUE`, `FALSE` o `UNVERIFIABLE`. El tipo `UNVERIFIABLE` es un veredicto de primera clase: los validadores que lo emiten son recompensados si ese tipo gana la supermayoría, y penalizados en caso contrario. El contrato emite el evento `ReputationUpdated` para cada dirección modificada.
11. **El indexador sincroniza el estado final.** El indexador del backend detecta los eventos `ValidationSubmitted`, `ConsensusReached` y `ReputationUpdated`, actualiza los registros correspondientes en PostgreSQL y marca la publicación con el veredicto de consenso.
12. **El resultado es visible en la interfaz.** La página de detalle de la publicación muestra el veredicto de consenso, la lista completa de validadores que participaron, sus votos y el impacto en sus reputaciones. Todo este historial es verificable de forma independiente por cualquier usuario consultando directamente los eventos de la *blockchain*.

La Tabla 4.6 resume los actores, contratos y eventos involucrados en el flujo completo.

Tabla 4.6: Actores, contratos y eventos en el flujo de publicación y validación.

Paso	Actor	Acción	Contrato / Evento
1–2	Usuario (frontend)	Conecta cartera; calcula hash	wagmi / viem
3	Usuario (frontend)	Registra hash on-chain	PublicationRegistry
3	Contrato	Emite evento de registro	PublicationRegistered
5	Usuario (frontend)	Envía contenido al backend	POST /publications
6	Indexador (backend)	Procesa evento; actualiza BD	PostgreSQL
8	Validador (frontend)	Emite voto on-chain	ValidationRegistry
9	Contrato	Registra voto; calcula quórum	ValidationSubmitted
9	Contrato	Emite consenso si hay quórum	ConsensusReached
10	Contrato	Actualiza reputaciones	ReputationSystem
10	Contrato	Emite eventos de reputación	ReputationUpdated
11	Indexador (backend)	Sincroniza estado final	PostgreSQL

5. Resultados

Este capítulo presenta los resultados obtenidos durante el desarrollo del prototipo de NewsEra. Los datos cuantitativos —direcciones de contratos, costes de gas, cobertura de tests y métricas de código— corresponden al estado del repositorio en Sprint 4 y se actualizan automáticamente desde el repositorio de implementación mediante el flujo de CI de *GitHub Actions* descrito en el §4.4.3. Esto garantiza que los valores reflejados en el documento siempre sean coherentes con el código desplegado, sin requerir edición manual del texto.

5.1. Resumen del prototipo implementado

El prototipo de NewsEra ha sido desarrollado de forma iterativa a lo largo de una serie de sprints de dos semanas. El estado que se evalúa en este capítulo corresponde al Sprint 4. El despliegue público en la red Sepolia está previsto para el Sprint 9.

El prototipo cubre en su estado actual la capa *on-chain*: los tres contratos inteligentes escritos en Solidity (**PublicationRegistry**, **ValidationRegistry** y **ReputationSystem**) han sido implementados y verificados con pruebas unitarias sobre *Hardhat Network*. La capa *off-chain* (API REST sobre Node.js/Hono y Prisma) y la interfaz de usuario (React/Vite con wagmi y RainbowKit) están diseñadas y planificadas en los Sprints 7 y 8 respectivamente, pero no forman parte del alcance de evaluación del presente capítulo.

En conjunto, los contratos implementados codifican los tres flujos fundamentales del sistema: el registro inmutable de publicaciones, el proceso de validación comunitaria con gobernanza multironda, y el cálculo y actualización de la reputación de los validadores.

5.2. Resultados de los contratos inteligentes

5.2.1. Despliegue en Sepolia

Los tres contratos han sido verificados sobre *Hardhat Network* durante el período de evaluación cubierto por este capítulo. El despliegue público en la red de pruebas Sepolia está previsto al concluir el Sprint 9; las direcciones de la Tabla 5.1 se actualizarán automáticamente mediante CI en ese momento.

Tabla 5.1: Direcciones de despliegue previstas en Sepolia (pendiente Sprint 9).

Contrato	Dirección
PublicationRegistry	pendiente
ValidationRegistry	pendiente
ReputationSystem	pendiente

Una vez completado el despliegue en Sepolia, las transacciones serán auditables en cualquier explorador de bloques compatible con Ethereum. El diseño de cada contrato se describe en el §4.3.

5.2.2. Análisis de coste de gas

El análisis de coste de gas de las operaciones principales está planificado para el Sprint 9 (**hardhat-gas-reporter**, HU-9.4, en el despliegue en Sepolia). La Tabla 5.2 recoge las operaciones a evaluar; los valores se actualizarán automáticamente mediante el flujo de CI al concluir dicha fase.

Tabla 5.2: Coste de gas de las operaciones principales de los contratos.

Operación	Contrato	Gas consumido
registerPublication	PublicationRegistry	—
submitValidation	ValidationRegistry	—
requestReopen	ValidationRegistry	—
claimRetroactiveReputation	ValidationRegistry	—
increaseReputation	ReputationSystem	—
decreaseReputation	ReputationSystem	—
Coste medio		—

El umbral habitualmente aceptado en la literatura para considerar una operación *eficiente* en Ethereum es de 100 000 gas (Buterin, 2014). El diseño de los contratos está orientado a mantenerse por debajo de este umbral en las operaciones de escritura; la verificación empírica de este objetivo está prevista al completar el Sprint 9, cuando se configure **hardhat-gas-reporter** (HU-9.4). No obstante, el coste real en moneda fiat depende del precio del gas en el momento de la transacción, variable que no puede garantizarse en la red principal. Esta cuestión se analiza con mayor detalle en el §5.7.1.

5.2.3. Cobertura de tests

La suite de tests de los contratos inteligentes alcanzó una cobertura de 96,25%, con 54 tests superados sobre un total de 54 casos de prueba. El objetivo mínimo fijado en la planificación del proyecto era del 80%, por lo que el resultado obtenido cumple el criterio establecido.

Los tests cubren los flujos principales de cada contrato (§4.3.1, §4.3.2 y §4.3.3), incluyendo casos de error, condiciones de frontera y comprobaciones de acceso basadas en reputación. Los tests de integración del contrato **ReputationSystem** son los más extensos, dado que la lógica de actualización de reputación implica múltiples transiciones de estado y eventos dependientes.

5.3. Métricas de implementación

La Tabla 5.3 recoge las líneas de código efectivas por capa del sistema, excluyendo comentarios, líneas en blanco y archivos de configuración generados automáticamente.

Tabla 5.3: Líneas de código por capa del sistema (Sprint 4).

Capa	Tecnología	Líneas de código (LOC)
Contratos inteligentes	Solidity	—
API <i>off-chain</i>	Node.js / TypeScript / Hono	—
Interfaz de usuario	React / Vite	—

La distribución de LOC refleja el carácter del prototipo: la capa de contratos inteligentes es relativamente compacta en términos absolutos, lo que es coherente con la filosofía de mantener en cadena exclusivamente la lógica que requiere inmutabilidad y trazabilidad, delegando el resto al *backend off-chain*. El grueso del código reside en la API y en el *frontend*, que implementan los flujos de interacción con el usuario y la integración con los contratos.

5.4. Evaluación del cumplimiento de objetivos

La Tabla 5.4 evalúa el grado de cumplimiento de cada objetivo específico definido en el §1.5, a la luz de los resultados del prototipo en Sprint 4.

Tabla 5.4: Grado de cumplimiento de los objetivos específicos del trabajo.

Obj.	Descripción resumida	Estado
OE1	Análisis del estado del arte	Completado — revisión bibliográfica desarrollada en el Capítulo 2.
OE2	Diseño de la arquitectura del sistema	Completado — arquitectura global definida en el §4.3, §4.5 y §4.6.
OE3	Publicación y verificación descentralizada	Completado — contratos PublicationRegistry y ValidationRegistry implementados y desplegados; ver §4.3.1 y §4.3.2.
OE4	Reputación y gobernanza de validadores	Completado — contrato ReputationSystem implementado con mecanismo de actualización de reputación basado en historial; ver §4.3.3.
OE5	Prototipo de interfaz de usuario	En desarrollo — diseño de la interfaz React/Vite documentado en §4.6; implementación prevista en el Sprint 8.
OE6	Integración de herramientas de IA	Opcional — no implementado en el alcance del prototipo actual. Identificado como línea de trabajo futuro.
OE7	Evaluación de viabilidad del prototipo	Completado — análisis desarrollado en el presente capítulo (§5.7) y en las conclusiones (Capítulo 6).

Los objetivos OE1–OE4 han sido completados en el periodo de implementación contemplado por este capítulo. OE5 (interfaz de usuario) está en desarrollo y se concluirá en el Sprint 8. El objetivo opcional OE6 queda fuera del alcance acordado, tal como se recoge en la planificación del §1.5. La evaluación de viabilidad (OE7) se desarrolla en la sección siguiente y se complementa con las conclusiones del trabajo.

5.5. Resultados de los experimentos de validación con usuarios

Esta sección recoge los resultados de los dos experimentos de validación diseñados en el §3.6.5 y cuyo instrumento completo se detalla en el Anexo B. Los datos piloto sintéticos generados durante el desarrollo (§B.4) quedan excluidos de los resultados aquí presentados.

5.5.1. Experimento 1: intensidad del problema

[PENDIENTE — placeholder a la espera de los resultados reales]

Tamaño muestral: $n = \dots$ respuestas reales recogidas.

Resultado sobre el criterio de gravedad (ítem 6): ... % de la muestra califica el problema con 4 o 5 (grave o muy grave) — criterio de éxito: $\geq 60\%$.

Resultado sobre el criterio de método fiable (ítem 7): ... % responde “No” o “No estoy seguro/a” — criterio de éxito: $\geq 50\%$.

Conclusión: [hipótesis validada / no validada / validada parcialmente], con el razonamiento correspondiente y, si procede, el contraste con los ítems 1–5 (contexto cualitativo sobre percepción de captura y manipulación de los canales actuales).

5.5.2. Experimento 2: idoneidad del producto

[PENDIENTE — placeholder a la espera de los resultados reales]

Tamaño muestral: $n = \dots$ respuestas reales recogidas.

Puntuación SUS media (Bloque A): ... sobre 100 — criterio de éxito: ≥ 68 .

Intención de uso (ítem 15): ... % responde “Sí” (... % adicional responde “Tal vez”, reportado por separado) — criterio de éxito: $\geq 60\%$ de respuestas “Sí”.

Confianza en las propiedades arquitectónicas (ítems 11–13): medias de inmutabilidad/descentralización, consenso colectivo y reputación P2P, respectivamente.

Fricción de la interacción Web3 (ítem 14): media del ítem, contrastada con los ítems 11–13 para diagnosticar si un resultado desfavorable proviene de la arquitectura o de la fricción de firma de transacciones (ver razonamiento en el Anexo B, §B.3).

Hallazgos cualitativos (ítem 16, respuesta abierta): temas recurrentes identificados en los comentarios libres.

Conclusión: [hipótesis validada / no validada / validada parcialmente], con implicaciones para el trabajo futuro (§6.4) si la fricción Web3 resulta ser la barrera dominante frente a la arquitectura del sistema.

5.6. Consideraciones de diseño: evolución del sistema

Esta sección documenta las decisiones de diseño que evolucionaron a lo largo del proceso de desarrollo del prototipo. Para cada una se describe el diseño inicial, el problema que motivó el cambio y la solución adoptada en la versión final. El objetivo es proporcionar trazabilidad del razonamiento de diseño sin mezclar la narrativa de evolución con la descripción técnica del sistema, que figura en el Capítulo 4 en su forma definitiva.

5.6.1. De abstención a veredicto: el tipo UNVERIFIABLE

Diseño inicial. El tipo de voto UNVERIFIABLE fue concebido originalmente como una *abstención*: los votos de este tipo no influirían en el cómputo del consenso ni acarrearían consecuencias reputacionales. La lógica era que si un validador no puede determinar la veracidad, su opinión no debería inclinar la balanza.

Problema identificado. Este enfoque introduce un fallo de incentivos grave: sin riesgo asociado, UNVERIFIABLE se convierte en la opción dominante para cualquier validador adversarial o pasivo. Un actor interesado en bloquear el consenso sobre un artículo puede hacerlo acumulando votos UNVERIFIABLE sin coste reputacional alguno. Del mismo modo, un validador que prefiere no emitir un juicio difícil simplemente vota UNVERIFIABLE y evita la penalización, lo que vacía de significado la participación (Lesaege et al., 2019).

Solución adoptada. UNVERIFIABLE se elevó a **veredicto de primera clase**: participa en el cómputo del consenso con el mismo peso que TRUE o FALSE, puede constituir el resultado definitivo de una ronda si obtiene supermayoría, y votar UNVERIFIABLE sobre un artículo que la comunidad luego califica como claramente verdadero o falso acarrea la misma penalización que haber votado erróneamente. Esto elimina el incentivo a usarlo como refugio de riesgo cero.

5.6.2. De única ronda a sistema multironda con reapertura

Diseño inicial. El sistema de validación fue concebido como de *única ronda*: una vez que una votación alcanzaba el quórum y producía un resultado (definitivo o disputado), el proceso quedaba permanentemente cerrado. Ningún validador podía revisar ese veredicto, independientemente de nueva evidencia disponible.

Problema identificado. El diseño de única ronda es inconsistente con el principio de Adler y de Alfaro (2007) que el propio marco teórico del trabajo adopta como fundamento del sistema de reputación: la calidad de un juicio se mide por su capacidad de resistir el escrutinio de nuevos participantes independientes a lo largo del tiempo. Si el consenso queda bloqueado tras la primera votación, ese principio se convierte en letra muerta. Además, la experiencia de sistemas de verificación de hechos reales muestra que la información disponible sobre un artículo puede cambiar: surgen pruebas nuevas, se corrige contexto o se detectan manipulaciones no visibles en el momento original de la votación.

También se consideró mantener la votación *permanentemente abierta* —cualquier validador podría votar en cualquier momento—, pero este diseño introduce el problema del *free rider*: un validador puede esperar a ver hacia dónde se dirige el consenso antes de votar, eliminando el valor informativo del juicio independiente.

Solución adoptada. Se adoptó un **mecanismo de reapertura por solicitudes**: la votación se cierra al alcanzar el quórum (como en el diseño original), pero cualquier validador con reputación suficiente que aún no haya votado ese artículo puede registrar una solicitud de reapertura. Cuando se acumulan `reopenThreshold` solicitudes, el contrato abre una nueva ronda de votación independiente. Los votos de cada ronda son inmutables: no pueden modificarse una vez emitidos, lo que preserva el valor informativo de cada juicio. El número de rondas posibles no está limitado.

5.6.3. De penalización simple a reputación retroactiva

Diseño inicial. Los efectos reputacionales se aplicaban únicamente al cierre de cada ronda y afectaban exclusivamente a los participantes de esa ronda: los que votaron la opción ganadora recibían recompensa; los que votaron otra opción recibían penalización. El historial de un validador no se veía afectado por rondas posteriores.

Problema identificado. El sistema de única ronda con efectos inmediatos no implementa el principio de supervivencia del veredicto: un validador que acertó en la primera ronda de un artículo pero que rondas posteriores contradicen no recibe ninguna señal de que su juicio original resultó equivocado a la luz de nueva evidencia. Del mismo modo, un validador que votó erróneamente en una ronda temprana no se beneficia si rondas posteriores le dan la razón. La reputación queda desconectada de la evolución del conocimiento colectivo sobre cada artículo.

Solución adoptada. Se diseñó un sistema de **reputación retroactiva** basado en el modelo *pull*: cada validador puede llamar a `claimRetroactiveReputation` para recibir ajustes de ± 1 punto por cada ronda DEFINITIVE posterior a la suya que confirme o contradiga el resultado de su ronda original, con un cap de ± 3 por artículo. Se eligió el modelo *pull* —en lugar de aplicar los ajustes automáticamente al cerrar cada ronda— para evitar que el coste de gas en `_checkConsensus` escalara de forma proporcional al número total de participantes históricos de un artículo. El validador activa el cálculo cuando lo desee, pagando únicamente el gas de sus propios ajustes.

5.6.4. De voto ponderado a voto plano con acceso gated

Diseño considerado. Una alternativa analizada es el **voto ponderado por reputación**: que el voto de un validador con alta reputación acumulada compute con más peso que el de uno recién llegado. El argumento intuitivo es que un historial de aciertos es señal de mejor criterio, y que ese criterio debería tener mayor influencia en el consenso.

Problema identificado. El voto ponderado destruye la propiedad epistémica central del sistema: la **independencia de los votos**. En un sistema de voto plano, cada validador tiene incentivo para evaluar el artículo con su propio criterio, porque su voto computa igual que el de cualquier otro. En un sistema ponderado, la estrategia racional pasa a ser observar hacia dónde se dirigen los validadores de alta reputación y sumarse a ellos antes de que el consenso cierre, ya que la probabilidad de obtener la recompensa depende más de alinearse con los actores dominantes que de emitir un juicio correcto. El resultado es un **comportamiento de manada** que convierte el sistema en un mecanismo de ratificación del criterio de una élite establecida, en lugar de una agregación de juicios independientes.

Este problema se agrava en combinación con el mecanismo de reapertura: los validadores de rondas iniciales que acumularon alta reputación tendrían mayor capacidad para resistir el desafío de nuevas rondas, haciendo el sistema progresivamente más resistente a la corrección cuando más evidencia nueva existe.

Además, la resistencia Sybil que el voto ponderado proporcionaría ya está cubierta por el umbral mínimo de reputación para acceder al rol de validador. No hay ventaja de seguridad que justifique los efectos oligárquicos del voto ponderado.

Solución adoptada. Se adoptó **voto plano con acceso *gated***: todos los votos de validadores acreditados computan igual, preservando la independencia de cada juicio. Para un hipotético escalado a una comunidad de validadores muy amplia, la **votación cuadrática** —peso proporcional a la raíz cuadrada de la reputación— constituye una evolución natural que reduce la dominancia de los grandes acumuladores sin eliminarla por completo; esta línea queda identificada como trabajo futuro.

5.7. Discusión de resultados

5.7.1. Viabilidad técnica del mecanismo de reputación

Los costes de gas medidos en la Tabla 5.2 indican que las operaciones del `ReputationSystem` son técnicamente viables en la red de pruebas Sepolia. Sin embargo, la extrapolación a la red principal de Ethereum requiere considerar varios factores que los datos del prototipo no pueden anticipar con certeza.

En primer lugar, el precio del gas en la red principal fluctúa significativamente en función de la demanda de transacciones, pudiendo oscilar entre unos pocos *gwei* en periodos de baja actividad y varios cientos de *gwei* en picos de congestión. Para una operación con un coste de — unidades de gas, el coste en moneda fiat podría oscilar entre fracciones de céntimo y varios euros dependiendo de las condiciones de red, lo que plantea incertidumbre sobre la accesibilidad del sistema para usuarios con recursos limitados.

En segundo lugar, la cadena de actualizaciones de reputación derivada de una sesión activa de validaciones puede generar varias transacciones por participante en un periodo corto. Si el sistema alcanzara un número elevado de validadores activos simultáneos, el coste agregado de las transacciones de reputación podría convertirse en una barrera de adopción. Una posible mitigación es el uso de soluciones de *layer 2* sobre Ethereum —como Optimism o Arbitrum— que reducen los costes de gas en uno o dos órdenes de magnitud manteniendo las garantías de seguridad de la cadena principal. Esta línea queda identificada como trabajo futuro.

En conjunto, los resultados obtenidos demuestran que el mecanismo de reputación es *técnicamente viable* para un prototipo funcional con un número moderado de participantes, y que su escalabilidad a producción requiere una estrategia de despliegue en *layer 2* o una revisión del modelo de cobros que permita subsidiar las transacciones de reputación para los validadores activos.

5.7.2. Limitaciones observadas durante la implementación

A lo largo de los sprints de desarrollo se identificaron las siguientes limitaciones e impedimentos técnicos relevantes para la evaluación del prototipo:

- **Alcance acotado a la capa de contratos.** Los sprints de implementación completados cubren los tres contratos inteligentes (`PublicationRegistry`, `ValidationRegistry` y `ReputationSystem`) con su suite de pruebas automatizadas. Las capas de backend (API REST e indexador) y frontend forman parte del alcance del prototipo pero se encuentran en fases de desarrollo posteriores al período de evaluación recogido en este capítulo.
 - **Cobertura de pruebas limitada a la capa on-chain.** Las métricas de cobertura reflejan exclusivamente los contratos inteligentes. La validación end-to-end del sistema completo —incluyendo la comunicación *backend–blockchain* y la experiencia de usuario en el frontend— queda fuera del perímetro de evaluación actual.
 - **Ausencia de auditoría de seguridad profesional.** Los contratos no han sido sometidos a una auditoría externa. El análisis estático realizado con Slither (Sprint 5) cubre la verificación automatizada, pero no sustituye una revisión manual por expertos en seguridad de contratos inteligentes, imprescindible antes de cualquier despliegue en red principal.
 - **Entorno de pruebas local.** Toda la evaluación se realizó sobre *Hardhat Network*, red local que simula el comportamiento de la EVM sin las condiciones reales de una *testnet* pública (latencia de bloques variable, competencia por gas, censura de mempool).
-

6. Conclusiones

Este capítulo sintetiza los resultados del presente TFG sobre el diseño e implementación de NewsEra, una plataforma descentralizada para la validación y difusión de información veraz mediante tecnología *blockchain*. El trabajo ha completado el análisis del estado del arte, el diseño de la arquitectura del sistema y la implementación de la capa *on-chain* del prototipo — los tres contratos inteligentes que gobiernan el registro, la validación comunitaria y la reputación de validadores — tal como se recoge en el Capítulo 5. El *backend* y la interfaz de usuario están diseñados y planificados en los Sprints 6 y 7. Las conclusiones que se exponen a continuación se derivan de los resultados verificables del prototipo desarrollado y del análisis técnico realizado a lo largo del trabajo.

6.1. Conclusiones principales

El análisis realizado a lo largo del presente trabajo permite extraer las siguientes conclusiones:

1. **La viabilidad técnica del diseño propuesto está respaldada por el estado del arte.** La revisión de la literatura científica y técnica realizada en el marco de OE1 confirma que todos los componentes del diseño de NewsEra tienen precedentes funcionales: los contratos inteligentes sobre plataformas compatibles con la EVM soportan lógica de gobernanza de la complejidad requerida (Buterin, 2014); los sistemas de identidad descentralizada estandarizados por la W3C proporcionan un mecanismo de acreditación de identidad sin dependencia de entidades centrales (World Wide Web Consortium, 2022); y los protocolos de resolución de disputas en cadena, como Kleros, demuestran la viabilidad de la validación comunitaria con incentivos económicos (Lesaege et al., 2019). El diseño propuesto combina estos precedentes en una arquitectura coherente orientada específicamente al problema de la desinformación.
2. **La tecnología *blockchain* es, entre las infraestructuras técnicas disponibles actualmente, la que ofrece mayores garantías de resistencia estructural a la captura.** El análisis del problema realizado en el capítulo 1 establece que la captura estructural de los sistemas de información opera a través del control sobre las reglas de publicación, verificación y distribución. La tecnología *blockchain* es, entre las opciones técnicas disponibles actualmente, la que ofrece mayores garantías de inmutabilidad e independencia respecto a cualquier actor —incluyendo los propios creadores del sistema— gracias a los contratos inteligentes públicos y auditables (Nakamoto, 2008). Las soluciones centralizadas o federadas pueden modificar sus reglas de forma unilateral, mientras que un contrato desplegado en una red pública solo puede modificarse mediante mecanismos transparentes que requieren consenso explícito.

3. **Los contratos inteligentes constituyen el mecanismo de gobernanza más adecuado para este tipo de plataformas.** El diseño propuesto codifica en Solidity las reglas de publicación, verificación y cálculo de reputación, garantizando su transparencia y resistencia a la manipulación (Ding et al., 2023). A diferencia de los sistemas de moderación de contenido centralizados, cuyas políticas pueden cambiar sin notificación ni rendición de cuentas, un contrato inteligente desplegado en la red solo puede modificarse mediante mecanismos de actualización explícitos y transparentes, auditables por cualquier participante. Esta propiedad es esencial para el problema que aborda NewsEra.
 4. **El mecanismo de reputación *on-chain* —OE4, núcleo del sistema— establece una primera barrera técnica eficaz contra los ataques Sybil y la colusión, si bien no los elimina por completo sin una capa adicional de identidad verificable.** El principal vector de ataque contra un sistema de validación comunitaria es la creación masiva de identidades falsas (*ataques Sybil*) o la coordinación encubierta entre validadores para sesgar los juicios (Feichtinger et al., 2023). El diseño propuesto desincentiva ambos vectores: la reputación de cada validador se calcula a partir de su historial verificable en cadena, lo que hace costosa la acumulación de influencia real con identidades múltiples; y los mecanismos de penalización —reducción de reputación por juicios divergentes del consenso— desincentivan la colusión sin necesidad de un árbitro central (Lesaage et al., 2019). El mecanismo de reapertura por rondas complementa esta protección: cuando un número suficiente de validadores nuevos solicita reconsiderar un veredicto, se abre una ronda adicional de votación independiente; los validadores que participaron en rondas anteriores reciben ajustes retroactivos de reputación en función de si sus juicios originales son confirmados o contradichos por las rondas posteriores, lo que alinea sus incentivos con la precisión a largo plazo siguiendo el principio de supervivencia del veredicto (Adler & de Alfaro, 2007). Esta aproximación mitiga el problema Sybil pero no lo resuelve completamente: la integración futura de identidad descentralizada (DIDs/VCs W3C) (World Wide Web Consortium, 2022) constituye la solución de nivel de producción prevista, descrita en la Sección 6.4.
 5. **NewsEra es un complemento a los medios existentes, no un sustituto.** La plataforma no pretende reemplazar a los medios de comunicación ni a los mecanismos institucionales de verificación de contenidos. Su función es proporcionar una infraestructura alternativa y complementaria que devuelva al ciudadano la capacidad de verificar de forma autónoma la autenticidad de un contenido informativo, sin dependencia de ningún intermediario particular. Esta naturaleza complementaria limita el alcance de las resistencias institucionales que puede encontrar y amplía el espacio de despliegue real del sistema.
 6. **Las limitaciones de la infraestructura *blockchain* actual son reales pero no insuperables en el plazo previsto.** El análisis técnico realizado identifica tres limitaciones principales del diseño actual: el coste de las transacciones en cadena (*gas*), el tiempo de finalidad del consenso en la red pública, y la brecha tecnológica que impide a los ciudadanos ordinarios interactuar directamente con contratos inteligentes sin mediación de herramientas de abstracción. Las tres limitaciones tienen soluciones técnicas
-

documentadas —redes de capa 2, abstracción de cuenta (*account abstraction*), interfaces de usuario Web3 simplificadas— cuya integración constituye una línea clara de trabajo futuro (véase la Sección 6.4).

7. **La viabilidad de NewsEra no es únicamente técnica: el diseño es condición necesaria, pero no suficiente, para resolver el problema de la coordinación social.** Un sistema de validación comunitaria descentralizada solo puede cumplir su función si concurren condiciones sociales que el diseño técnico no puede garantizar por sí solo: una masa crítica de validadores con disposición honesta, incentivos alineados con el interés colectivo y una comunidad activa dispuesta a comprometer tiempo y atención en el proceso de verificación. La resistencia estructural a la captura que proporciona la tecnología *blockchain* es un habilitador necesario, pero la sostenibilidad del sistema depende en última instancia de las condiciones de gobernanza comunitaria que lo rodean. Este reconocimiento no invalida el valor del diseño propuesto, pero sitúa correctamente su alcance: NewsEra es una infraestructura técnica que puede apoyar la coordinación social veraz, no un mecanismo que la produce de forma autónoma.

6.2. Contribuciones del trabajo

Las contribuciones del presente trabajo se articulan en tres dimensiones:

Contribución técnica: El diseño de una arquitectura de referencia para plataformas de validación de información descentralizadas. La arquitectura de NewsEra integra de forma original cuatro tecnologías previamente desarrolladas de forma independiente —contratos inteligentes EVM, sistemas de identidad descentralizada W3C (World Wide Web Consortium, 2022), mecanismos de resolución de disputas en cadena (Lesaege et al., 2019) y *frameworks* Web3 de nueva generación— en un sistema coherente orientado específicamente al problema de la desinformación. Esta integración, documentada en los capítulos de diseño y desarrollo, puede servir de referencia para proyectos con objetivos similares. En particular, el diseño del mecanismo de reputación y gobernanza de validadores —OE4— aporta una formulación concreta de cómo implementar incentivos anti-Sybil y anti-colusión en un sistema de validación comunitaria sin autoridad central.

Contribución metodológica: La aplicación y adaptación del marco de trabajo Scrum a un proyecto de investigación-implementación individual en un dominio técnico emergente (Schwaber & Sutherland, 2020). La metodología empleada —descrita en el Capítulo 3— demuestra la viabilidad de gestionar la incertidumbre técnica inherente al desarrollo con tecnología *blockchain* mediante ciclos de sprints cortos con entregables incrementales, y proporciona un modelo replicable para trabajos similares en titulaciones de ingeniería informática.

Contribución académica: La síntesis del estado del arte en la intersección de tres campos —tecnología *blockchain* y contratos inteligentes, gobernanza distribuida y sistemas de reputación descentralizados, y técnicas computacionales aplicadas a la detección de desinformación— que han evolucionado de forma relativamente independiente. El marco teórico desarrollado en el Capítulo 2 ofrece una visión integrada de este espacio de

soluciones, identifica las convergencias entre ellos y señala las brechas que justifican la propuesta de NewsEra.

6.3. Limitaciones

El presente trabajo presenta las siguientes limitaciones que conviene reconocer explícitamente:

1. **Alcance limitado a prototipo funcional.** NewsEra es un prototipo de investigación, no un sistema de producción. El diseño y la implementación están orientados a demostrar la viabilidad técnica del enfoque propuesto, no a cubrir todos los requisitos de seguridad, escalabilidad y resiliencia que exigiría un despliegue real con usuarios masivos. En particular, los contratos inteligentes desarrollados en el marco de OE3 y OE4 no han sido sometidos a una auditoría de seguridad profesional, que sería imprescindible antes de cualquier despliegue en red principal.
2. **Evaluación con datos sintéticos, no con usuarios reales.** La evaluación del prototipo, recogida en el Capítulo 5, está basada en escenarios de prueba con datos sintéticos y simulados. No se han realizado pruebas con usuarios reales en condiciones de uso natural, lo que impide extraer conclusiones sobre el comportamiento del sistema ante los patrones de uso reales —incluyendo estrategias de ataque activas por parte de actores malintencionados— y sobre la usabilidad percibida por ciudadanos sin experiencia previa con tecnología Web3.
3. **Dependencia de Ethereum Sepolia y sus condiciones de red.** El prototipo está desplegado en la red de pruebas Ethereum Sepolia, cuyas condiciones —coste de gas, tiempo de confirmación, disponibilidad de nodos— son distintas de las de la red principal y pueden variar significativamente. Los resultados de rendimiento obtenidos en este entorno no son directamente extrapolables a un despliegue en red principal ni a redes de capa 2, cuyo comportamiento diferirá en parámetros relevantes para la experiencia de usuario.
4. **OE6 marcado como opcional y potencialmente fuera del prototipo.** La integración de herramientas de IA para el análisis de contenido —detección de duplicados, identificación de sesgos lingüísticos, análisis de imágenes manipuladas— está definida como objetivo opcional en el alcance del trabajo. Su implementación depende de la disponibilidad de tiempo una vez completados los objetivos obligatorios. Si OE6 no se integra en la versión final del prototipo, el sistema carecerá de la capa de análisis automático prevista, lo que limita su capacidad de apoyo al criterio humano de los validadores.
5. **Ausencia de evaluación formal de usabilidad con usuarios finales.** El diseño de la interfaz de usuario desarrollada en el marco de OE5 no ha sido objeto de un proceso formal de evaluación de usabilidad con participantes externos. La brecha tecnológica entre los usuarios ciudadanos sin experiencia en Web3 y las interfaces actuales de interacción con *wallets* y contratos inteligentes es una de las limitaciones más importantes

del ecosistema *blockchain* en general (Krombholz et al., 2016), y este trabajo no resuelve ese problema estructural aunque sí procura minimizarlo en el diseño de la interfaz.

6. **Riesgo de concentración de poder en validadores de alta reputación acumulada.** Aunque el diseño impide que un único actor capture unilateralmente las reglas del protocolo, no impide que un grupo coordinado de validadores con alta reputación acumulada llegue a dominar el proceso de consenso. Esta concentración no constituye un fallo del diseño sino una limitación estructural inherente a los sistemas de reputación *stake-based*: los primeros participantes y los más activos tienden a acumular ventaja progresiva, lo que puede derivar en oligopolios de validación difíciles de revertir. Las mitigaciones identificadas para futuras versiones del sistema incluyen mecanismos de rotación de validadores, quórum aleatorio ponderado y decaimiento temporal de la reputación inactiva.
7. **Vulnerabilidad al ataque de *whitewashing* (lavado de reputación).** En los sistemas de reputación acumulativa, el ataque *whitewashing* consiste en que un validador cuya reputación ha sido degradada —por juicios incorrectos o sancionados— abandona su dirección y crea una nueva para reiniciar desde cero. En NewsEra este ataque tiene un coste reducido: únicamente el coste de arranque en gas necesario para comenzar a acumular historial en cadena con la nueva identidad. Esta es una limitación conocida de los mecanismos de reputación *on-chain* sin capa de identidad verificable: la ausencia de vinculación entre la dirección *blockchain* y una identidad persistente hace que la penalización reputacional sea eludible. Esta limitación es precisamente una de las razones por las que la integración de Identificadores Descentralizados y Credenciales Verificables (DIDs/VCs W3C) (World Wide Web Consortium, 2022) queda identificada como línea de trabajo futuro prioritaria en la Sección 6.4.

6.4. Trabajo futuro

El análisis realizado y el desarrollo del prototipo identifican siete líneas de trabajo futuro que permitirían evolucionar NewsEra desde un prototipo de investigación hacia un sistema con potencial de despliegue real:

1. **Migración a una red de capa 2 (Arbitrum o Polygon) para reducir costes y aumentar el rendimiento.** El principal obstáculo práctico para la adopción masiva de NewsEra es el coste de las transacciones en la red principal de Ethereum. Las redes de capa 2 —en particular Arbitrum y Polygon— ofrecen compatibilidad total con la EVM y los contratos desarrollados en Solidity, con costes de transacción entre uno y tres órdenes de magnitud inferiores y tiempos de confirmación significativamente menores. La migración técnica es directa; el esfuerzo principal radicaría en la actualización de la infraestructura de deploy y en la verificación del comportamiento del mecanismo de gobernanza en el nuevo entorno de red.
2. **Implementación de identidad descentralizada (DID/VC W3C) para resistencia Sybil de nivel de producción.** El diseño actual del mecanismo de reputación desincentiva los ataques Sybil mediante el coste acumulativo de construir historial verificable, pero no impide la creación de identidades múltiples. La integración del estándar

de Identificadores Descentralizados y Credenciales Verificables de la W3C (World Wide Web Consortium, 2022) permitiría vincular la identidad *on-chain* de cada participante a una acreditación criptográficamente verificable emitida por una entidad de confianza —universitaria, gubernamental o comunitaria—, proporcionando resistencia Sybil de nivel de producción sin comprometer el anonimato del participante frente a terceros. La gestión de identidad corporativa en *blockchain* es un campo activo: trabajos como Balastegui-García et al. (2023) exploran soluciones para acreditación de identidad organizacional en redes públicas, cuyas técnicas son aplicables a la acreditación de validadores institucionales en NewsEra.

3. **Integración del módulo de IA (OE6): detección de duplicados y análisis de sesgo.** Si OE6 no se completa en el alcance del prototipo actual, su implementación constituye la primera línea de trabajo futuro de carácter técnico. El estado del arte revisado en el Capítulo 2 identifica las técnicas más prometedoras —modelos de lenguaje para la detección de sesgos semánticos, hashing perceptual para duplicados de imágenes, modelos de clasificación de desinformación— y los conjuntos de datos de referencia disponibles para su entrenamiento y evaluación. La integración como servicio *off-chain* consultado por los contratos inteligentes a través de oráculos preservaría la inmutabilidad y la descentralización del sistema principal. Trabajos relacionados, como Barański et al. (2025), abordan la combinación de servicios anónimos con almacenamiento descentralizado en *blockchain*, técnicas directamente aplicables a la capa de análisis de contenido que contempla OE6.
 4. **Evaluación con usuarios reales y estudio formal de usabilidad.** Una línea de trabajo fundamental para medir el impacto real de la plataforma es la realización de un estudio de usuario con participantes sin experiencia previa en tecnología Web3. Este estudio permitiría identificar las barreras de adopción más significativas, validar las decisiones de diseño de la interfaz implementadas en OE5 y obtener métricas de usabilidad (Brooke, 1996) comparables con las de plataformas de información convencionales. Los resultados informarían una segunda iteración del diseño de interfaz orientada específicamente a reducir la fricción cognitiva asociada a la interacción con *wallets* y contratos inteligentes.
 5. **Gobernanza evolutiva: actualización de parámetros del sistema mediante votación *on-chain*.** El diseño actual codifica los parámetros del mecanismo de reputación y gobernanza —umbrales de acceso al rol de validador, pesos de penalización, quórum de validación— como constantes en los contratos inteligentes. Una evolución natural del sistema es permitir que la propia comunidad de participantes modifique esos parámetros mediante votación *on-chain*, convirtiendo NewsEra en una Decentralized Autonomous Organization (DAO) capaz de adaptar sus reglas de forma colectiva y transparente sin necesidad de una nueva versión de los contratos. Este mecanismo requiere un diseño cuidadoso para evitar los vectores de ataque a la gobernanza documentados por Feichtinger et al. (2023), en particular las propuestas maliciosas de baja visibilidad y la concentración de poder de voto en un número reducido de participantes.
 6. **Migración del índice *off-chain* a The Graph para eliminar el servidor centralizado.** El backend de NewsEra actúa en el prototipo actual como indexador centrali-
-

zados de eventos blockchain: escucha los eventos de los contratos y mantiene una réplica consultable en una base de datos PostgreSQL que corre en un servidor bajo control del equipo de desarrollo. Este componente constituye el único punto de centralización del sistema, ya que su caída impide las consultas aunque los datos permanezcan íntegros en la blockchain e IPFS. La solución de producción identificada es la migración a **The Graph Protocol**: una red descentralizada de nodos indexadores que permite definir un *subgraph* —un esquema de datos y un conjunto de manejadores de eventos en AssemblyScript— que la red indexa y expone mediante una API GraphQL sin depender de ningún servidor centralizado. The Graph es utilizado en producción por los principales protocolos DeFi (Uniswap, Aave, Compound) y representa el estándar de facto para la indexación descentralizada de datos *on-chain*. Esta migración completaría la arquitectura de NewsEra eliminando el único componente con dependencia centralizada, haciendo el sistema plenamente resistente a la captura incluso a nivel de infraestructura de consulta.

7. **Veredicto automatizado y verificable *on-chain* mediante zkML y oráculos descentralizados.** Una pregunta central sobre el diseño de NewsEra es si los contratos inteligentes podrían contribuir a la determinación de la veracidad de un artículo de forma automática, más allá de agregar los juicios humanos. La respuesta en el diseño actual es negativa —los contratos no tienen acceso a información externa ni capacidad de razonamiento semántico— pero el estado del arte en las intersecciones entre *blockchain* e IA abre una línea de trabajo futuro genuinamente prometedora.

La restricción estructural de los contratos inteligentes es que no pueden acceder a datos fuera de la cadena. Sin embargo, dos tecnologías en plena maduración permiten superar esa limitación de forma verificable:

- **Oráculos descentralizados** (Chainlink Functions, API3): permiten que un contrato inteligente solicite la ejecución de código arbitrario fuera de la cadena —incluida la llamada a un modelo de clasificación de texto— y reciba el resultado con garantías criptográficas sobre la integridad de la computación. La red de nodos del oráculo actúa como verificador distribuido, reduciendo la dependencia en un único proveedor de confianza.
- **zkML (Zero-Knowledge Machine Learning)**: proyectos como EZKL, Modulus Labs u Ora Protocol permiten ejecutar un modelo de *machine learning* fuera de la cadena y generar una *prueba de conocimiento cero* (ZK proof) que demuestra que *ese modelo concreto* —cuyo hash es público y auditable— produjo *ese resultado concreto* sobre *ese contenido concreto*. El contrato inteligente solo necesita verificar la prueba, una operación computacionalmente económica. El resultado es certificado y no puede ser falsificado sin invalidar la prueba.

Aplicado a NewsEra, este mecanismo permitiría introducir un cuarto tipo de veredicto —**AI_VERDICT**— emitido por un modelo de clasificación de desinformación verificado mediante zkML. Este veredicto podría participar en el cómputo del consenso con un peso ponderado diferente al de los validadores humanos, actuando como señal técnica complementaria al juicio colectivo de la comunidad. El modelo utilizado sería público, auditable y parte del protocolo: su reemplazo requeriría el mismo consenso explícito que

cualquier modificación de los contratos. De esta forma, la verificación automática no recentralizaría el sistema, sino que añadiría una señal objetiva y verificable al proceso de consenso humano.

Esta línea conecta directamente con trabajos del ámbito de *blockchain* aplicada a problemas de verificación e integridad en dominios donde la calidad de la información tiene consecuencias directas, como los sistemas de gobierno inteligente (Mora et al., 2021) o los sistemas de integridad deportiva (Sirvent-Llamas, Mora et al., 2025), que demuestran la aplicabilidad de la tecnología a contextos de verificación socialmente relevantes.

6.5. Valoración personal

El desarrollo de este trabajo ha supuesto un proceso de aprendizaje que ha trascendido el ámbito estrictamente técnico. El problema que aborda NewsEra —la captura estructural de los sistemas de información— es, en primer lugar, un problema político y social antes que tecnológico. Comprenderlo con la profundidad necesaria para diseñar una respuesta técnica rigurosa ha requerido un recorrido por la teoría del espacio público (Habermas, 1989), la economía política de los medios (Herman & Chomsky, 1988; McChesney, 2008) y la literatura sobre los efectos de la algoritmización de la distribución de información (Pariser, 2011). Ese recorrido ha modificado de forma duradera la forma en que me relaciono con el ecosistema informativo digital, más allá de lo que este trabajo puede reflejar en sus páginas.

El trabajo en el dominio de la tecnología *blockchain* y el desarrollo Web3 ha presentado desafíos técnicos de una complejidad diferente a la que se encuentra en el desarrollo de software convencional. La necesidad de razonar simultáneamente sobre la seguridad criptográfica, el modelo de ejecución de la EVM, el coste computacional de las operaciones en cadena y la experiencia del usuario final en sistemas con confirmación asíncrona ha sido exigente y, al mismo tiempo, intelectualmente estimulante. En particular, el diseño del mecanismo de reputación y gobernanza de validadores —componente central del sistema— ha requerido múltiples iteraciones para lograr un equilibrio entre la robustez ante los vectores de ataque identificados y la practicabilidad del sistema para usuarios reales.

Finalmente, la perspectiva que este trabajo me ha permitido desarrollar sobre el potencial de las tecnologías descentralizadas es matizada. La *blockchain* no es una solución universal ni está exenta de limitaciones serias: los costes de transacción, la complejidad de la experiencia de usuario, la concentración de poder que puede darse en los propios sistemas de gobernanza distribuida (Feichtinger et al., 2023) y la brecha digital que separa a los ciudadanos ordinarios de las herramientas Web3 son obstáculos reales que no pueden minimizarse. Sin embargo, para el problema específico que aborda NewsEra —garantizar que las reglas que gobiernan la publicación y la verificación de información no puedan ser capturadas por ningún actor con suficiente poder económico o político— la descentralización criptográfica ofrece, hoy por hoy, la respuesta técnica más sólida disponible. Desarrollar ese potencial de forma responsable, con atención a sus limitaciones y a las condiciones sociales necesarias para su adopción real, es el desafío que queda abierto.

Bibliografía

- Adler, B. T., & de Alfaro, L. (2007). A Content-Driven Reputation System for the Wikipedia. *Proceedings of the 16th International Conference on World Wide Web*, 261-270. <https://doi.org/10.1145/1242572.1242608>
- Allen, C. (2016). The Path to Self-Sovereign Identity. *Life with Alacrity*. <https://www.lifewithalacrity.com/2016/04/the-path-to-self-sovereign-identity.html>
- Balastegui-García, A., et al. (2023). Corporate Digital Identity Management on Blockchain. *Proceedings of the International Conference on Ubiquitous Computing and Ambient Intelligence*. https://doi.org/10.1007/978-3-031-19560-0_55
- Barański, M., Szymański, J., & Mora, H. (2025). Anonymous Services on Blockchain with Decentralised Storage. *International Journal of Information Security*. <https://doi.org/10.1007/s10207-025-01052-w>
- Benet, J. (2014). *IPFS — Content Addressed, Versioned, P2P File System* (Technical Report). Protocol Labs. Consultado el 15 de enero de 2025, desde <https://arxiv.org/abs/1407.3561>
- Brooke, J. (1996). SUS: A Quick and Dirty Usability Scale. En P. W. Jordan, B. Thomas, B. A. Weerdmeester & I. L. McClelland (Eds.), *Usability Evaluation in Industry* (pp. 189-194). Taylor & Francis.
- Buterin, V. (2014). A Next-Generation Smart Contract and Decentralized Application Platform. <https://ethereum.org/en/whitepaper/>
- Ding, Q., Liebau, D., Wang, Z., & Xu, W. (2023). A Survey on Decentralized Autonomous Organizations (DAOs) and Their Governance. *World Scientific Annual Review of Fintech*, 1. <https://doi.org/10.1142/S281100482350001X>
- Douceur, J. R. (2002). The Sybil Attack. *Peer-to-Peer Systems*, 251-260. https://doi.org/10.1007/3-540-45748-8_24
- Ethereum Foundation. (2022). *The Merge*. Consultado el 15 de enero de 2025, desde <https://ethereum.org/en/roadmap/merge/>
- Feichtinger, R., Fritsch, R., Vonlanthen, Y., & Wattenhofer, R. (2023). The Hidden Shortcomings of (D)AOs — An Empirical Study of On-Chain Governance. <https://arxiv.org/abs/2302.12125>
- Habermas, J. (1989). *The Structural Transformation of the Public Sphere*. MIT Press.
- Hasan, H. R., & Salah, K. (2019). Combating Deepfake Videos Using Blockchain and Smart Contracts. *IEEE Access*, 7, 41596-41606. <https://doi.org/10.1109/ACCESS.2019.2905689>
- Herman, E. S., & Chomsky, N. (1988). *Manufacturing Consent: The Political Economy of the Mass Media*. Pantheon Books.
- Institute of Electrical and Electronics Engineers. (1998). *IEEE Std 830-1998 - IEEE Recommended Practice for Software Requirements Specifications* (Standard). IEEE.

- Krombholz, K., Judmayer, A., Gusenbauer, M., & Weippl, E. (2016). The Other Side of the Coin: User Experiences with Bitcoin Security and Privacy. *Financial Cryptography and Data Security*, 555-580. https://doi.org/10.1007/978-3-662-54970-4_33
- Lesaëge, C., Ast, F., & George, W. (2019). Kleros: Short Paper v1.0.7 — A Decentralized Court. <https://kleros.io/kleros.pdf>
- Mazzocca, C., Acar, A., Uluagac, S., Montanari, R., Bellavista, P., & Conti, M. (2025). A Survey on Decentralized Identifiers and Verifiable Credentials. *IEEE Communications Surveys & Tutorials*. <https://doi.org/10.1109/COMST.2025.3543197>
- McChesney, R. W. (2008). *The Political Economy of Media: Enduring Issues, Emerging Dilemmas*. Monthly Review Press.
- Messias, J., Pahari, V., Chandrasekaran, B., Gummadi, K. P., & Loiseau, P. (2023). Understanding Blockchain Governance: Analyzing Decentralized Voting to Amend DeFi Smart Contracts. <https://arxiv.org/abs/2305.17655>
- Mora, H., et al. (2021). Disruptive Technologies for Smart Government. En *Proceedings of the International Conference on Technology and Innovation in Learning, Teaching and Education*. Springer. https://doi.org/10.1007/978-3-030-62066-0_6
- Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. <https://bitcoin.org/bitcoin.pdf>
- Onyekachi, N. C. (2019). Applying Scrum Agile Framework to Academic Research and Development in the University Environment. *International Journal of Scientific Research and Management*, 7(3), 1089-1095. <https://doi.org/10.18535/ijstrm/v7i3.ec02>
- Pariser, E. (2011). *The Filter Bubble: What the Internet Is Hiding from You*. Penguin Press.
- Ries, E. (2011). *The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses*. Crown Business.
- Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. <https://scrumguides.org/scrum-guide.html>
- Shu, K., Sliva, A., Wang, S., Tang, J., & Liu, H. (2017). Fake News Detection on Social Media: A Data Mining Perspective. *ACM SIGKDD Explorations Newsletter*, 19(1), 22-36. <https://doi.org/10.1145/3137597.3137600>
- Sirvent-Llamas, A., Mora, H., et al. (2025). Blockchain-Based Anti-Doping System for Sports Integrity. *International Journal of Sports Medicine*. <https://doi.org/10.1177/17479541251394511>
- Vosoughi, S., Roy, D., & Aral, S. (2018). The Spread of True and False News Online. *Science*, 359(6380), 1146-1151. <https://doi.org/10.1126/science.aap9559>
- Wang, X., Xie, H., Ji, S., Liu, L., & Huang, D. (2023). Blockchain-based fake news traceability and verification mechanism. *Heliyon*, 9(7), e17084. <https://doi.org/10.1016/j.heliyon.2023.e17084>
- Wardle, C., & Derakhshan, H. (2017). Information Disorder: Toward an Interdisciplinary Framework for Research and Policy Making. *Council of Europe Report*, 27. <https://rm.coe.int/information-disorder-report-november-2017/1680764666>
- Weyl, E. G., & Posner, E. A. (2019). *Radical Markets: Uprooting Capitalism and Democracy for a Just Society*. Princeton University Press.
- World Wide Web Consortium. (2022). *Decentralized Identifiers (DIDs) v1.0: Core Architecture, Data Model, and Representations* (inf. téc.). W3C. <https://www.w3.org/TR/did-core/>

Zhou, X., & Zafarani, R. (2020). A Survey of Fake News: Fundamental Theories, Detection Methods, and Opportunities. *ACM Computing Surveys*, 53(5), 1-40. <https://doi.org/10.1145/3395046>

Lista de Acrónimos y Abreviaturas

ABI	Application Binary Interface. 19, 22,
ACID	Atomicity, Consistency, Isolation, Durability.
APA	American Psychological Association.
API	Application Programming Interface. xv, 13, 18, 19, 22, 23, 34, 42, 49, 52, 55, 63, 65, 70, 77, , 92
AWS	Amazon Web Services.
BDIMS	Blockchain-based Decentralized Identity Management System.
CD	Continuous Delivery.
CDN	Content Delivery Network.
CI	Continuous Integration. 63, 64,
CID	Content Identifier. 13, 15, 22, 23, 52, , 95
CLI	Command Line Interface.
CNN	Convolutional Neural Network. 14,
CRUD	Create, Read, Update, Delete.
CSS	Cascading Style Sheets. 22,
DAO	Decentralized Autonomous Organization. 10, 32, 76, 103
DBMS	Database Management System.
DID	Decentralized Identifier. 12,
DL	Deep Learning.
DNS	Domain Name System.
EPS	Escuela Politécnica Superior.
ETL	Extract, Transform, Load.
EVM	Ethereum Virtual Machine. 9, 10, 21, 70, 71, 73, 75, 78,
GAN	Generative Adversarial Network.
GCP	Google Cloud Platform.
GUI	Graphical User Interface.
HTML	HyperText Markup Language.
HTTP	HyperText Transfer Protocol. 23, 33,
HTTPS	HyperText Transfer Protocol Secure.
IA	Inteligencia Artificial. 3–5, 7, 13, 65, 74, 76, 77,
IaaS	Infrastructure as a Service.
IDE	Integrated Development Environment. 23,
IEEE	Institute of Electrical and Electronics Engineers.
IETF	Internet Engineering Task Force.
IP	Internet Protocol.

IPFS	InterPlanetary File System. ix, xi, 12, 13, 15, 22, 23, 33, 34, 37, 52, , 94, 95
ISO	International Organization for Standardization.
JSON	JavaScript Object Notation.
JWT	JSON Web Token.
K8s	Kubernetes.
LLM	Large Language Model.
LSTM	Long Short-Term Memory.
ML	Machine Learning.
MVC	Model-View-Controller.
MVP	Minimum Viable Product. 25,
MVVM	Model-View-ViewModel.
NFT	Non-Fungible Token.
NLP	Natural Language Processing. 13,
NoSQL	Not Only SQL.
OAuth	Open Authorization.
ORM	Object-Relational Mapping. 22, 34, 37,
P2P	Peer-to-Peer. 3,
PaaS	Platform as a Service.
RBAC	Role-Based Access Control.
REST	Representational State Transfer. 13, 19, 23, 37, 55, 63, 70,
RNN	Recurrent Neural Network.
RSS	Really Simple Syndication.
SaaS	Software as a Service.
SDK	Software Development Kit.
SPA	Single Page Application. 22, 34, 36, 38, 59,
SQL	Structured Query Language.
SSL	Secure Sockets Layer.
SUS	System Usability Scale. 26, 66, , 106–108
TCP	Transmission Control Protocol.
TDD	Test-Driven Development.
TFG	Trabajo Fin de Grado. ix, 3, 17, 20, 32, 71, 103, 109
TFM	Trabajo Fin de Máster.
TLS	Transport Layer Security.
UA	Universidad de Alicante.
UDP	User Datagram Protocol.
URI	Uniform Resource Identifier.
URL	Uniform Resource Locator.
UX	User Experience. 23, 59,
VC	Verifiable Credential. 12,
VPN	Virtual Private Network.
W3C	World Wide Web Consortium. 12,
XML	eXtensible Markup Language.

Glosario de Términos

A

algoritmo Conjunto ordenado y finito de operaciones que permite hallar la solución de un problema. Los algoritmos son la base de la programación y definen los pasos lógicos para resolver tareas computacionales.

B

backend Parte del software que procesa la entrada desde el frontend, gestiona la lógica de negocio y se comunica con la base de datos. También conocido como lado del servidor. 3, 5, 21, 23, 29, 32–34, 36–38, 54, 55, 59–61, 70, 90, 92, 96

C

cifrado Proceso de codificación de información para que solo las partes autorizadas puedan acceder a ella. Puede ser simétrico (misma clave) o asimétrico (par de claves pública/privada).

compilador Programa informático que traduce código escrito en un lenguaje de programación (código fuente) a otro lenguaje (código objeto), típicamente código máquina ejecutable.

D

deploy Proceso de poner una aplicación o actualización a disposición de los usuarios finales. Incluye la instalación, configuración y activación del software en el entorno de producción. 75

E

entorno Bloque de código en $\text{\LaTeX}$ delimitado por `\begin{nombre}` y `\end{nombre}`. Define un contexto especial para el contenido.

escalabilidad Capacidad de un sistema para manejar una cantidad creciente de trabajo, o su potencial para ser ampliado para acomodar ese crecimiento. Puede ser vertical (más recursos) u horizontal (más instancias).

F

firewall Sistema de seguridad de red que monitoriza y controla el tráfico de red entrante y saliente según reglas de seguridad predeterminadas.

flotante Elemento (figura o tabla) que $\text{\LaTeX}$ puede mover de su posición en el código fuente para optimizar la maquetación. Se controla con especificadores como [htbp].

framework Estructura conceptual y tecnológica de soporte definido, normalmente con artefactos o módulos de software concretos, que puede servir de base para la organización y desarrollo de software.

frontend Parte del software que interactúa directamente con el usuario. Incluye la interfaz gráfica, formularios y todos los elementos visuales de una aplicación. 19, 21, 23, 29, 33, 34, 37, 38, 52, 70, 95, 96

H

hash Función que convierte una entrada de datos de cualquier tamaño en una salida de tamaño fijo. Se usa para verificar integridad de datos y almacenar contraseñas de forma segura. 3, 8, 22, 37, 38, 40, 42, 43, 60, 94, 95, 97

I

índice Estructura de datos que mejora la velocidad de las operaciones de búsqueda en una tabla de base de datos, a costa de espacio adicional y tiempo de escritura.

intérprete Programa que ejecuta instrucciones escritas en un lenguaje de programación línea por línea, sin necesidad de compilación previa. Python y JavaScript son ejemplos de lenguajes interpretados.

L

latencia Tiempo que transcurre desde que se envía una solicitud hasta que se recibe la respuesta. En sistemas distribuidos, se mide típicamente en milisegundos.

M

macro Comando definido por el usuario en $\text{\LaTeX}$ que representa una secuencia de instrucciones. Permite automatizar tareas repetitivas y crear abstracciones.

microservicio Estilo arquitectónico que estructura una aplicación como una colección de servicios pequeños, autónomos y débilmente acoplados. Cada microservicio implementa una funcionalidad de negocio específica.

middleware Software que actúa como puente entre el sistema operativo o base de datos y las aplicaciones, especialmente en una red. Facilita la comunicación y gestión de datos entre sistemas distribuidos.

monolito Arquitectura de software donde todos los componentes de la aplicación están interconectados y son interdependientes, desplegándose como una única unidad.

N

normalización Proceso de organización de datos en una base de datos relacional para reducir la redundancia y mejorar la integridad de los datos. Incluye varias formas normales (1NF, 2NF, 3NF, BCNF).

P

pipeline Secuencia automatizada de pasos para compilar, probar y desplegar software. En CI/CD, los pipelines automatizan el flujo desde el código fuente hasta producción. 24

preámbulo Parte del documento `LATEX` entre `\documentclass` y `\begin{document}`. Contiene la configuración del documento y la carga de paquetes.

Q

query Consulta o solicitud de datos a una base de datos. En SQL, las queries se escriben usando comandos como SELECT, INSERT, UPDATE y DELETE.

R

refactorización Proceso de reestructurar código existente sin cambiar su comportamiento externo. Mejora la legibilidad, reduce la complejidad y facilita el mantenimiento.

S

sprint Período de tiempo fijo (generalmente 2-4 semanas) durante el cual se completa un conjunto específico de trabajo en metodologías ágiles como Scrum. 17–19, 73

T

throughput Cantidad de datos o transacciones que un sistema puede procesar en una unidad de tiempo. Es una medida clave del rendimiento de sistemas.

token Cadena de caracteres que representa una credencial de seguridad o sesión. En autenticación, los tokens permiten verificar la identidad sin transmitir contraseñas. 11

transacción Unidad lógica de trabajo en una base de datos que debe completarse en su totalidad o no ejecutarse en absoluto. Las transacciones garantizan la integridad de los datos.

A. Catálogo de casos de uso detallados

Este anexo desarrolla, con actor, precondiciones, flujo principal, flujos alternativos (cuando existen) y postcondiciones, los 42 casos de uso (UC) definidos en la Especificación de Requisitos de Software del Capítulo 4 (§4.1.1), agrupados por las cinco funcionalidades de alto nivel (FEAT) de **STQR 1. Sistema de publicación y verificación de contenidos**. Los casos de uso puramente navegacionales o de solo lectura se documentan con un nivel de detalle proporcional a su sencillez.

A.1. FEAT 1. Gestión de cuenta

Caso de uso: UC 1 — Conectar cartera

Actor(es): Visitante.

Precondiciones: el actor dispone de una cartera Ethereum compatible (MetaMask u otra soportada por WalletConnect).

Flujo principal:

1. El actor pulsa el botón **ConnectButton** de RainbowKit, visible en la cabecera de toda la aplicación.
2. El sistema solicita la conexión a la cartera instalada en el navegador.
3. El actor autoriza la conexión desde su cartera.
4. El sistema obtiene la dirección activa mediante `useAccount()` de `wagmi` y desbloquea FEAT 1, FEAT 2 y FEAT 4.

Flujos alternativos:

- Si el actor no tiene ninguna cartera instalada, RainbowKit ofrece enlaces de instalación de carteras compatibles.
- Si el actor rechaza la conexión, el sistema permanece en el perfil público de solo lectura (FEAT 3).

Postcondiciones: la dirección activa queda disponible en toda la aplicación; no se realiza ninguna escritura *on-chain* ni ninguna transacción es firmada por este caso de uso.

Caso de uso: UC 2 — Actualizar / modificar datos del perfil

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 1 completado.

Flujo principal:

1. El actor edita nombre, avatar o email en el formulario de perfil.
2. El sistema solicita al actor que firme un mensaje mediante `personal_sign` para demostrar la propiedad de la dirección.
3. El actor firma el mensaje desde su cartera — sin coste de gas, no es una transacción *on-chain*.
4. El backend verifica la firma contra la dirección declarada y persiste los cambios en la tabla `UserProfile`.

Flujos alternativos:

- Si la firma no corresponde a la dirección activa, el backend rechaza la actualización con un error `FORBIDDEN`.
- El campo email es opcional; si se omite, no se activan notificaciones por correo.

Postcondiciones: el perfil enriquecido queda actualizado sin que se haya requerido contraseña alguna ni transacción *on-chain*.

Caso de uso: UC 3 — Revisar artículos favoritos

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 1 completado.

Flujo principal:

1. El actor accede a su perfil y selecciona la pestaña de favoritos.
2. El sistema solicita `GET /api/v1/profile/favorites`.
3. El sistema muestra el listado paginado de artículos guardados mediante UC 31.

Flujos alternativos: si no hay favoritos guardados, el sistema muestra un estado vacío.

Postcondiciones: ninguna escritura; el actor visualiza el listado.

Caso de uso: UC 4 — Revisar artículos publicados propios

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 1 completado.

Flujo principal:

1. El actor accede a su perfil y selecciona “Mis publicaciones”.
2. El sistema solicita `GET /api/v1/publications?author=:address`.
3. El sistema muestra el listado con título, estado de consenso y fecha de cada artículo propio.

Postcondiciones: ninguna escritura.

Caso de uso: UC 5 — Revisar votaciones anteriores

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 1 completado.

Flujo principal:

1. El actor accede a su perfil y selecciona “Mis votaciones”.
2. El sistema solicita `GET /api/v1/validators/:address/history`.
3. El sistema clasifica cada voto según el estado de la ronda: **ganada** (DEFINITIVE, opción coincide con el voto), **perdida** (DEFINITIVE, opción distinta) o **sin resolver** (DISPUTED o ronda aún PENDING).
4. El sistema muestra el porcentaje de aciertos (*accuracy*) junto al listado.

Postcondiciones: ninguna escritura; ninguna ronda DISPUTED se contabiliza como ganada ni perdida.

Caso de uso: UC 6 — Revisar reputación

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 1 completado.

Flujo principal:

1. El actor accede a su perfil.
2. El sistema invoca `ReputationSystem.getReputation(address)` mediante `useReadContract`.
3. El sistema muestra la puntuación actual.

Postcondiciones: lectura *on-chain* directa, sin escritura.

Caso de uso: UC 7 — Revisar evolución de la reputación

Actor(es): Usuario con cartera conectada.

Precondiciones: el backend ha indexado al menos un evento `ReputationUpdated` para la dirección.

Flujo principal:

1. El actor accede a su perfil y selecciona “Evolución de reputación”.
2. El sistema solicita al backend la serie temporal de variaciones registradas por el indexador (evento `ReputationUpdated` con su bloque y delta).
3. El sistema representa la evolución en un gráfico o listado cronológico.

Postcondiciones: ninguna escritura. Requiere un endpoint de agregación histórica aún no especificado en el diseño de la API (§4.5.2) — pendiente de Sprint 7.

Caso de uso: UC 8 — Reclamar reputación neta

Actor(es): Usuario con cartera conectada.

Precondiciones: idénticas a UC 37.

Flujo principal: comparte el flujo íntegro con UC 37 (Reclamar reputación retroactiva, §A.4); este caso de uso difiere únicamente en el punto de entrada, que es el listado de reclamaciones pendientes del perfil en lugar del detalle del artículo.

Postcondiciones: ver UC 37.

Caso de uso: UC 9 — Revisar solicitudes de reapertura

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 1 completado.

Flujo principal:

1. El actor accede a su perfil y selecciona “Solicitudes de reapertura”.
2. El sistema solicita el listado de `ReopenRequest` cuyo `requesterAddress` coincide con la dirección activa.
3. El sistema muestra cada solicitud con el estado del artículo (pendiente de alcanzar el umbral / ronda ya reabierta).

Postcondiciones: ninguna escritura.

Caso de uso: UC 10 — Consultar notificaciones

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 1 completado.

Flujo principal:

1. El actor abre el panel de notificaciones.
2. El sistema solicita el listado de **Notification** con **userAddress** igual a la dirección activa.
3. El actor puede marcar una notificación como leída.

Flujos alternativos: si no hay notificaciones nuevas, el sistema muestra un estado vacío.

Postcondiciones: al marcar como leída, **read** pasa a **true**; no hay ningún efecto *on-chain*.

Caso de uso: UC 11 — Desconectar cartera

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 1 completado.

Flujo principal:

1. El actor pulsa “Desconectar” en el menú de cuenta.
2. El sistema invoca **disconnect()** de **wagmi**.
3. El sistema vuelve al perfil público (FEAT 3), bloqueando FEAT 1, FEAT 2 y FEAT 4.

Postcondiciones: la sesión de cartera se cierra en el cliente; ningún dato *on-chain* ni *off-chain* se modifica.

A.2. FEAT 2. Publicar artículo

Caso de uso: UC 12 — Redactar artículo

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 1 completado.

Flujo principal:

1. El actor accede a **/publish**.
2. El actor completa el formulario siguiendo la plantilla estándar de redacción: título, cuerpo, etiquetas (UC 14), enlaces internos (UC 15) y referencias bibliográficas (UC 16).

Postcondiciones: el contenido queda en el estado del formulario, sin persistir todavía.

Caso de uso: UC 13 — Guardar borrador

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 12 en curso.

Flujo principal:

1. El actor pulsa “Guardar borrador”.
2. El sistema persiste el contenido localmente, sin subida a IPFS ni transacción *on-chain*.

Flujos alternativos: al reabrir `/publish`, el sistema recupera el último borrador guardado.

Postcondiciones: el borrador queda disponible para retomar la redacción; no genera ningún registro *on-chain* ni *off-chain* permanente.

Caso de uso: UC 14 — Etiquetar artículo por categoría/temática

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 12 en curso.

Flujo principal: el actor introduce una o varias etiquetas libres (*tags*) en el formulario de publicación.

Postcondiciones: las etiquetas se incluyen en el *payload* de UC 19 (campo **tags**).

Caso de uso: UC 15 — Enlazar otros artículos de NewsEra

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 12 en curso.

Flujo principal: el actor inserta un hipervínculo interno (`/article/:hash`) hacia otro artículo de NewsEra dentro del cuerpo del texto.

Postcondiciones: el hipervínculo forma parte del **body** y, por tanto, del **contentHash** calculado en UC 18; cualquier edición posterior del enlace altera el hash.

Caso de uso: UC 16 — Referencias bibliográficas

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 12 en curso.

Flujo principal: el actor añade las fuentes citadas en la sección de referencias de la plantilla.

Postcondiciones: las referencias forman parte del `body` y del `contentHash`, igual que UC 15.

Caso de uso: UC 17 — Vista previa del artículo antes de confirmar

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 12 completado.

Flujo principal:

1. El actor pulsa “Vista previa”.
2. El sistema renderiza el artículo con la plantilla final, tal como se mostrará una vez publicado.

Postcondiciones: ninguna escritura; el actor puede volver a editar o continuar a UC 18.

Caso de uso: UC 18 — Mostrar hash calculado antes de firmar

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 17 completado.

Flujo principal:

1. El sistema calcula `contentHash = keccak256(body)` con `viem`.
2. El sistema muestra el valor al actor antes de solicitar la firma de UC 19.

Postcondiciones: ninguna escritura; el hash mostrado debe coincidir con el que se registrará *on-chain* en UC 19.

Caso de uso: UC 19 — Confirmar publicación on-chain

Actor(es): Usuario con cartera conectada.

Precondiciones: el artículo ha sido redactado (UC 12) y el actor ha revisado la vista previa (UC 17) y el `contentHash` calculado (UC 18).

Flujo principal:

1. El frontend calcula `contentHash = keccak256(body)` con `viem`.
2. El frontend sube el cuerpo del artículo a IPFS mediante la API de Pinata y obtiene el CID.
3. El actor firma la transacción `registerPublication( contentHash)` desde su cartera.
4. El sistema espera la confirmación *on-chain* mediante `useWaitForTransactionReceipt` y muestra su estado.

5. Tras la confirmación, el frontend envía `POST /api/v1/publications` con `{contentHash, ipfsCid, title, body, tags}` al backend.
6. El indexador detecta el evento `PublicationRegistered` y actualiza la réplica *off-chain*.

Flujos alternativos:

- Si el `contentHash` ya existe, la transacción revierte con `PublicationAlreadyExists`; no se publica contenido duplicado.
- Si el actor rechaza la firma, no se produce ningún cambio y el borrador (UC 13) se conserva para un intento posterior.

Postcondiciones: el artículo queda registrado de forma inmutable *on-chain*, con autoría y fecha asignadas automáticamente por el sistema, y es visible en el *feed* (UC 20) tras la sincronización del indexador.

A.3. FEAT 3. Explorar contenido

Caso de uso: UC 20 — Consultar feed de artículos recientes

Actor(es): Visitante o Usuario.

Precondiciones: ninguna — accesible sin cartera conectada.

Flujo principal:

1. El actor navega a `/`.
2. El sistema solicita `GET /api/v1/publications` (página 1).
3. El sistema muestra una lista paginada con título, autor, estado de consenso y número de votos por artículo.

Flujos alternativos: si el backend no responde, el sistema muestra un error con opción de reintentar (RI 5); si no hay publicaciones, muestra un estado vacío.

Postcondiciones: ninguna escritura.

Caso de uso: UC 21 — Buscar / filtrar artículos

Actor(es): Visitante o Usuario.

Flujo principal:

1. El actor introduce un término de búsqueda o selecciona un filtro (estado de consenso, etiqueta, autor).

2. El sistema solicita `GET /api/v1/publications` con los parámetros de filtro correspondientes.
3. El sistema actualiza el listado.

Postcondiciones: ninguna escritura.

Caso de uso: UC 22 — Ver detalle de artículo

Actor(es): Visitante o Usuario.

Flujo principal:

1. El actor selecciona un artículo del *feed* o de un resultado de búsqueda.
2. El sistema solicita `GET /api/v1/publications/:hash`.
3. El sistema muestra el contenido completo y los metadatos del artículo.

Flujos alternativos: si el hash no existe, el sistema muestra un error 404.

Postcondiciones: ninguna escritura.

Caso de uso: UC 23 — Ver estado de consenso del artículo

Actor(es): Visitante o Usuario.

Precondiciones: UC 22 en curso.

Flujo principal: el sistema muestra el campo `consensusState` (PENDING/DEFINITIVE/DISPUTED) obtenido en la respuesta de UC 22.

Postcondiciones: ninguna escritura.

Caso de uso: UC 24 — Ver recuento de votos del artículo

Actor(es): Visitante o Usuario.

Precondiciones: UC 22 en curso.

Flujo principal: el sistema muestra, por ronda, el número de votos de cada tipo (TRUE/FALSE/UNVERIFIABLE), obtenido de `roundVoteCount` y de la réplica *off-chain*.

Postcondiciones: ninguna escritura.

Caso de uso: UC 25 — Consultar ranking de validadores por reputación

Actor(es): Visitante o Usuario.

Flujo principal:

1. El actor navega a `/validators`.

2. El sistema solicita `GET /api/v1/validators`, ordenado por `reputationScore` descendente.
3. El sistema muestra el listado paginado.

Postcondiciones: ninguna escritura.

Caso de uso: UC 26 — Consultar perfil público de una dirección

Actor(es): Visitante o Usuario.

Flujo principal:

1. El actor navega a `/validators/:address` o accede desde UC 30 (perfil de un autor).
2. El sistema solicita `GET /api/v1/validators/:address` y `GET /api/v1/publications?author=:address`.
3. El sistema muestra reputación, historial de validación y artículos publicados por esa dirección.

Flujos alternativos: si la dirección no está registrada como validador, el sistema muestra igualmente sus artículos publicados, sin datos de reputación.

Postcondiciones: ninguna escritura.

Caso de uso: UC 27 — Acceder a la verificación del artículo

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 22 en curso.

Flujo principal: el actor pulsa el enlace o botón “Verificar” en el detalle del artículo, que navega a las herramientas de FEAT 4 (§A.4).

Postcondiciones: ninguna escritura; es un caso de uso de navegación puro.

Caso de uso: UC 28 — Ordenar resultados del feed/búsqueda

Actor(es): Visitante o Usuario.

Precondiciones: UC 20 o UC 21 en curso.

Flujo principal: el actor selecciona un criterio de ordenación (fecha, número de votos, estado de consenso); el sistema reordena el listado ya cargado o solicita una nueva página con el parámetro de orden.

Postcondiciones: ninguna escritura.

Caso de uso: UC 29 — Seguir un artículo

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 22 en curso.

Flujo principal:

1. El actor pulsa “Seguir” en el detalle del artículo.
2. El sistema registra la relación de seguimiento asociada a la dirección del actor.
3. A partir de este momento, los cambios de estado del artículo generan una notificación (UC 10), según RD 20–RD 21.

Postcondiciones: el actor queda suscrito a las notificaciones de ese artículo.

Caso de uso: UC 30 — Compartir artículo

Actor(es): Visitante o Usuario.

Precondiciones: UC 22 en curso.

Flujo principal: el actor pulsa “Compartir”; el sistema copia o expone el enlace directo `/article/:hash`.

Postcondiciones: ninguna escritura.

Caso de uso: UC 31 — Guardar / quitar artículo como favorito

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 22 en curso.

Flujo principal:

1. El actor pulsa el icono de favorito en el detalle o en una tarjeta del *feed*.
2. El sistema invoca `POST /api/v1/favorites/:hash` (guardar) o `DELETE /api/v1/favorites/:hash` (quitar).

Postcondiciones: el artículo aparece o desaparece del listado de UC 3.

A.4. FEAT 4. Verificar artículo

Caso de uso: UC 32 — Consultar bibliografía y enlaces internos citados

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 22 en curso.

Flujo principal: el sistema muestra, dentro del detalle del artículo, las referencias bibliográficas (UC 16) y los hipervínculos internos (UC 15) con acceso directo a cada fuente.

Postcondiciones: ninguna escritura.

Caso de uso: UC 33 — Comparar con otros artículos de NewsEra sobre el mismo tema

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 22 en curso; el artículo tiene al menos una etiqueta (UC 14).

Flujo principal: el sistema solicita `GET /api/v1/publications?tags=...` y muestra artículos relacionados por etiqueta compartida, para que el actor pueda contrastar la información antes de votar.

Postcondiciones: ninguna escritura.

Caso de uso: UC 34 — Consultar historial de rondas anteriores y su resultado

Actor(es): Usuario con cartera conectada.

Precondiciones: UC 22 en curso; `currentRound > 1`.

Flujo principal: el sistema muestra, por cada ronda anterior, su resultado (`result`) y estado (`state`), obtenidos de `rounds` (*on-chain*) y de la réplica `Round` (*off-chain*).

Postcondiciones: ninguna escritura.

Caso de uso: UC 35 — Votar artículo

Actor(es): Usuario con cartera conectada y reputación igual o superior a `MIN_REPUTATION_TO_VALIDATE`.

Precondiciones: el artículo está en estado `PENDING`; el actor no ha votado previamente ese artículo, en ninguna ronda; `canValidate(address) == true`.

Flujo principal:

1. El actor accede al detalle del artículo (UC 22) y selecciona una opción: `TRUE`, `FALSE` o `UNVERIFIABLE`.
2. El sistema invoca `submitValidation(contentHash, vote)` mediante `useWriteContract`.
3. El actor firma la transacción desde su cartera.
4. El sistema espera la confirmación *on-chain* y muestra su estado.
5. El indexador procesa el evento `ValidationSubmitted` y actualiza la réplica *off-chain*.
6. Si se alcanza el quórum, el sistema refleja el nuevo estado de consenso (`DEFINITIVE` o `DISPUTED`) y el efecto reputacional inmediato sobre los votantes de la ronda.

Flujos alternativos:

- Si `canValidate == false`, el control de voto aparece deshabilitado indicando el motivo (reputación insuficiente).
- Si el actor rechaza la firma, no se realiza ningún cambio.
- Si la transacción revierte (`AlreadyValidated`, `VotingNotOpen`), el sistema traduce el error a lenguaje natural y no permite reintentar la misma acción sobre ese artículo.

Postcondiciones: el voto queda registrado de forma inmutable *on-chain*; si la ronda alcanzó **DEFINITIVE**, la reputación de los votantes de esa ronda se actualiza según UC 40.

Caso de uso: UC 36 — Solicitar reapertura de un artículo concluido

Actor(es): Usuario con cartera conectada y reputación suficiente.

Precondiciones: el artículo está en estado **DEFINITIVE** o **DISPUTED**; el actor no ha votado ese artículo y no ha solicitado ya su reapertura.

Flujo principal:

1. El actor solicita la reapertura desde el detalle del artículo.
2. El sistema invoca `requestReopen(contentHash)`.
3. El actor firma la transacción desde su cartera.
4. El contrato incrementa el contador de solicitudes y emite **ReopenRequested**.
5. Si el contador alcanza `reopenThreshold`, el contrato abre una nueva ronda y emite **VotingReopened**; el contador de solicitudes se reinicia a cero para ese artículo.

Flujos alternativos:

- Si el estado del artículo no es **DEFINITIVE** ni **DISPUTED**, la transacción revierte con **ReopenNotAvailable**.
- Si el actor ya votó ese artículo, revierte con **AlreadyValidated**; si ya solicitó la reapertura, revierte con **AlreadyRequestedReopen**.
- Si la reputación del actor es insuficiente, revierte con **InsufficientReputation**.

Postcondiciones: la solicitud queda registrada de forma permanente; si se alcanzó el umbral, el artículo vuelve a estado **PENDING** en una nueva ronda, visible en UC 34 y UC 38.

Caso de uso: UC 37 — Reclamar reputación retroactiva

Actor(es): Usuario con cartera conectada que votó una ronda anterior a la ronda actual del artículo.

Precondiciones: existen rondas **DEFINITIVE** posteriores a la ronda en la que votó el actor, aún no reclamadas por él.

Flujo principal:

1. El actor solicita la reclamación desde su perfil (UC 8) o desde el detalle del artículo (UC 37 propiamente dicho).
2. El sistema invoca `claimRetroactiveReputation(contentHash)`.
3. El actor firma la transacción desde su cartera.
4. El contrato recorre las rondas **DEFINITIVE** posteriores a la del actor, aplica **RETROACTIVE_DELTA** (+1/-1) por cada una según confirmen o contradigan su voto original, limita el resultado a **RETROACTIVE_CAP** y emite **RetroactiveClaimed** con el **netDelta** resultante — pérdidas y ganancias combinadas en una única cifra.

Flujos alternativos:

- Si el actor no votó ese artículo o no existen rondas nuevas que reclamar, la transacción revierte con **NothingToClaim**.

Postcondiciones: la reputación del actor se ajusta en **netDelta**; la reclamación queda registrada en **RetroactiveClaim** y no puede repetirse para las mismas rondas ya procesadas.

Caso de uso: UC 38 — Ver progreso hacia el quórum

Actor(es): Usuario con cartera conectada.

Precondiciones: el artículo está en estado **PENDING**.

Flujo principal: el sistema muestra **roundVoteCount** de la ronda actual frente a **quorumThreshold**, como una barra de progreso o fracción.

Postcondiciones: ninguna escritura.

Caso de uso: UC 39 — Consultar qué validadores ya han votado en la ronda actual

Actor(es): Usuario con cartera conectada.

Precondiciones: el artículo está en estado **PENDING**.

Flujo principal: el sistema invoca `getRoundVoters(contentHash, currentRound)` y muestra el listado de direcciones que ya emitieron su voto en la ronda activa.

Postcondiciones: ninguna escritura; no se revela el sentido del voto de cada validador, solo su participación.

Caso de uso: UC 40 — Ver el efecto reputacional estimado antes de votar

Actor(es): Usuario con cartera conectada.

Precondiciones: el artículo está en estado **PENDING**.

Flujo principal: antes de que el actor confirme su voto en UC 35, el sistema muestra el efecto potencial sobre su reputación según el resultado de la ronda: **+REPUTATION_REWARD** si su opción resulta ganadora, **-REPUTATION_PENALTY** en caso contrario, **0** si la ronda queda **DISPUTED**.

Postcondiciones: ninguna escritura; es información orientativa, no vinculante hasta que se calcula el consenso real.

A.5. FEAT 5. Información institucional

Caso de uso: UC 41 — Consultar landing page

Actor(es): Visitante.

Precondiciones: ninguna.

Flujo principal: el actor accede a la landing page del proyecto; el sistema muestra la propuesta de valor y los tres pilares (Inmutabilidad, Validación colectiva, Resistencia a la captura).

Postcondiciones: ninguna escritura.

Caso de uso: UC 42 — Consultar página “Sobre el proyecto”

Actor(es): Visitante.

Precondiciones: ninguna.

Flujo principal: el actor navega a **/about**; el sistema muestra el funcionamiento del proyecto, la descripción de la DAO y el enlace público a la memoria del TFG.

Postcondiciones: ninguna escritura.

B. Instrumentos de los experimentos de validación

Este anexo recoge el contenido íntegro, ya implementado y en producción, de los dos experimentos de validación descritos en la Sección 3.6.5 (hipótesis y criterios de éxito). Los resultados obtenidos al ejecutarlos se recogen en el Capítulo 5.

■ **Orden de administración** Las dos encuestas se administran en una única sesión, en el orden fijo descrito a continuación: primero la Encuesta 1 (Experimento 1, intensidad del problema), después una página explicativa del funcionamiento de NewsEra y un recorrido interactivo por el prototipo, y finalmente la Encuesta 2 (Experimento 2, idoneidad del producto). Este orden es deliberado: preguntar por la gravedad percibida del problema *después* de mostrar la solución contaminaría la respuesta con la propia propuesta de valor que se le acaba de presentar al participante.

B.1. Encuesta 1 — Experimento 1: intensidad del problema

Se administra antes de cualquier mención a NewsEra. Introducción mostrada al participante: *“Muchas gracias por participar en este estudio. Las respuestas son completamente anónimas y se utilizarán exclusivamente con fines de investigación académica en el marco de un Trabajo Fin de Grado en Ingeniería Informática. Por favor, responde con total sinceridad en base a tu experiencia diaria en internet.”*

Bloque A: tu experiencia con las noticias actuales

Escala Likert de 5 puntos (1 = Totalmente en desacuerdo, 5 = Totalmente de acuerdo):

1. Con frecuencia encuentro en las redes sociales o en los diarios digitales noticias que me parecen dudosas, falsas o exageradas.
2. Confío en que los medios de comunicación tradicionales (tanto públicos como privados) ofrecen la información de forma completamente independiente, sin dejarse influir por partidos políticos o empresas que los financian.
3. Confío en que las grandes plataformas de internet (redes sociales, buscadores) muestran las publicaciones de manera neutral y transparente, sin ocultar o potenciar contenidos según sus propios intereses.
4. En alguna ocasión he compartido, interactuado o dado por buena una noticia en internet que más tarde resultó ser falsa o un bulo manipulado.
5. Cuando dudo de una noticia en internet, me resulta muy difícil, pesado o casi imposible comprobar por mí mismo/a quién la escribió originalmente, si ha sido modificada a escondidas o si las fuentes que cita son reales.

Bloque B: gravedad del problema

6. (**Ítem de contraste directo contra el criterio de gravedad.**) En general, ¿qué nivel de gravedad le otorgas al problema de que la información de actualidad esté controlada por unos pocos grupos de poder o manipulada en las redes sociales? (1 = Nada grave, 5 = Muy grave)
7. (**Ítem de contraste directo contra el criterio de método fiable.**) ¿Conoces o utilizas actualmente algún método o herramienta digital que te permita saber con absoluta certeza si un texto de internet ha sido modificado de forma oculta desde que se publicó? (Sí / No / No estoy seguro/a)

Los ítems 1–5 aportan contexto cualitativo sobre la percepción de captura y manipulación de los canales de información actuales, y permiten detectar inconsistencias en las respuestas; los ítems 6 y 7 son los que se contrastan directamente contra el criterio de éxito fijado en §3.6.5 (al menos el 60 % califica el problema con 4 o 5 en el ítem 6; al menos el 50 % responde “No” o “No estoy seguro/a” en el ítem 7).

B.2. Página explicativa

Entre la Encuesta 1 y la Encuesta 2 se presenta al participante una página que explica el funcionamiento de NewsEra (publicación, validación comunitaria, reputación) y un recorrido interactivo por el prototipo. El contenido de esta página se mantiene deliberadamente descriptivo, no promocional: un tono de venta contaminaría las medidas de confianza percibida y usabilidad de la Encuesta 2, que dejarían de evaluar la interfaz para evaluar la persuasión del texto. Además, siguiendo el principio de **abstracción funcional**, se evita el vocabulario técnico de la arquitectura Web3 (“*blockchain*”, “*smart contract*”, “*wallet*”): las propiedades del sistema se explican por sus beneficios perceptibles (“nadie puede borrar ni modificar lo publicado”, “la veracidad la decide una votación de la comunidad”, “ganas o pierdes puntos según lo bien que acierten tus valoraciones”), para no sesgar ni confundir a una muestra sin conocimientos previos de la tecnología subyacente.

B.3. Encuesta 2 — Experimento 2: idoneidad del producto

Se administra únicamente después de que el participante haya interactuado con el prototipo (listado de artículos, publicación, sistema de votación y reputación de los revisores).

B.3.1. Bloque A: evaluación de la usabilidad (System Usability Scale)

Los diez ítems estándar del SUS (Brooke, 1996), con una única adaptación léxica deliberada —se sustituye el término abstracto “sistema” por “página web” o “aplicación” para facilitar una lectura natural—, sin alterar el orden ni la alternancia entre enunciados positivos y negativos: alterarlos sí invalidaría la comparabilidad de la puntuación con la literatura de referencia. Escala 1-5 (1 = Totalmente en desacuerdo, 5 = Totalmente de acuerdo):

1. Creo que me gustaría utilizar esta página web con frecuencia para informarme.
-

2. He encontrado que la página web es innecesariamente compleja.
3. Me ha parecido que la página web es fácil de usar.
4. Creo que necesitaría la ayuda de una persona experta para poder manejarme en esta página web.
5. Me ha parecido que las distintas funciones de la aplicación (leer, publicar, votar) están muy bien integradas entre sí.
6. He percibido demasiadas contradicciones o cosas incoherentes al navegar por la página web.
7. Imagino que la mayoría de la gente aprendería a utilizar esta página web de forma muy rápida.
8. Navegar y realizar acciones en esta página web me ha resultado incómodo o pesado.
9. Me he sentido muy seguro/a y con el control de la situación al confirmar acciones dentro de la página web.
10. He necesitado aprender o entender demasiadas cosas antes de poder empezar a usar esta página web de forma cómoda.

Cálculo de la puntuación (estándar de Brooke, no debe alterarse): para los ítems impares (1, 3, 5, 7, 9), puntuación parcial = respuesta -1 ; para los ítems pares (2, 4, 6, 8, 10), puntuación parcial = $5 - \text{respuesta}$. La suma de las diez puntuaciones parciales, multiplicada por 2.5, da la puntuación SUS final en una escala de 0 a 100. El criterio de éxito fijado en §3.6.5 es una media ≥ 68 .

B.3.2. Bloque B: confianza en las propiedades de la aplicación

Escala 1-5 salvo donde se indica. Cada ítem aísla, sin nombrar la tecnología subyacente, una propiedad arquitectónica distinta del sistema —de modo que un resultado desfavorable pueda diagnosticarse con precisión (¿es la arquitectura la que no convence, o solo la fricción de la interacción *Web3*?)—:

11. **(Inmutabilidad y descentralización — PublicationRegistry.)** Me genera mucha confianza saber que, una vez que se publica una noticia en esta web, queda registrada de forma que NADIE (ni gobiernos, ni empresas, ni los propios creadores de la web) puede borrarla o modificarla a escondidas.
 12. **(Consenso colectivo — ValidationRegistry.)** Me parece acertado que la veracidad de una noticia se decida mediante una votación transparente de revisores independientes, en lugar de dejar la decisión en manos del director de un medio o de los filtros de una red social.
 13. **(Reputación P2P — ReputationSystem.)** El sistema de “puntos de reputación” (donde un revisor gana puntos si acierta con la comunidad y los pierde si se equivoca de forma continuada) me parece una forma justa y transparente de dar más peso a quienes demuestran un historial honesto.
-

14. **(Fricción de la interacción Web3, aislada del resto de propiedades.)** Tener que confirmar cada voto o publicación con una firma digital (una especie de “llave” personal que demuestra que eres tú) me ha resultado un paso raro o molesto para el uso normal de la web.
15. **(Ítem de contraste directo contra el criterio de intención de uso.)** ¿Utilizarías de forma habitual NewsEra para consultar actualidad verificada si la página web contara con periodistas y publicaciones diarias? (Sí / No / Tal vez)
16. En tus propias palabras, ¿qué es lo que más te aporta o convence de esta página web? ¿Cuál crees que es su mayor dificultad para el público general? (*respuesta abierta, opcional*)

El ítem 15 se contrasta contra el criterio de éxito de §3.6.5 (al menos el 60 % expresa intención de uso). Solo las respuestas “Sí” cuentan como intención de uso positiva en el cálculo del porcentaje; las respuestas “Tal vez” se reportan por separado, sin mezclarlas en el mismo porcentaje. Si el ítem 14 puntúa alto en desacuerdo (fricción percibida) mientras los ítems 11-13 puntúan alto en acuerdo (confianza en las propiedades), el hallazgo indicaría que el problema no está en la arquitectura de NewsEra sino en la fricción de entrada de las tecnologías Web3 actuales —una distinción con implicaciones directas para el trabajo futuro (§6.4).

B.4. Datos piloto sintéticos y su exclusión del análisis

Durante el desarrollo, antes de que el prototipo estuviera terminado, se insertaron 20 respuestas sintéticas por encuesta —perfiles ficticios redactados a mano, con posturas variadas (críticas, escépticas, entusiastas)— con el único propósito de comprobar que el formulario y el cálculo de la puntuación SUS funcionaban correctamente antes de disponer de respuestas reales. Estas filas quedan marcadas de forma explícita en la base de datos y se excluyen sistemáticamente de cualquier resultado reportado en el Capítulo 5; no se han utilizado, ni se utilizarán, como dato de investigación.

B.5. Recomendaciones de muestra

La Encuesta 1 no depende de la interacción con el prototipo y puede difundirse de forma independiente (redes sociales, foros) para ampliar su alcance sin coste adicional; se recomienda un tamaño muestral mínimo de 50 participantes. La Encuesta 2 exige completar el recorrido interactivo por el prototipo; se recomienda un mínimo de 30 participantes. La literatura de usabilidad considera que muestras de 12-15 participantes ya son suficientes para detectar los problemas de usabilidad más graves (Brooke, 1996), pero una muestra mayor da mayor fiabilidad a la media SUS reportada en la memoria.

C. NewsEra explicado para un público no técnico

Los capítulos anteriores describen NewsEra con el rigor técnico que exige una memoria de TFG: contratos inteligentes, estructuras de datos, mecanismos de consenso. Este anexo tiene un propósito distinto y complementario: explicar el proyecto a un lector sin conocimientos previos de *blockchain*, evitando deliberadamente el vocabulario técnico de la arquitectura Web3. No es una simplificación por comodidad, sino una pieza funcional del propio trabajo: es, en esencia, la misma explicación —adaptada a prosa formal— que se muestra a los participantes del Experimento 2 (§B.2) antes de interactuar con el prototipo, y que también presenta la página “Sobre el proyecto” del propio sistema (UC 42). Que ambos textos coincidan es intencionado: la claridad expositiva del prototipo y la de esta memoria deben ser la misma.

C.1. El problema: un punto de control es un punto de fallo

Cuando la verdad de una noticia depende de una redacción, un algoritmo o un gobierno concretos, basta con capturar ese único punto de control para torcerla. No hace falta un villano deliberado: con que ese punto tenga intereses propios —editoriales, comerciales o políticos— ya es suficiente para que deje de ser neutral. La literatura revisada en el Capítulo 2 documenta este fenómeno desde ángulos complementarios: el modelo de propaganda de Herman y Chomsky (1988) describe cómo los intereses comerciales de los medios de comunicación condicionan estructuralmente qué información llega al público, y McChesney (2008) extiende este análisis a la concentración de la propiedad mediática en un número reducido de grupos empresariales. A esto se suma un problema adicional, ya no de quién publica sino de cómo se propaga: el estudio de Vosoughi et al. (2018), que analizó la difusión de más de cien mil historias en Twitter a lo largo de varios años, encontró que las noticias falsas se difunden de forma significativamente más rápida y alcanzan a más personas que las verdaderas —la desinformación no solo puede fabricarse desde un punto de control capturado, sino que además viaja mejor que la verdad una vez publicada.

C.2. La solución: un periódico que no tiene redacción

NewsEra es un periódico comunitario: nadie decide en su nombre qué es noticia ni qué es verdad. Esa decisión se traslada desde una redacción, una plataforma o un gobierno hacia miles de personas corrientes. Cómo se vota, cuándo se declara “verdad” y cómo se reparte la reputación está escrito en un contrato inteligente —código público que se ejecuta igual para todos, sin excepciones ni favores—. Por primera vez, ni siquiera quien creó el sistema puede cambiar las reglas del juego de un día para otro sin que la comunidad entera lo note: los parámetros que gobiernan el consenso quedan fijados de forma permanente en el momento del despliegue (RNF 10, §4.1.3).

C.3. ¿Qué es “verdad” aquí?

No la dicta una autoridad. La vota un jurado. Cuando se publica un artículo, se abre una votación. La comunidad decide, con su propia reputación en juego, si es verdadero, falso, o si sencillamente no hay pruebas suficientes todavía —esta última no es una opción de “abstención” cómoda, sino un veredicto con las mismas consecuencias que cualquier otro—. Hace falta una mayoría de dos tercios —no una simple mitad más uno— para dar el veredicto por definitivo, de modo que una votación reñida no se confunda con un consenso real. El resultado queda anotado para siempre, junto con quién votó qué.

C.4. Los pilares

Tres reglas que no se rompen:

- **Inmutable.** Una vez publicado o votado, nada se borra ni se reescribe.
- **Colectivo.** Ninguna persona ni entidad decide sola qué es verdad.
- **Resistente.** Sin una autoridad central que capturar, no hay un único punto de fallo.

C.5. Lo nuevo: no mejora al árbitro, cambia el juego

Los verificadores de hechos tradicionales añaden una capa de revisión encima de un sistema que ya tiene sus propios intereses. NewsEra no revisa ese sistema: lo sustituye por uno nuevo, donde publicar, votar y ganar reputación siguen exactamente las mismas reglas para quien fundó el proyecto que para la primera persona que se registra hoy.

C.6. La tecnología: un cuaderno que nadie puede tachar

La *blockchain* es, en esencia, un registro compartido por miles de ordenadores en vez de guardado en un único servidor. Cada anotación nueva se enlaza criptográficamente a todas las anteriores, de modo que alterar una implicaría rehacer todo el historial a la vista de toda la red —en la práctica, imposible—. Tres piezas de software conforman ese cuaderno en NewsEra: una que registra qué se ha publicado (**PublicationRegistry**), otra que registra cómo se ha votado (**ValidationRegistry**), y una tercera que lleva la cuenta de quién ha demostrado buen criterio a lo largo del tiempo (**ReputationSystem**). El Capítulo 4 detalla el funcionamiento exacto de cada una.

C.7. En una frase

La verdad no tiene dueño. Por primera vez, miles de personas deciden juntas qué es cierto, sin que nadie —ni siquiera quien lo creó— pueda cambiar las reglas a mitad de partida.

D. Guía de reproducibilidad y despliegue local

Este anexo documenta los pasos necesarios para levantar el prototipo completo de NewsEra en un entorno local, con el objetivo de que el tribunal evaluador —o cualquier lector— pueda verificar de forma directa el funcionamiento del sistema descrito en el Capítulo 4, sin depender de un despliegue público. El repositorio completo está disponible en <https://github.com/jrdjl/NewsEra>.

D.1. Requisitos previos

- **Docker Desktop** (o Docker Engine + Compose v2), para orquestar las tres capas del sistema sin instalar PostgreSQL ni un nodo Ethereum de forma nativa.
- **Node.js 20 LTS**, requerido para ejecutar los comandos de Hardhat, Prisma y los scripts de utilidad desde el sistema anfitrión.
- **GNU Make** y una terminal compatible con *Git Bash* en Windows —el ‘Makefile’ usa sintaxis *shell* POSIX (`rm -rf`, bucles `until/done`) que `cmd.exe` no interpreta correctamente.

D.2. Puesta en marcha en un solo paso

El repositorio incluye un objetivo de `make` que automatiza la secuencia completa —levantar los tres contenedores, desplegar los contratos, sembrar un estado inicial de prueba y sincronizar el *backend*— pensado precisamente para resolver el problema descrito en el §5.6¹:

Terminal

```
1 git clone https://github.com/jrdjl/NewsEra.git
2 cd NewsEra
3 make fresh-start
```

Este único comando:

1. Levanta los cuatro contenedores (*frontend*, *backend*, nodo Hardhat y PostgreSQL) definidos en `docker-compose.yml`.
2. Espera a que el nodo Hardhat esté operativo antes de continuar.
3. Despliega los tres contratos inteligentes (`PublicationRegistry`, `ValidationRegistry`, `ReputationSystem`) mediante Hardhat Ignition.

¹Hardhat Network es una cadena efímera en memoria: pierde todo su estado cada vez que el contenedor se reinicia, por lo que el despliegue de los contratos debe repetirse en cada arranque.

4. Siembra un conjunto de transacciones reales de prueba sobre esos contratos (publicaciones, votos, reputación) para no partir de un sistema vacío.
5. Actualiza las variables de entorno del *backend* con las direcciones de contrato recién desplegadas.
6. Trunca y resincroniza la base de datos *off-chain* para que quede consistente con el estado *on-chain* recién sembrado.

Al finalizar, el sistema queda accesible en:

- **Frontend:** <http://localhost:8080>
- **Backend (API REST):** <http://localhost:3001>
- **Nodo Hardhat (JSON-RPC):** <http://localhost:8545>
- **PostgreSQL:** localhost:5433 (usuario, contraseña y base de datos: newsera)

D.3. Comandos habituales

La Tabla D.1 resume los objetivos de **make** más relevantes para el desarrollo y la verificación del sistema; el listado completo se obtiene con **make help** o consultando el propio **Makefile** del repositorio.

Tabla D.1: Comandos de **make** más habituales.

Comando	Efecto
make up	Levanta los cuatro servicios sin redespargar contratos ni sembrar datos
make down	Detiene todos los servicios
make logs	Sigue los registros de todos los contenedores en tiempo real
make test	Ejecuta la suite de tests de los contratos inteligentes (Hardhat + Chai)
make test-backend	Ejecuta los tests de integración del backend (requiere make postgres y detener el contenedor backend , ver nota)
make test-frontend	Ejecuta los tests del frontend (Vitest)
make coverage	Calcula la cobertura de los contratos inteligentes
make deploy-sepolia	Despliega los contratos en la testnet pública Sepolia
make clean	Detiene los servicios y elimina los volúmenes de datos de PostgreSQL

■ **Nota sobre **make test-backend**** El contenedor del *backend* ejecuta su propio indexador de eventos en tiempo real contra el mismo nodo Hardhat y la misma base de datos que usan los tests de integración. Si ambos se ejecutan a la vez, se produce una condición de carrera (el indexador procesa eventos justo cuando los tests truncan las ta-

blas). Antes de ejecutar **make test-backend** conviene detener el contenedor con **make stop-backend** —la propia suite de tests arranca su instancia del *backend* en proceso, sin necesitar el contenedor activo— y volver a levantarlo al terminar con **make backend**.

D.4. Variables de entorno

Cada carpeta del proyecto (**blockchain/**, **backend/**, **frontend/**) es un proyecto Node.js independiente con su propio **.env**, generado a partir del **.env.example** correspondiente. **make fresh-start** rellena automáticamente las direcciones de contrato en **backend/.env** tras el despliegue; el resto de variables (claves de Sepolia, credenciales de Pinata) deben configurarse manualmente solo si se desea desplegar en la testnet pública, no para el uso local descrito en este anexo. El detalle completo de cada variable se documenta en el **README.md** del repositorio.